



# Modelling, Simulation and Digital Twin

Dr Adele Nasti, SRH Berlin and Rolls-Royce Deutschland Ltd & Co KG

2) Change altitude

RR Engine 10101

# Course outline

# 01

# Course contents

- **Modelling and Simulation**
- **The Engineering Design Process**
- **Mathematical Methods for Engineering**
- **Simulation Tools for Design**
- **Integrated Design Systems**
- **The Digital Twin**

# References

- NASA Systems Engineering Handbook:  
<https://www.nasa.gov/connect/ebooks/nasa-systems-engineering-handbook>
- Advanced Engineering Mathematics, Erwin Kreyszig:  
[https://www.bau.edu.jo/UserPortal/UserProfile/PostsAttach/59003\\_3812\\_1.pdf](https://www.bau.edu.jo/UserPortal/UserProfile/PostsAttach/59003_3812_1.pdf)
- Digital Twin: Definition & Value:  
[https://www.aiaa.org/docs/default-source/uploadedfiles/issues-and-advocacy/policy-papers/digital-twin-institute-position-paper-\(december-2020\).pdf](https://www.aiaa.org/docs/default-source/uploadedfiles/issues-and-advocacy/policy-papers/digital-twin-institute-position-paper-(december-2020).pdf)
- ASME Performance Test Codes 19.1: Test Uncertainty:  
<https://www.asme.org/codes-standards/find-codes-standards/ptc-19-1-test-uncertainty>
- ASME GT2020-15623: Process Automation for Aero Engines Secondary Air System Seal Design, Adele Nasti:  
<https://asmedigitalcollection.asme.org/GT/proceedings-abstract/GT2020/84188/V07CT14A023/1095072>



# What is the Digital Twin?

# 02

# What is the Digital Twin?

Rolls-Royce – Pioneering the Intelligent Engine:

<https://www.youtube.com/watch?v=9CcbYQ5QA70>



# The digital twin

*A hard multi-disciplinary technical problem of great potential across industry*

Siemens

<https://www.youtube.com/watch?v=ObGhB9CCHP8>

Digital twin cities – ASME report

<https://www.youtube.com/watch?v=HftDI09LVI0>

General Electric – Power digital solutions

<https://www.youtube.com/watch?v=5Ji8dZqkECM>

*Every engineering sector has a great interest in making digital twins work end to end*

# The digital twin

The **digital twin** is the digital representation of a physical product throughout its lifecycle: the **virtual product**.

# The digital twin

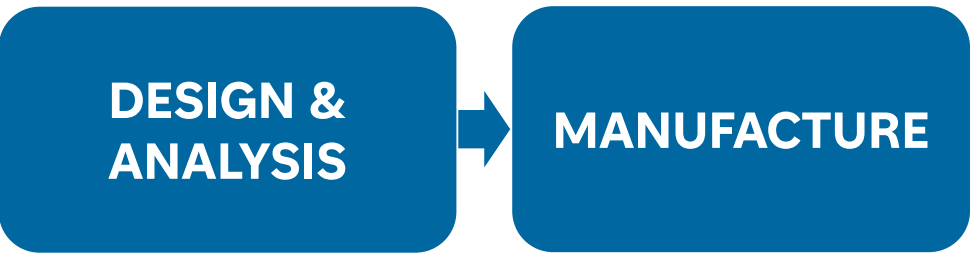
The **digital twin** is the digital representation of a physical product throughout its lifecycle: the **virtual product**.

DESIGN &  
ANALYSIS

*Key steps of an engineering product lifecycle*

# The digital twin

The **digital twin** is the digital representation of a physical product throughout its lifecycle: the **virtual product**.



```
graph LR; A[DESIGN & ANALYSIS] --> B[MANUFACTURE]
```

DESIGN &  
ANALYSIS

MANUFACTURE

*Key steps of an engineering product lifecycle*



# The digital twin

The **digital twin** is the digital representation of a physical product throughout its lifecycle: the **virtual product**.

DESIGN &  
ANALYSIS



MANUFACTURE



BUILD &  
ASSEMBLY

*Key steps of an engineering product lifecycle*

# The digital twin

The **digital twin** is the digital representation of a physical product throughout its lifecycle: the **virtual product**.

DESIGN &  
ANALYSIS



MANUFACTURE



BUILD &  
ASSEMBLY



EXPERIMENTAL  
TESTING

*Key steps of an engineering product lifecycle*

# The digital twin

The **digital twin** is the digital representation of a physical product throughout its lifecycle: the **virtual product**.

DESIGN &  
ANALYSIS



MANUFACTURE



BUILD &  
ASSEMBLY



EXPERIMENTAL  
TESTING



SERVICE

*Key steps of an engineering product lifecycle*

# The digital twin

The **digital twin** is the digital representation of a physical product throughout its lifecycle: the **virtual product**.

DESIGN &  
ANALYSIS

MANUFACTURE

BUILD &  
ASSEMBLY

EXPERIMENTAL  
TESTING

SERVICE

*Key steps of an engineering product lifecycle*

At each step of the product lifecycle, the digital twin offers the opportunity to optimise and de-risk the real product, simulating in the computer the product behaviour for specific operating conditions.

Feeding data from the real product operation into a system-level simulation model, it is possible to generate a different virtual model for each real product (the *digital twin*). This enables optimisation of the key performance attributes of each product and predictions on future operation (e.g. performance degradation)

Given that the cost of simulation is  $\ll$  than the cost of experimental testing, the virtual product allows further de-risking of the real product operation. The digital twin does not replace experimental testing.

# The virtual product and the real product

## THE VIRTUAL PRODUCT

DESIGN &  
ANALYSIS

MANUFACTURE

BUILD &  
ASSEMBLY

EXPERIMENTAL  
TESTING

SERVICE

Product as  
designed in  
engineering  
drawings

Manufacturing  
variations

Assembly &  
build tolerances

Aspects of  
physics not  
captured by  
simulation

Real product  
operating  
conditions

## THE REAL PRODUCT

The product lifecycle introduces multiple variations, it is crucial for the **digital twin** to capture as many **real product attributes** as possible, in order to provide the **most sophisticated digital version of the reality**.

# Simulation at each step of the product lifecycle

## THE VIRTUAL PRODUCT

**DESIGN &  
ANALYSIS**

**MANUFACTURE**

**BUILD &  
ASSEMBLY**

**EXPERIMENTAL  
TESTING**

**SERVICE**

Product as  
designed in  
engineering  
drawings

Manufacturing  
variations

Assembly &  
build tolerances

Aspects of  
physics not  
captured by  
simulation

Real product  
operating  
conditions

Simulate  
nominal (as  
designed)  
product

Simulate  
product as  
manufactured

Simulate  
product as built

Calibrate  
simulation  
models

Feed product  
operation data  
into simulation  
models

# Complex engineering systems

# 03

# Complex engineering systems

## THE VIRTUAL PRODUCT

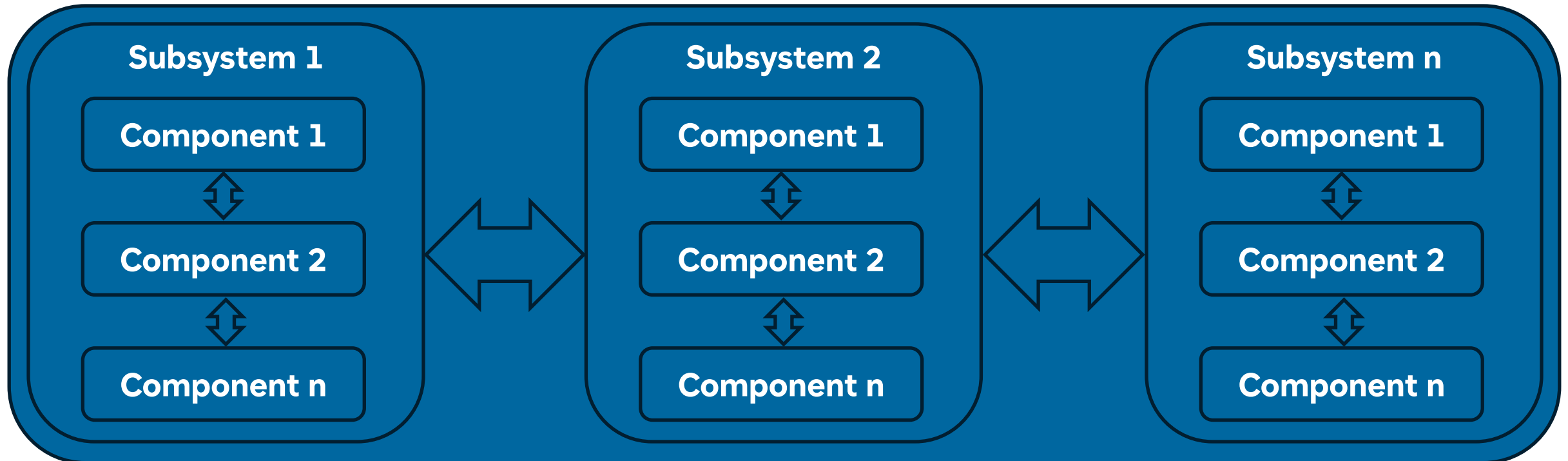
*How do we create a virtual product of a complex engineering system?*



# Complex engineering systems

## THE VIRTUAL PRODUCT

In order to manage its complexity, an engineering product (or **system**) is typically split in a number of **subsystems**, each of them constituted by multiple **components**. All the components and subsystems are interconnected via multiple interactions → **physical** and **functional** interfaces



# Complex engineering systems

## THE VIRTUAL PRODUCT

In order to manage its complexity, an engineering product (or **system**) is typically split in a number of **subsystems**, each of them constituted by multiple **components**. All the components and subsystems are interconnected via multiple interactions → **physical** and **functional** interfaces.

The behaviour all the components and subsystems is governed by the laws of physics. To model an engineering system (or also a single component) it is important to develop **multi-disciplinary simulation models** that capture with a sufficient level of **accuracy** all the aspects of the physics of the system and their **coupling**

- The accuracy or **fidelity**\* of the simulation models can vary depending on the needs (e.g. phase of the product lifecycle, objective of the simulation, time available to perform the analysis)



# Complex engineering systems

## THE VIRTUAL PRODUCT

In order to manage its complexity, an engineering product (or **system**) is typically split in a number of **subsystems**, each of them constituted by multiple **components**. All the components and subsystems are interconnected via multiple interactions → **physical** and **functional** interfaces.

The behaviour all the components and subsystems is governed by the laws of physics. To model an engineering system (or also a single component) it is important to develop **multi-disciplinary simulation models** that capture with a sufficient level of **accuracy** all the aspects of the physics of the system and their **coupling**

- The accuracy or **fidelity** of the simulation models can vary depending on the needs (e.g. phase of the product lifecycle, objective of the simulation, time available to perform the analysis)

**The core of the digital twin is a fully coupled multi-disciplinary and multi-fidelity parametric simulation model of the system and its interacting subsystems and components**

# Parametric geometry model

## THE VIRTUAL PRODUCT

DESIGN &  
ANALYSIS



MANUFACTURE



BUILD &  
ASSEMBLY



EXPERIMENTAL  
TESTING



SERVICE

The design of an engineering product is typically captured in a **geometry model** (e.g. CAD model)

A **parametric** model allows simulating the product throughout its lifecycle (e.g. as designed, as manufactured)

The level of detail captured in a geometry model (or **fidelity\***) depends on the purpose of the model

**Idealisation** of the geometry model is usually needed to generate a simulation model

**The core of the digital twin is a fully coupled multi-disciplinary and multi-fidelity parametric simulation model of the system and its interacting subsystems and components**

# The engineering design process

# 04

# The engineering design process

## THE VIRTUAL PRODUCT

**DESIGN &  
ANALYSIS**



**MANUFACTURE**



**BUILD &  
ASSEMBLY**



**EXPERIMENTAL  
TESTING**



**SERVICE**

# The engineering design process

## THE VIRTUAL PRODUCT

DESIGN &  
ANALYSIS

MANUFACTURE

BUILD &  
ASSEMBLY

EXPERIMENTAL  
TESTING

SERVICE

idea

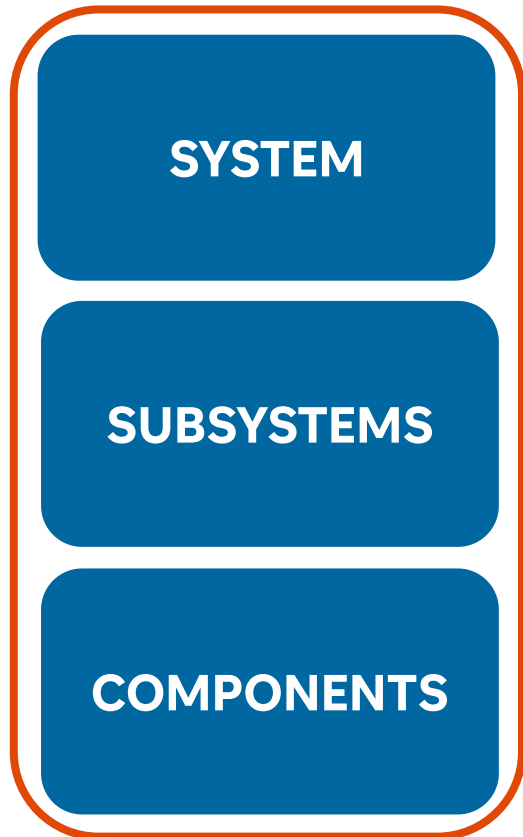
CONCEPTUAL  
DESIGN

PRELIMINARY  
DESIGN

DETAILED  
DESIGN

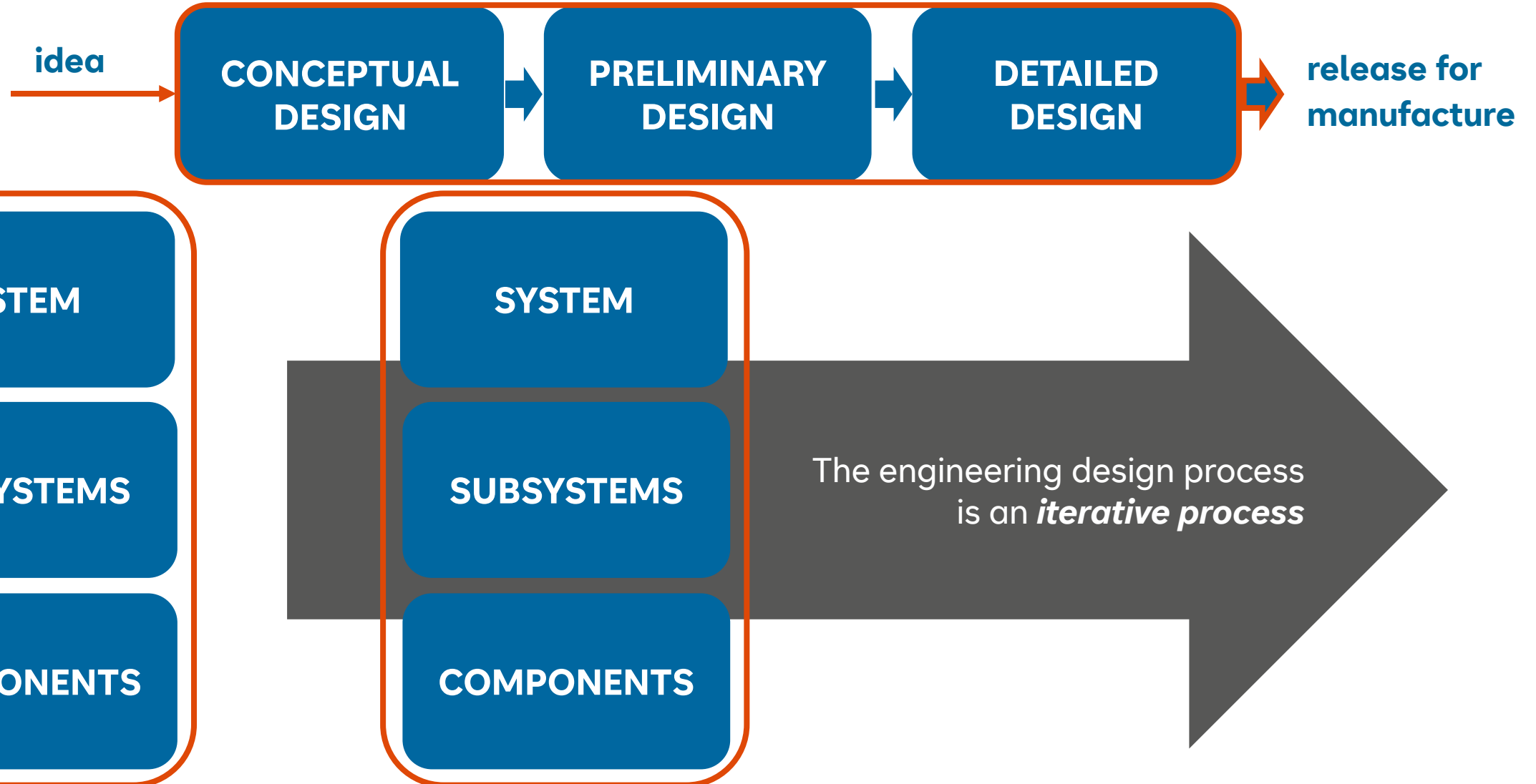
release for  
manufacture

# The engineering design process

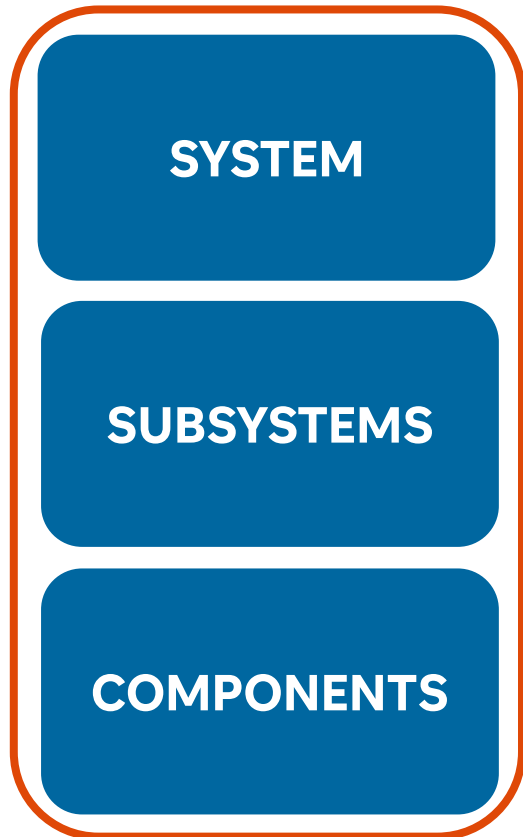




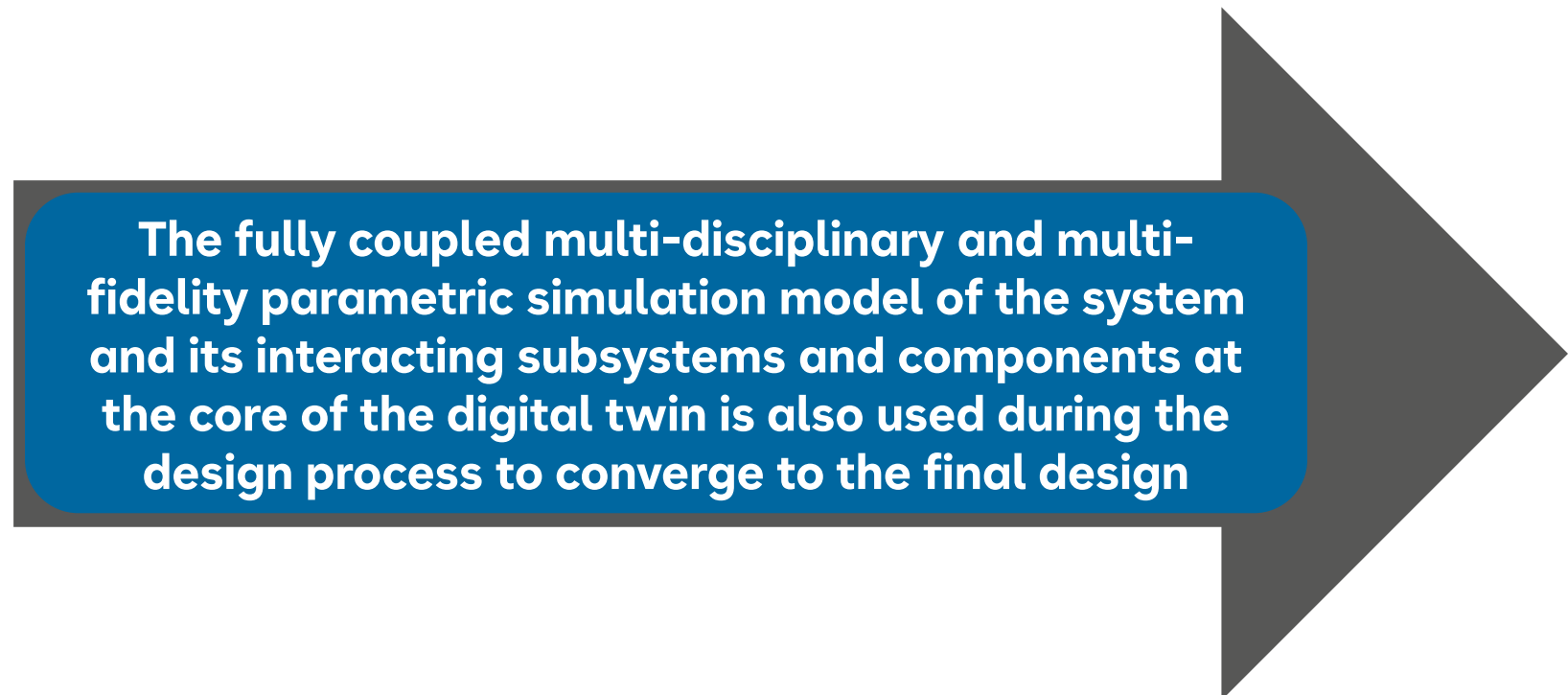
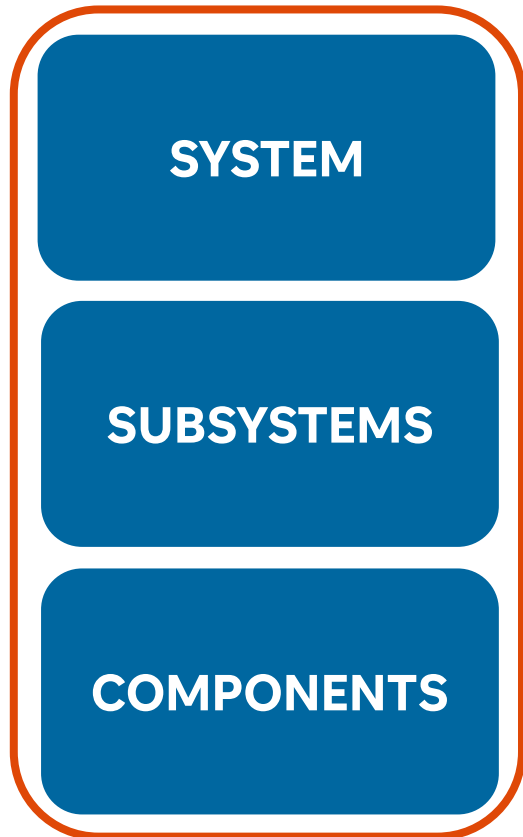
# The engineering design process



# The engineering design process



# The engineering design process



# Modelling and simulation of engineering products

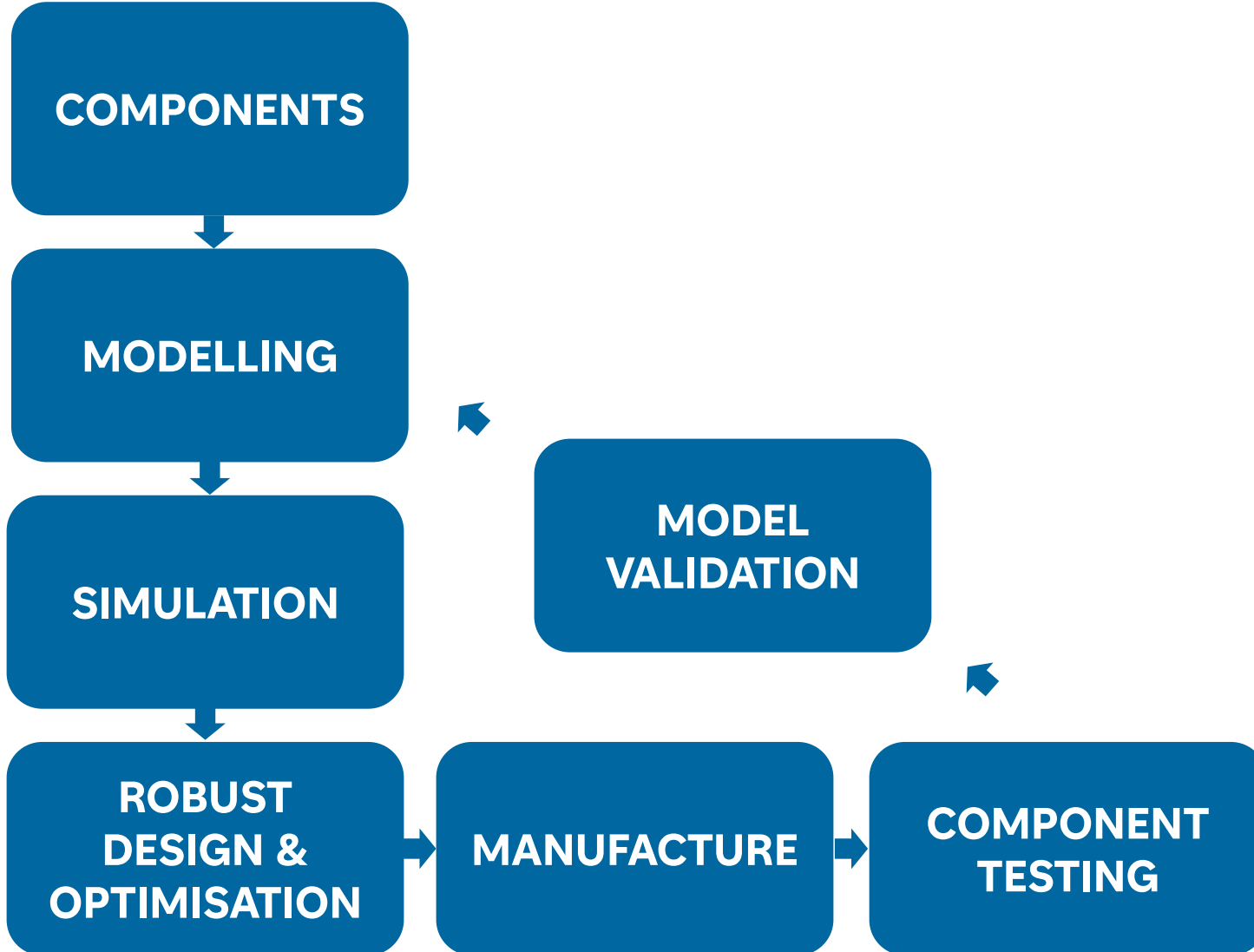
# 05

# Modelling & simulation of engineering products

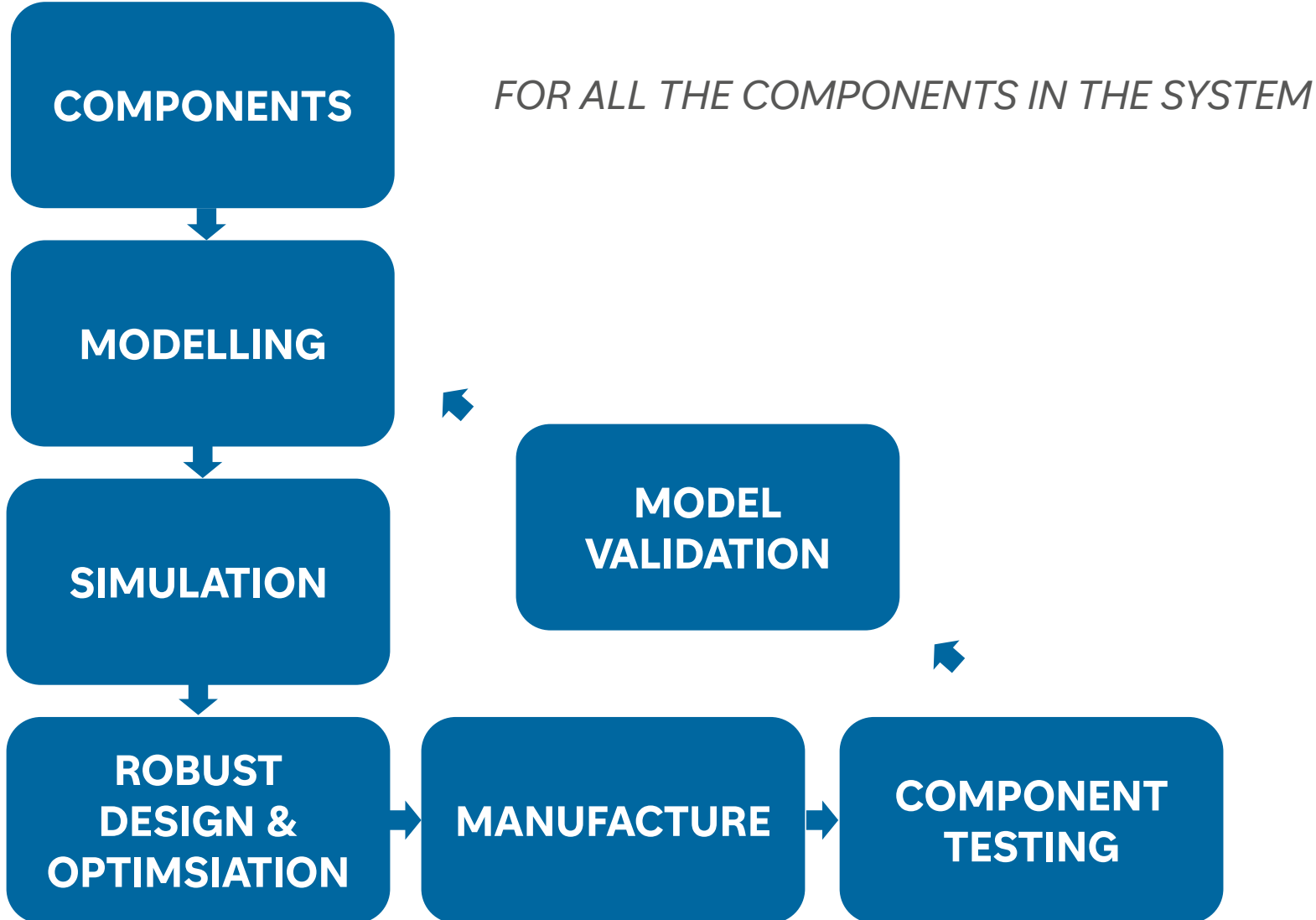
The core of the digital twin is a fully coupled multi-disciplinary and multi-fidelity parametric simulation model of the system and its interacting subsystems and components

*How do we build and validate a system-level simulation model?*

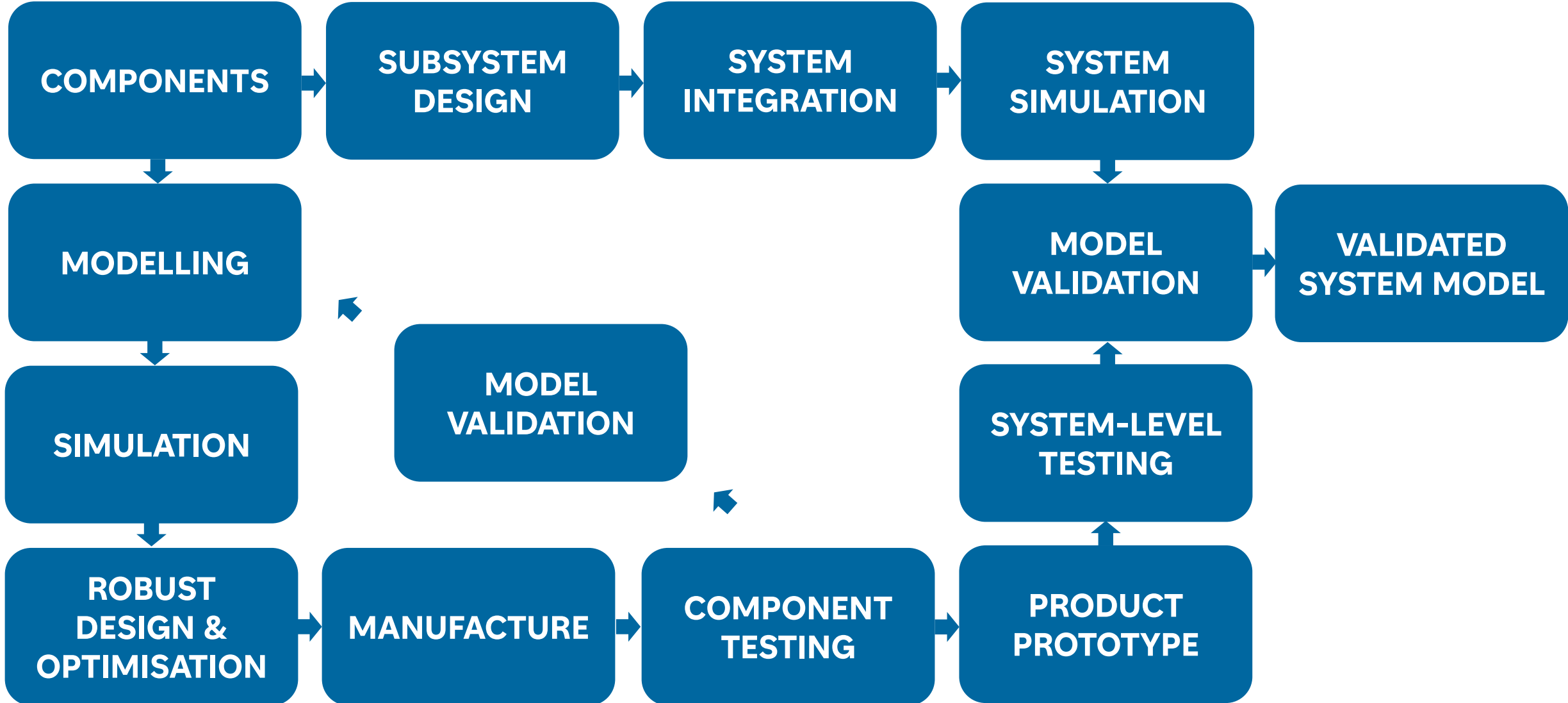
# Modelling & simulation of engineering products



# Modelling & simulation of engineering products



# Modelling & simulation of engineering products





# Technology Readiness Levels

[https://www.nasa.gov/directorates/heo/scan/engineering/technology/technology\\_readiness\\_level](https://www.nasa.gov/directorates/heo/scan/engineering/technology/technology_readiness_level)

[https://web.archive.org/web/20171011071816/http://www.innovation.cc/discussion-papers/22\\_2\\_3\\_heder\\_nasa-to-eu-trl-scale.pdf](https://web.archive.org/web/20171011071816/http://www.innovation.cc/discussion-papers/22_2_3_heder_nasa-to-eu-trl-scale.pdf)

## TRL 9

- Actual system “flight proven” through successful mission operations

## TRL 8

- Actual system completed and “flight qualified” through test and demonstration (ground or space)

## TRL 7

- System prototype demonstration in a space environment

## TRL 6

- System/subsystem model or prototype demonstration in a relevant environment (ground or space)

## TRL 5

- Component and/or breadboard validation in relevant environment

## TRL 4

- Component and/or breadboard validation in laboratory environment

## TRL 3

- Analytical and experimental critical function and/or characteristic proof-of-concept

## TRL 2

- Technology concept and/or application formulated

## TRL 1

- Basic principles observed and reported

# Examples of modelling & simulation in engineering

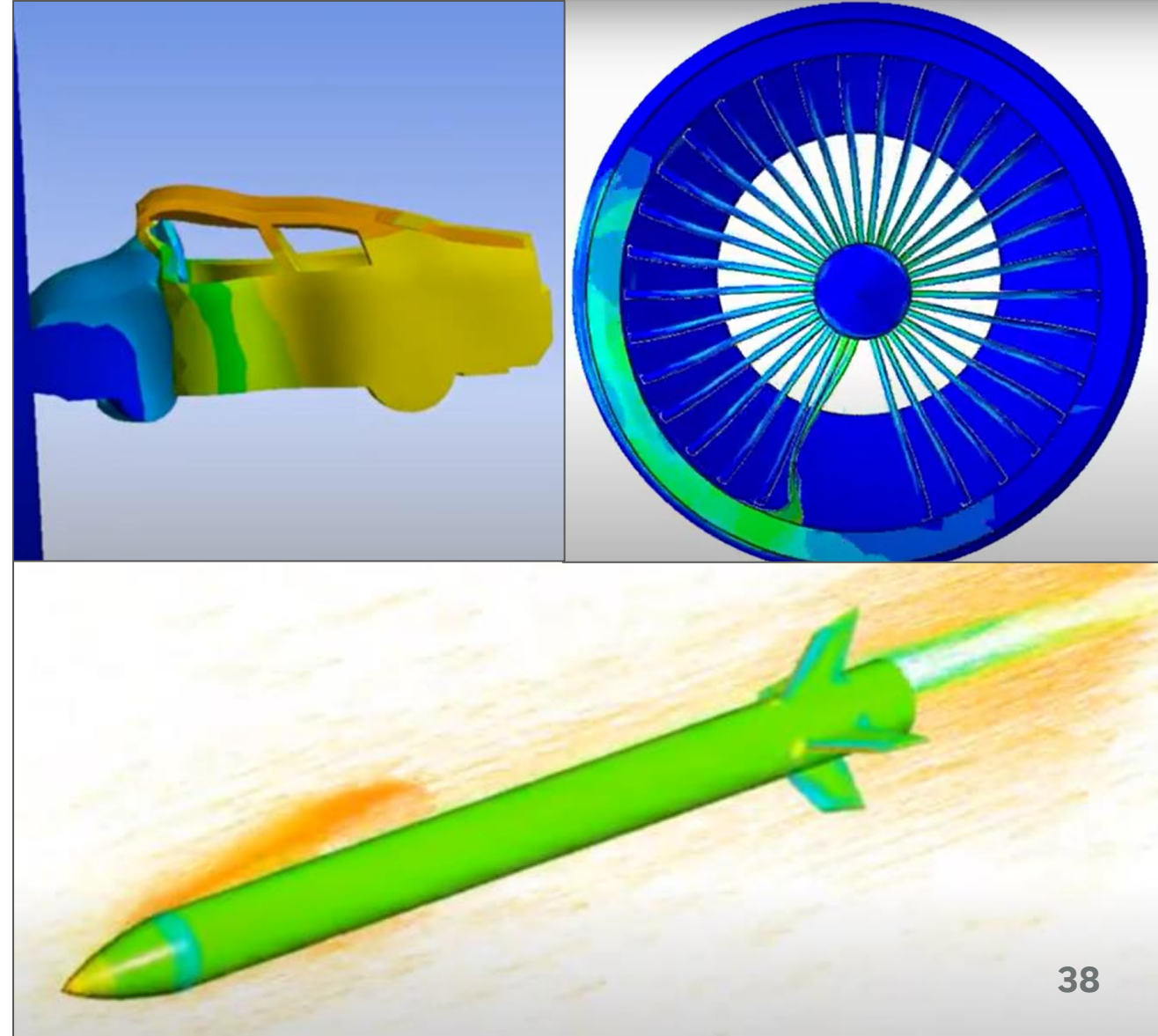
Car crash simulation with Ansys:

Aero engine fan blade off simulation with Abaqus:

<https://www.youtube.com/watch?v=8u6dYTuBymA>

Rocket computational fluid dynamics simulation:

<https://www.youtube.com/watch?v=64N2BY747Cw>



# Examples of modelling & simulation in engineering

Building ventilation CFD simulation:

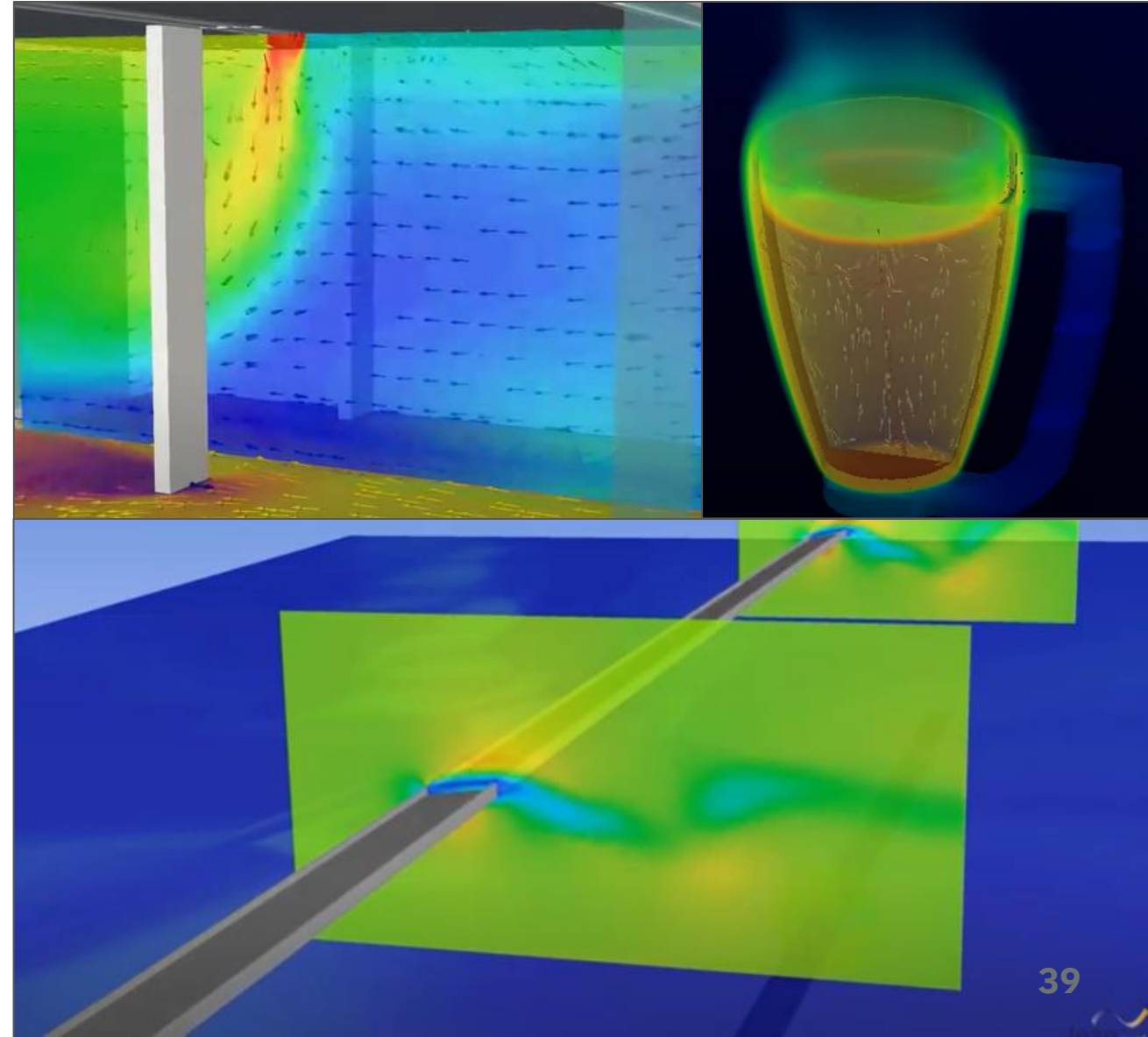
[https://www.youtube.com/watch?v=z\\_g-ov61DNw](https://www.youtube.com/watch?v=z_g-ov61DNw)

Cup of tea heat transfer simulation:

<https://www.youtube.com/watch?v=cHThndE20oI>

Bridge aeroelastic flutter simulation:

[https://www.youtube.com/watch?v=pcLg6C\\_WlHg](https://www.youtube.com/watch?v=pcLg6C_WlHg)



# Principles of mathematical modelling

# 06

# Physics-based modelling vs empirical modelling

There are two fundamental approaches to **mathematical modelling in engineering**, being that for a component or for a complex system: physics-based modelling and empirical modelling.

**Physics-based modelling** exploits our understanding of the laws of physics that govern the world, and in particular the system being modelled, to define the mathematical relationships that describe the behaviour of the system starting from first principles.

**Empirical modelling** starts instead from observations from the real world, for example from sets of experimental data, to derive an understanding of the behaviour of the system. Multiple statistical methods exist to extract information from raw data and process it.

Data-driven approaches can be more suitable for data-rich problems (e.g. machine learning applications).

Physics-based modelling can be more suitable when the physics of the system is simple or fully understood.

In most engineering applications, both physics principles and experimental data are important to develop a good model. Also for physics-based models, a crucial step in the process is the **experimental validation** of the model against test results. Remember that a model is mainly a *representation of the reality*, and as such it is possible that some aspects of the physics have been missed in the model or are unknown (*calibration*).

# Model validation and model calibration

A crucial step in the process of developing of a good physics-based model for an engineering system (or component) is the **validation** of the model against **experimental test results**.

**Model validation** is the process of comparing the predictions from a model with results of a series of experiments. The experiments are carefully designed to acquire as much information as possible on the behaviour of the engineering system, both for nominal operating conditions and for extreme events.

**Model calibration** is the process of tuning some of the model parameters on the basis of the experimental results, in such a way to match as much as possible the model predictions to the observed data. Model calibration is a mechanism to incorporate in the model aspects of the physics that had been missed in the model but are present in the real data.

Experimental data always come with a **measurement uncertainty**, related to the uncertainty of the measurement devices used in the experimental test facility and to the propagation of this uncertainty throughout the test setup. To post process and use test results it is crucial to understand the test uncertainty.

- ASME (The American Society of Mechanical Engineers) Performance Test Codes 19.1: **Test Uncertainty**  
Reference: <https://www.asme.org/codes-standards/find-codes-standards/ptc-19-1-test-uncertainty>

# Verification vs validation

## **Verification: *Have I done the maths right?***

Verification, usually referred to software development, is one of the crucial steps in the development of simulation software. It consists in thoroughly checking (*verifying*) the outputs of the software against a set of expected results, calculated on the basis of the algorithm implemented. It is the process that verifies that the algorithms that translate the mathematical models into simulation have been implemented correctly and hence it guarantees the quality of the software.

## **Validation: *Have I done the right maths?***

Validation, usually referred to models but also applicable to software that implements simulation models, is the process to check (*validate*) the predictions of a model or the outputs of a software against a set of expected results, based on an understanding of the real-world behaviour (the most common validation is performed against experimental test results). It is the process that checks whether the model and model assumptions are suitable to represent reality with an acceptable (potentially subjective) level of accuracy.



# Models as input-output relationships

The modelling of an engineering system (or a component), being that based on physics principles or empirical, is typically characterised by a series of input-output relationships between dependent variables and independent variables. The behaviour of the system under specific conditions is typically captured via a series of product attributes. So, *what are these inputs and outputs?*



# Models as input-output relationships

The modelling of an engineering system (or a component), being that based on physics principles or empirical, is typically characterised by a series of input-output relationships between dependent variables and independent variables. The behaviour of the system under specific conditions is typically captured via a series of product attributes. So, *what are these inputs and outputs?*

Typical **inputs** can be:

- Geometric parameters that describe the design of the system (e.g. dimensions on a component drawing)
- Operating conditions or constraints in which the system operates (e.g. ambient temperature)

Typical **outputs** can be:

- Simple product attributes that can be measured in experiments (e.g. the metal temperature of a component)
- Derived attributes that require experiments plus further post processing to be estimated (e.g. efficiency)

# Regression analysis and curve fitting

**Regression analysis** is a powerful set of mathematical techniques to estimate the relationship between a dependent variable and one or more independent variables in a set of data. It is used to interpret experimental data, understand the relationship between the variables and make predictions for unknown data points.

For example, **curve fitting** allows approximating the relationship between the variables in a dataset, e.g.  $(x_1, y_1), (x_2, y_2), \dots (x_n, y_n)$ , with a mathematical function, e.g.  $y = f(x)$ , or *curve*.

The mathematical function can have different levels of complexity, depending on the data

- The simplest is for example a line (first degree polynomial)  $y = ax + b$  (exact fit for 2 points)
- Second degree polynomial:  $y = ax^2 + bx + c$  (exact fit for 3 points)
- For more than  $n + 1$  points (constraints), where  $n$  is the degree of the polynomial, a method is needed to estimate the accuracy of the fit. For example, the **least square method** minimises the sum of the square of the residuals  $\sum_{i=1}^n (y_i - f(x_i))^2$  where the residual is the delta between the value of the independent variable as predicted by the mathematical function  $f(x_i)$  and the real value from the dataset  $y_i$ .

*Extrapolation* allows predictions beyond the range used to create the function, with a certain *uncertainty*.

# Multi-disciplinary modelling: coupling

# 07

# Multi-disciplinary models: coupling

The core of the digital twin is a fully coupled multi-disciplinary and multi-fidelity parametric simulation model of the system and its interacting subsystems and components

The behaviour all the components and subsystems is governed by the laws of physics. To model an engineering system (or also a single component) it is important to develop **multi-disciplinary simulation models** that capture with a sufficient level of **accuracy** all the aspects of the physics of the system and their **coupling**

- The accuracy or **fidelity**\* of the simulation models can vary depending on the needs (e.g. phase of the product lifecycle **or of the design process**, objective of the simulation, time available to perform the analysis)



# Multi-disciplinary models: coupling

The core of the digital twin is a fully coupled multi-disciplinary and multi-fidelity parametric simulation model of the system and its interacting subsystems and components

The behaviour all the components and subsystems is governed by the laws of physics. To model an engineering system (or also a single component) it is important to develop **multi-disciplinary simulation models** that capture with a sufficient level of **accuracy** all the aspects of the physics of the system and their **coupling**

- The accuracy or **fidelity**\* of the simulation models can vary depending on the needs (e.g. phase of the product lifecycle **or of the design process**, objective of the simulation, time available to perform the analysis)
- Different aspects of the physics can influence each other (*outputs of one analysis can affect the inputs of another one*) → in order to get a **holistic view** of the system it is crucial to **couple** the associated disciplines



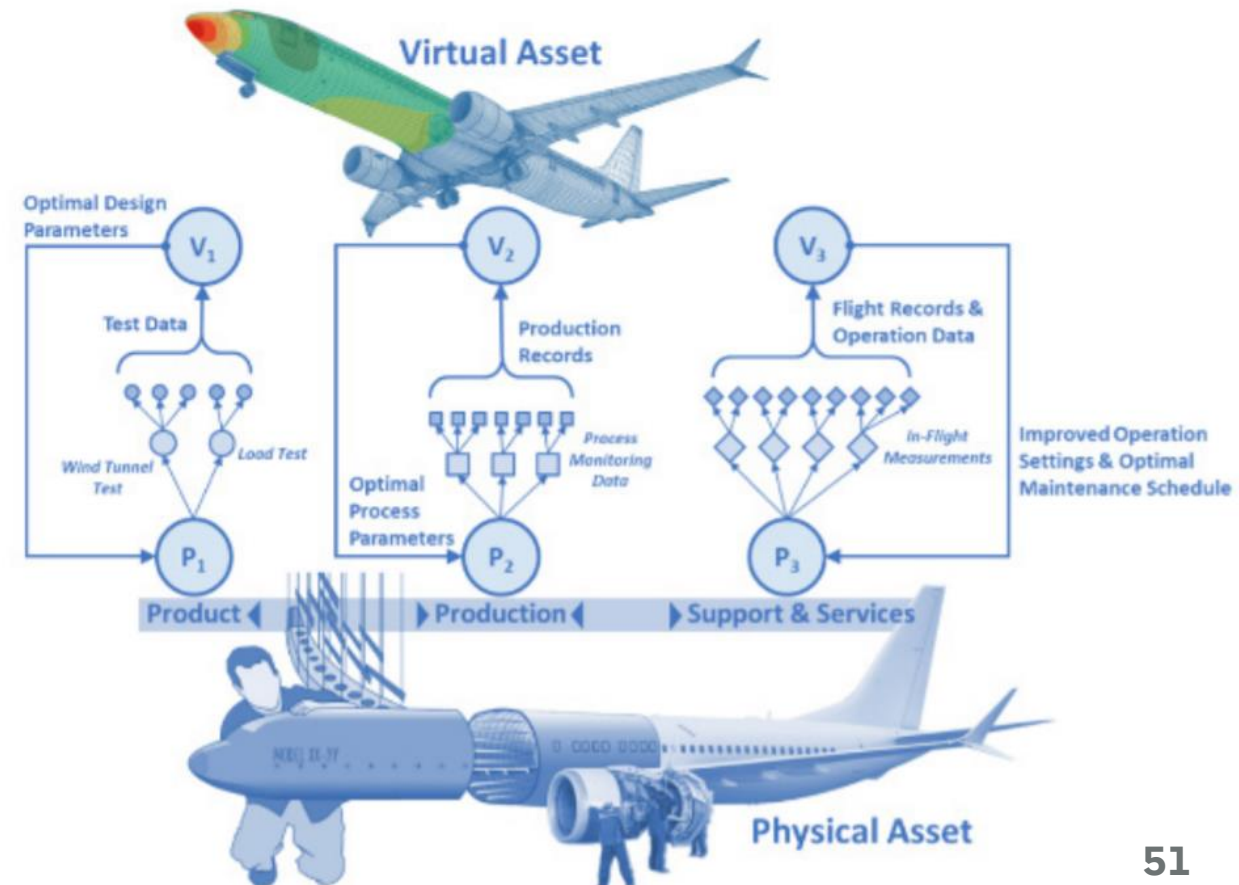
# Discussion

## Discussion - Exercise 2

### AIAA white paper Digital Twin: Definition and Value

[https://www.aiaa.org/docs/default-source/uploadedfiles/issues-and-advocacy/policy-papers/digital-twin-institute-position-paper-\(december-2020\).pdf](https://www.aiaa.org/docs/default-source/uploadedfiles/issues-and-advocacy/policy-papers/digital-twin-institute-position-paper-(december-2020).pdf)

- What is a use of the digital twin that could have value for a company?
- What are examples of products that could have a digital twin?
- Ideas for the final exam



## Discussion - Exercise 2

### AIAA white paper **Digital Twin: Definition and Value**

[https://www.aiaa.org/docs/default-source/uploadedfiles/issues-and-advocacy/policy-papers/digital-twin-institute-position-paper-\(december-2020\).pdf](https://www.aiaa.org/docs/default-source/uploadedfiles/issues-and-advocacy/policy-papers/digital-twin-institute-position-paper-(december-2020).pdf)

### Digital Twin applications and Value Examples

#### **As designed**

- E.g. Digital Twin Calibration, Performance Monitoring, Design Optimisation

#### **As built**

- E.g. As built configuration, Factory Simulation, Material & Process Modelling

#### **As used**

- E.g. Health monitoring, Performance optimisation, Failure (root cause) analysis, Condition-based maintenance, End of life decisions, Anomaly detection



# Example of coupling in aero engine seals

- ASME GT2020-15623: Process Automation for Aero Engines Secondary Air System Seal Design, Dr Adele Nasti, Rolls-Royce Deutschland Ltd & Co KG, presented at ASME Turbo Expo 2020:  
<https://asmedigitalcollection.asme.org/GT/proceedings-abstract/GT2020/84188/V07CT14A023/1095072>

# 08

© 2021 Rolls-Royce plc

The information in this document is the property of Rolls-Royce plc and may not be copied or communicated to a third party, or used for any purpose other than that for which it is supplied without the express written consent of Rolls-Royce plc.

This information is given in good faith based upon the latest information available to Rolls-Royce plc, no warranty or representation is given concerning such information, which must not be taken as establishing any contractual or other commitment binding upon Rolls-Royce plc or any of its subsidiary or associated companies.

TotalCare®, SelectCare®, Life®, Term®, Flex®, Trent® and UltraFan® are registered trade marks of Rolls-Royce plc.

Non-confidential | Not subject to Export Control | ©Rolls-Royce

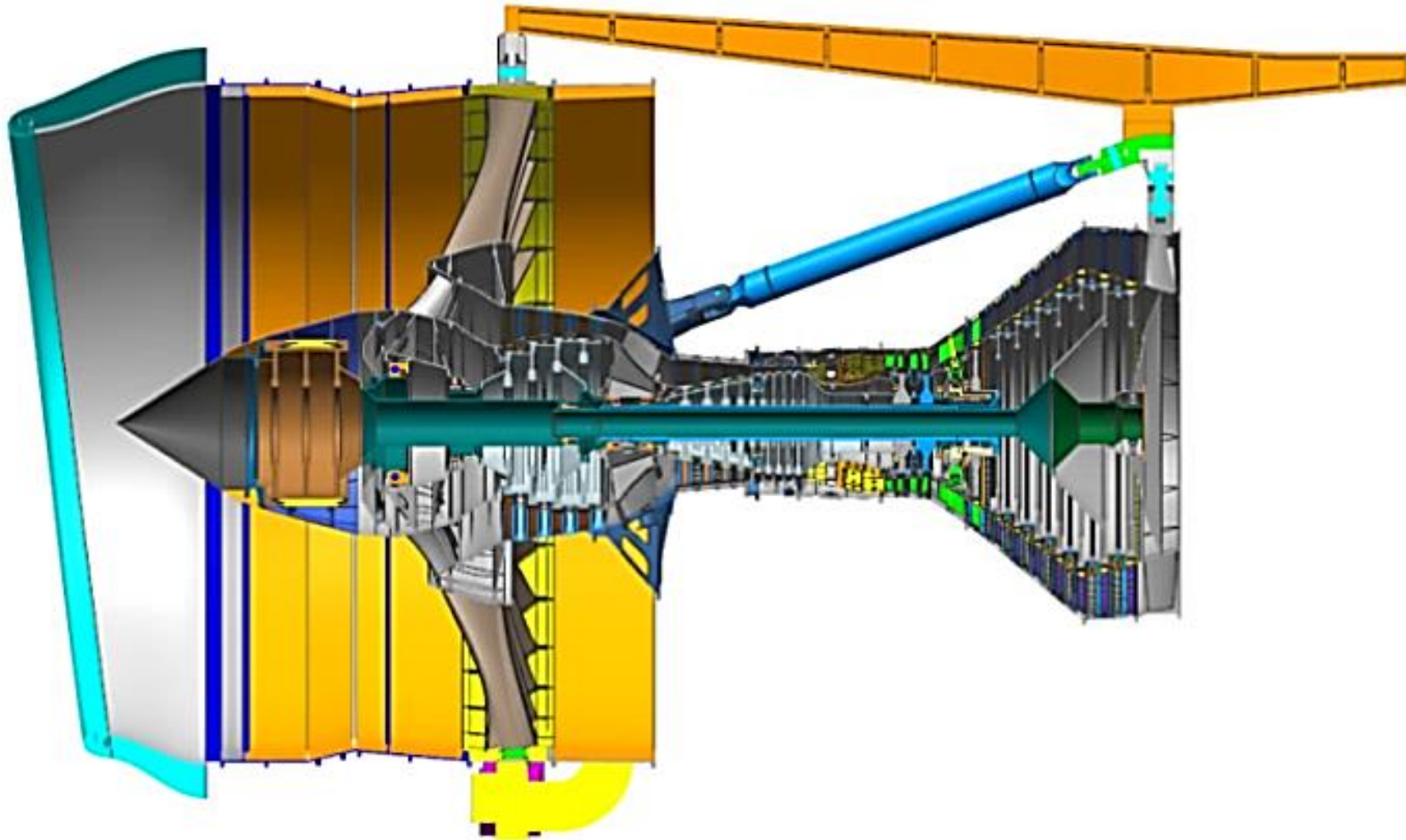
# Secondary air system seals in aero engine design

- Secondary air system seals are crucial in aero engine design as they have a direct impact on specific fuel consumption
- They sit at the intersection between all the major aspects of the physics of the engine, hence their performance can significantly impact other systems
- Their behaviour is affected by the air system, the engine thermal physics, the effect of flight loads and several other effects
- As a consequence, their design is a complex and iterative process, highly dependent on the specific engine location, the operating conditions and the seal supporting structure



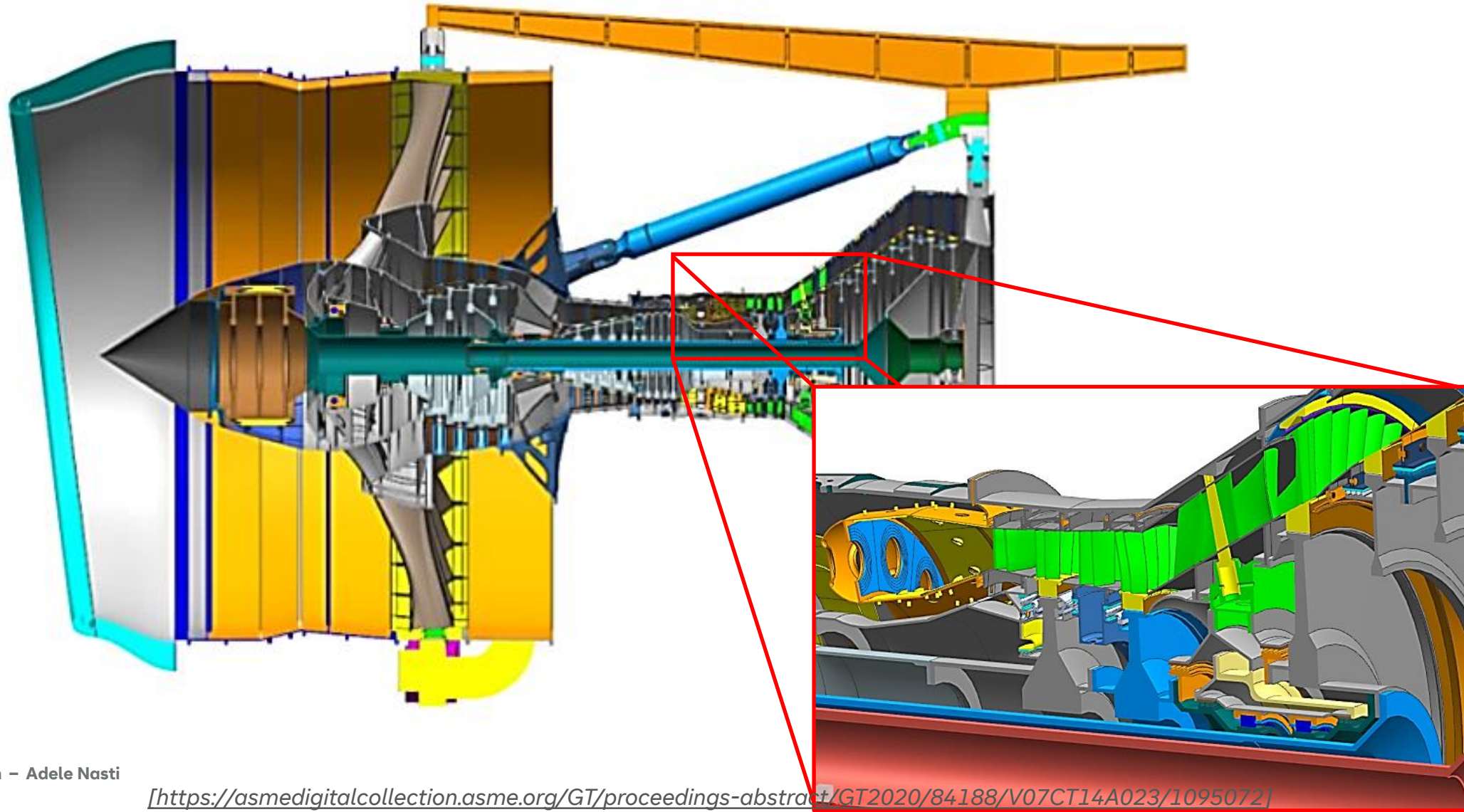
Airbus A350-900 with Rolls-Royce XWB-84k

# Aero engine assembly

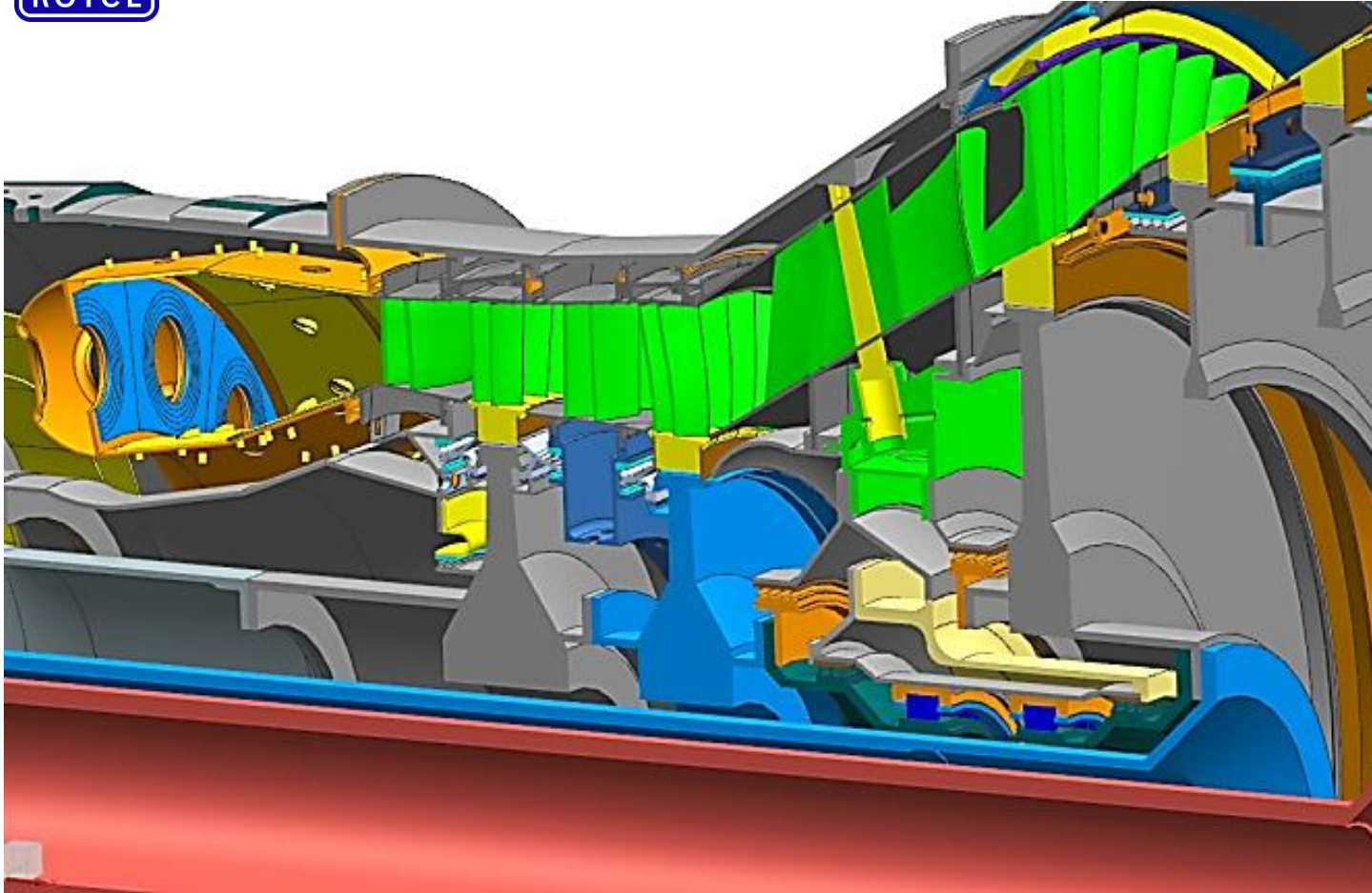




# Aero engine assembly



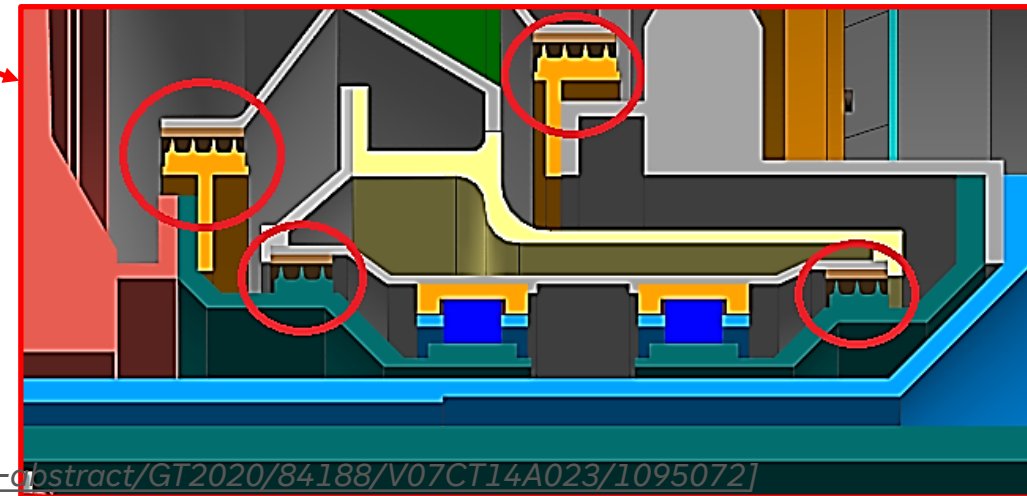
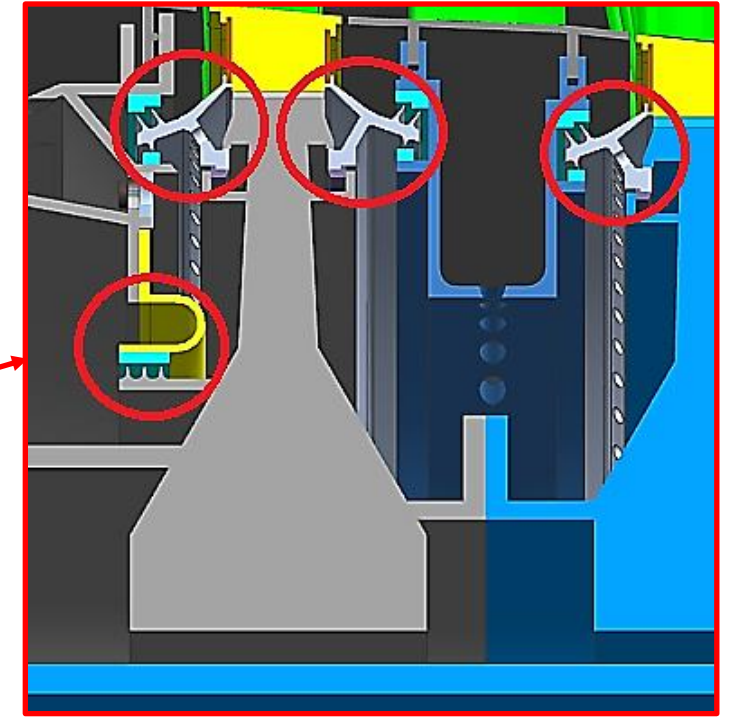
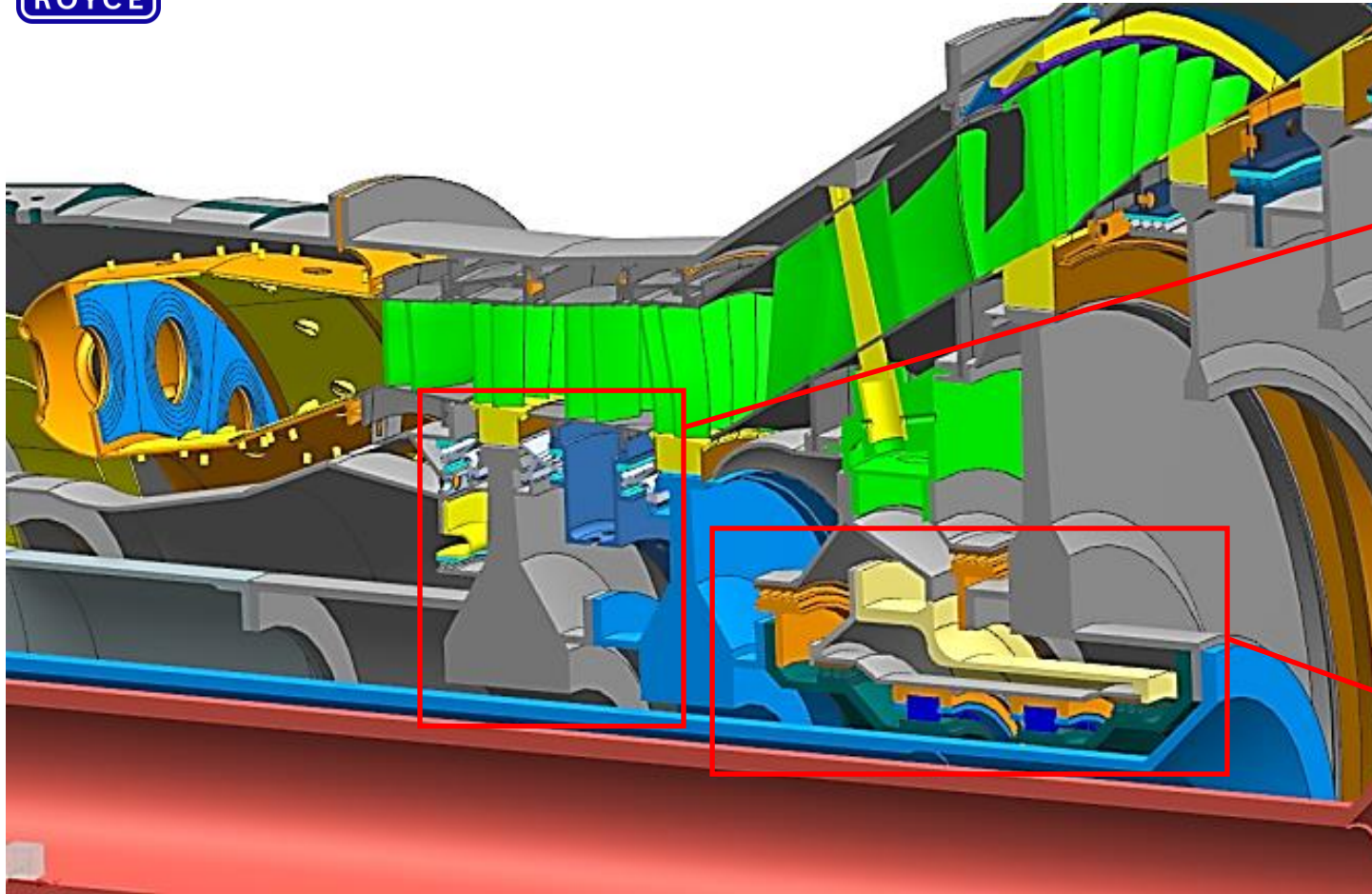
# Seals in aero engines







# Seals in aero engines



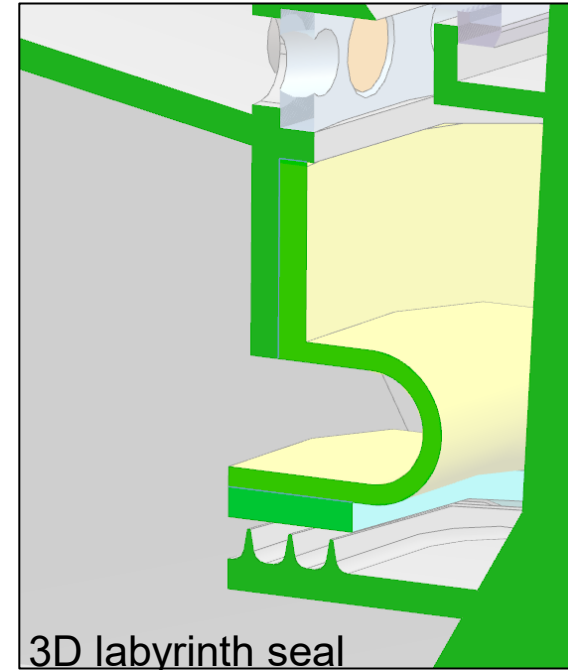
Different size, operating conditions, structures and features

# Requirements on seal functionality

A **system** would typically have a **set of requirements** onto each **component**

For a **secondary air system seal**:

- Maintain effective flow area as constant as possible
- Optimum performance at specific operating conditions
- Tolerant to large incursions and extreme events
- Performance retention throughout seal life
- Robustness to sources of variations
  - e.g., manufacturing tolerances or uncertainties in functional inputs

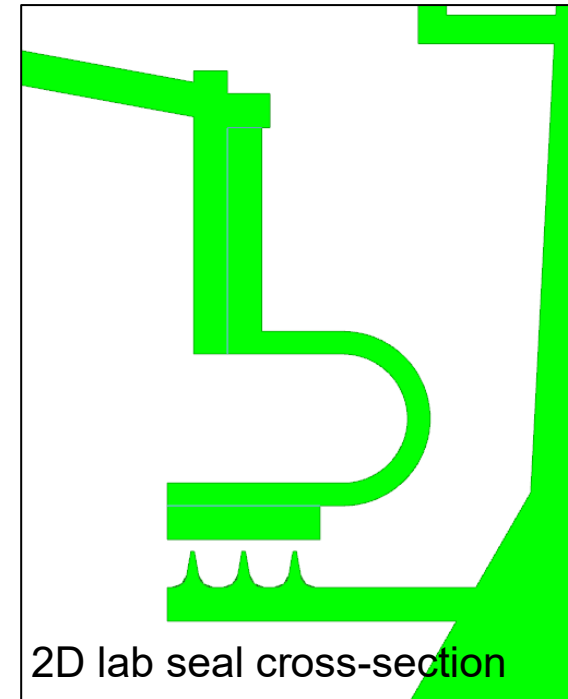
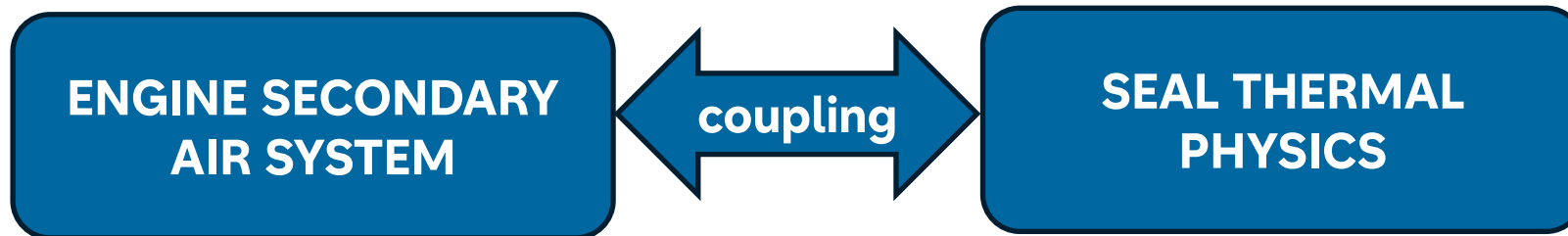


3D labyrinth seal

# Multi-disciplinary coupling in engine seals

The **engine secondary air system** and the **seal thermal physics** are **closely coupled** because the engine thermo-mechanical displacements affect the input operating conditions (*for example the pressures upstream and downstream of the seal*), which in turn affect the engine thermo-mechanical displacements (*being for example the pressure one of the boundary conditions of the thermal model*)

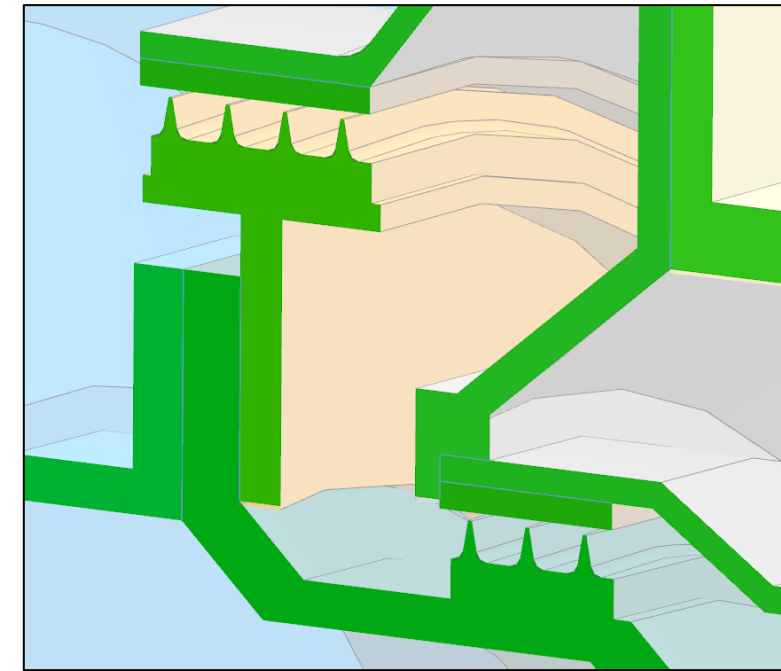
When multi-disciplinary coupling is present in a model, an **iterative process** is often used to converge to the solution that takes into account both aspects of the physics and their interaction



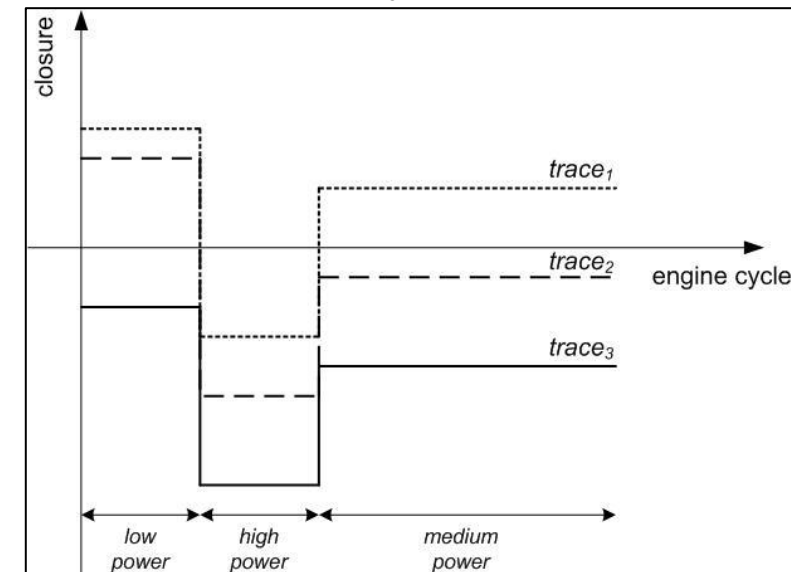


# Further coupling between system & seal

- The operation of one seal can affect other seals
- Seals should be designed for multiple conditions
- Closure traces over engine cycles: rotor to static structure relative displacements
  - closure =  $\Delta s_{\text{stator}} - \Delta s_{\text{rotor}} = \Delta s - \Delta r$
  - if rotor inside stator, negative closure: *closing* system
  - if rotor inside stator, positive closure: *opening* system
- Effect of a rub due to stator-rotor incursion on subsequent seal performance and on seal life

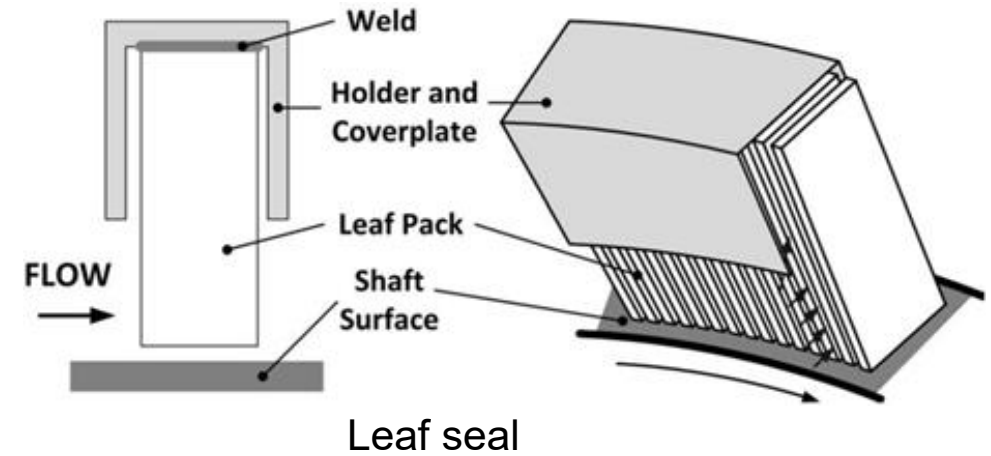


Two-seal assembly

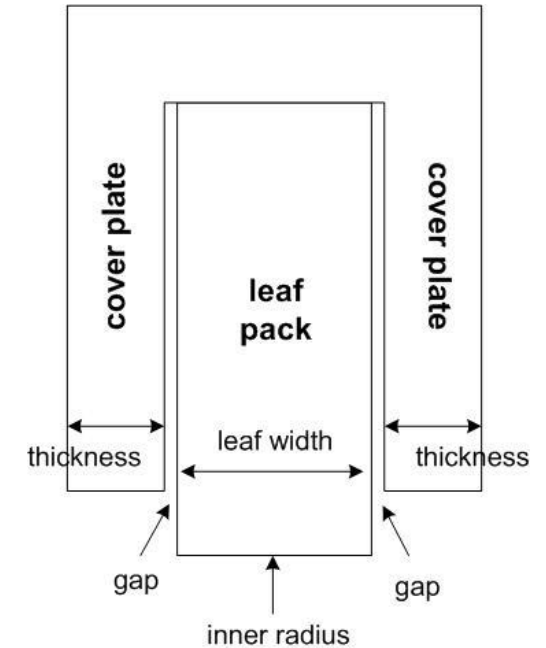


Steady-state closure traces over engine cycle

# Compliant contacting seals



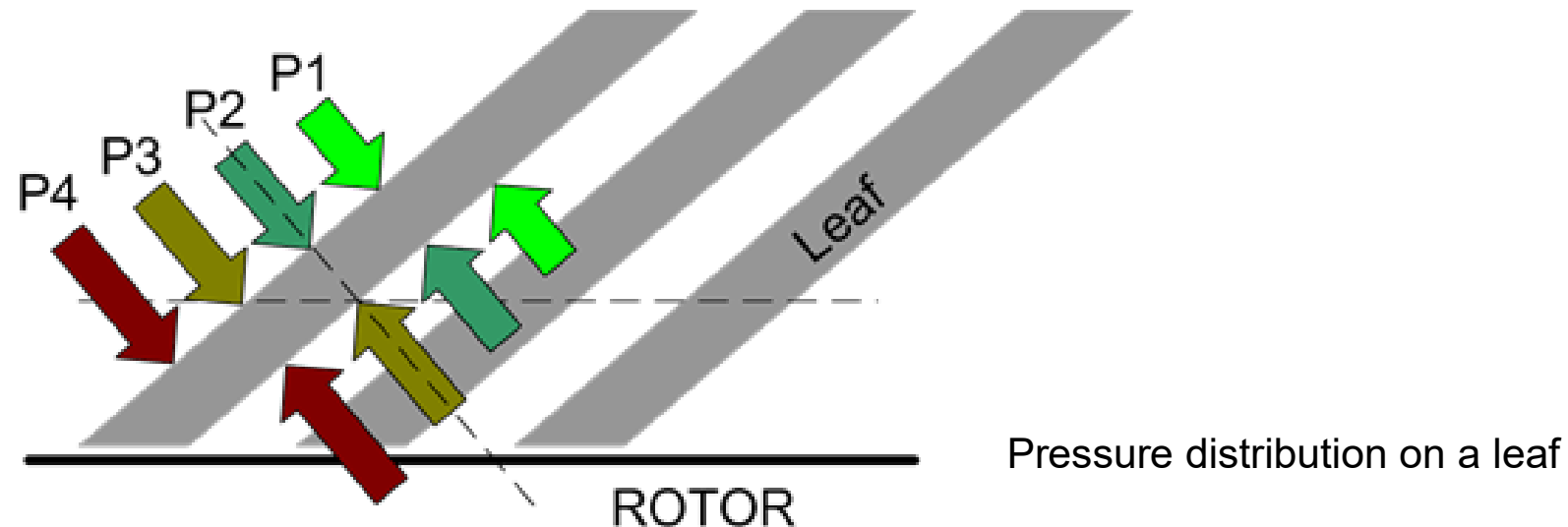
- Better performance and performance retention
  - Lower leakage at key operating conditions
  - More uniform effective flow area
  - Compliance to relative movements, including eccentricity, minimizes the effect of worst-case incursions (tip wear versus labyrinth seal rub)
- Blow-down and/or lift-up (negative blow-down) mechanism
- Frictional heat generated at the interface with the rotor



Key geometric parameters

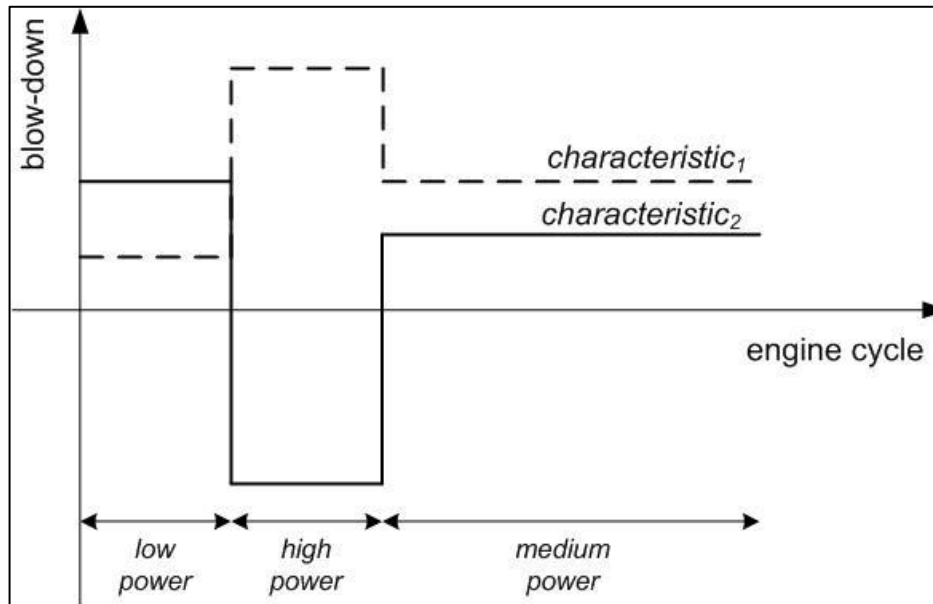
# Blowdown and liftup mechanism

- It has been proven that these seals are characterized by a blow-down or lift-up mechanism which is caused by the combination of a pressure force and an inertia force
- Due to the effect of the mass flow through the seal and to the local seal geometry – leaves with a certain thickness at an angle – the pressure field exhibits a mismatch between the sides of a leaf, which causes the pressure force
- The blow-down effect has also been demonstrated experimentally

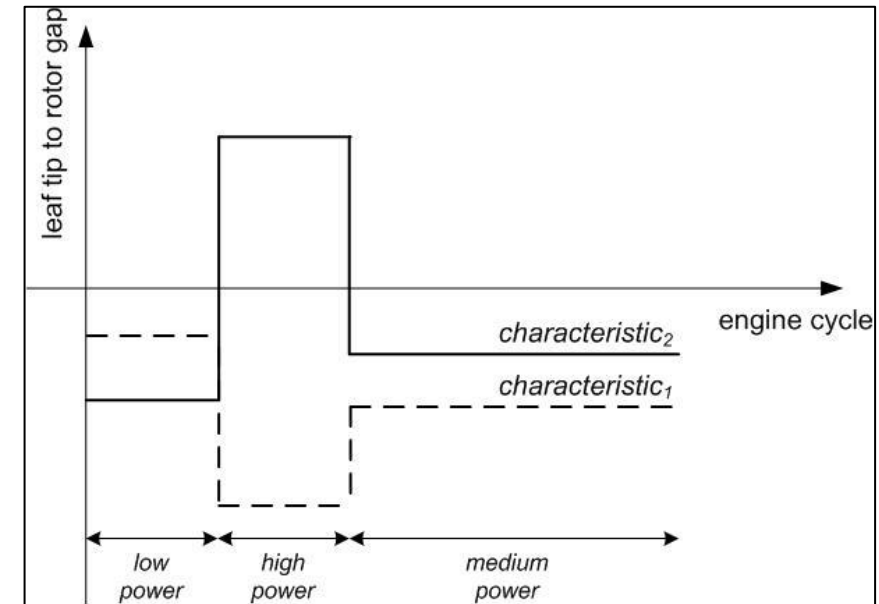


# Leaf seal blowdown and liftup

- A leaf seal blow-down or lift-up characteristic depends on the seal geometric parameters (leaves, leaf pack and cover plates)
- Blow-down and lift-up are also a function of the differential pressure across the seal, hence they vary for different engine operating conditions (as characteristics 1 and 2 at low, high and medium power) and can have cross-over points

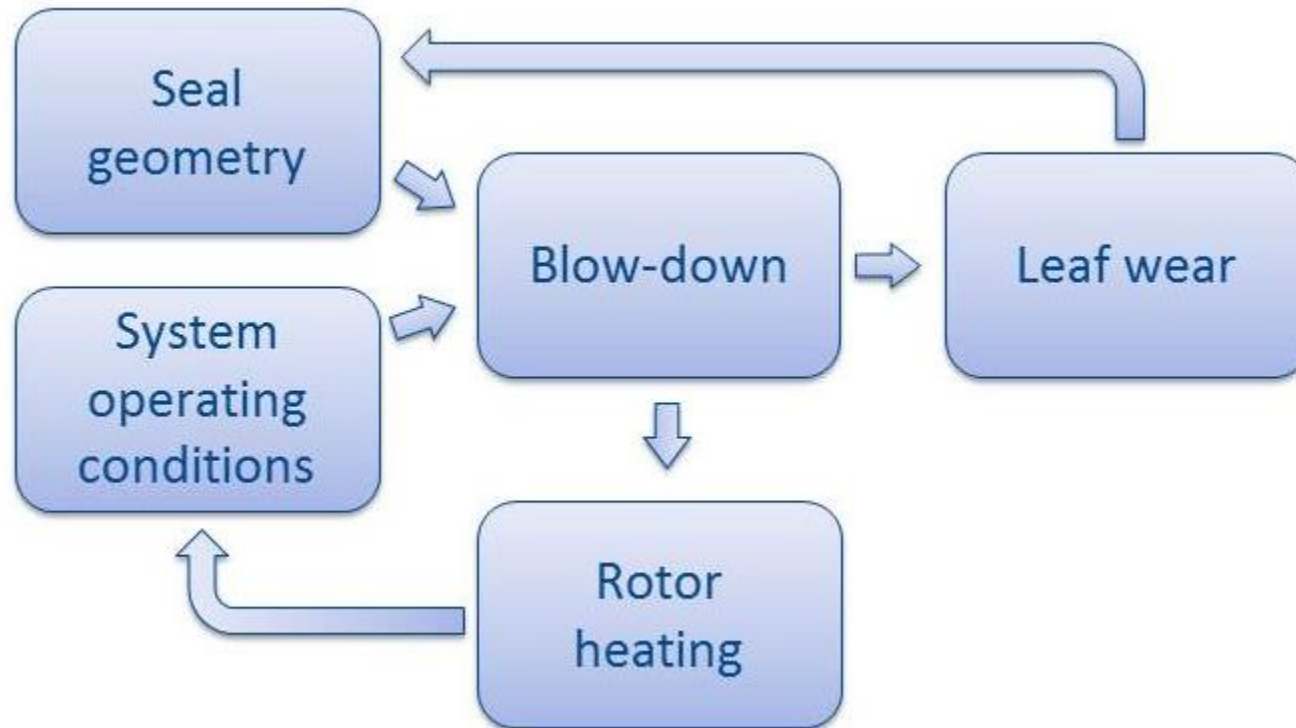


Blow-down characteristics over engine cycle



Leaf tip to rotor gap due to blow-down

# Coupling mechanism for a leaf seal



- More complex component geometries bring more complex coupling mechanisms, which are more complex to capture, model and iterate on
- Coupling is of course not limited to two disciplines. To a degree **all the physics disciplines involved in the modelling of a system or component are coupled** into what is the *real* behaviour of the system or component. The level of coupling needed is also associated to the additional accuracy that the coupling brings.
- Ultimately, **modelling** is always a **trade-off** between **getting accurate answers** and **getting them quickly**.

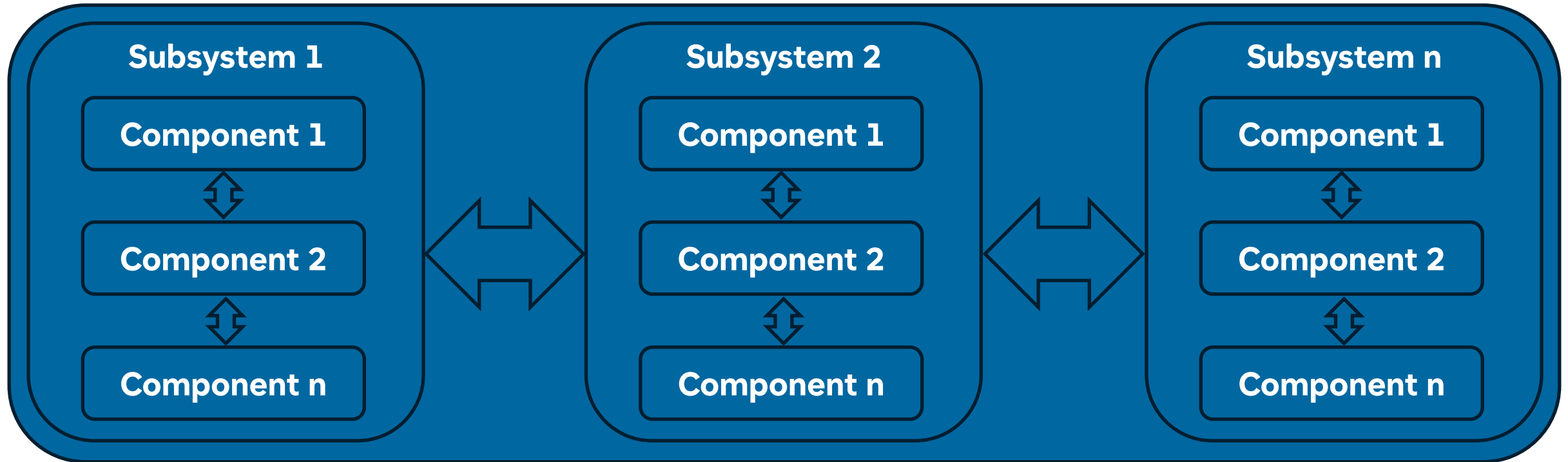
# Design and Systems Engineering

- NASA Systems Engineering Handbook, chapter 2:  
<https://www.nasa.gov/connect/ebooks/nasa-systems-engineering-handbook>

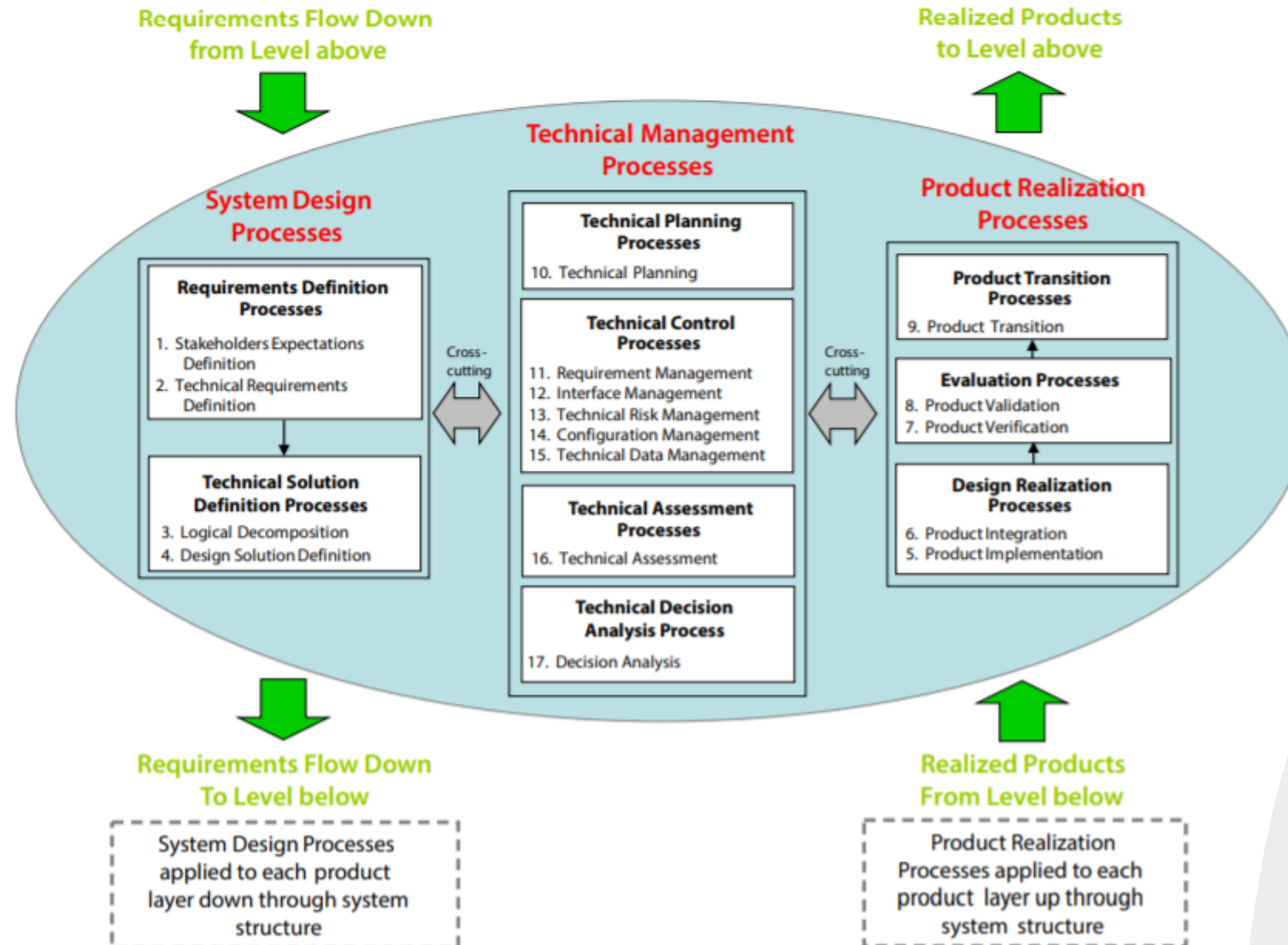
# 09

# How do we address the design of complex systems?

In order to manage its complexity, an engineering product (or **system**) is typically split in a number of **subsystems**, each of them constituted by multiple **components**. All the components and subsystems are interconnected via multiple interactions → **physical** and **functional** interfaces



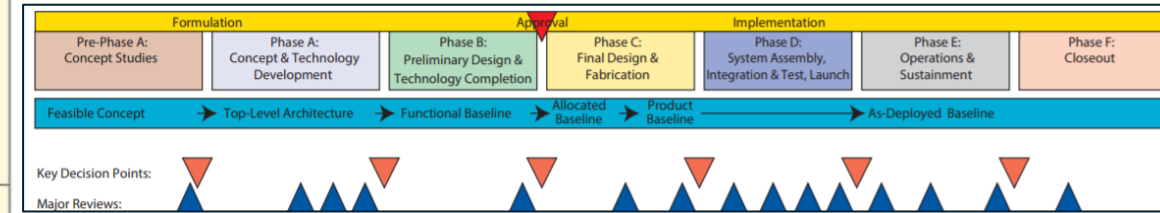
# The Systems Engineering Engine





# Product lifecycle

| Phase           | Purpose                                                         | Typical Outcomes                                                                                                                                                                                                                                                                                                             |                                                                                                                             |
|-----------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Pre-Formulation | <b>Pre-Phase A</b><br>Concept Studies                           | To produce a broad spectrum of ideas and alternatives for missions from which new programs/projects can be selected. Determine feasibility of desired system, develop mission concepts, draft system-level requirements, assess performance, cost, and schedule feasibility; identify potential technology needs, and scope. | Feasible system concepts in the form of simulations, analysis, study reports, models, and mock-ups                          |
| Formulation     | <b>Phase A</b><br>Concept and Technology Development            | To determine the feasibility and desirability of a suggested new system and establish an initial baseline compatibility with NASA's strategic plans. Develop final mission concept, system-level requirements, needed system technology developments, and program/project technical management plans.                        | System concept definition in the form of simulations, analysis, engineering models and mock-ups, and trade study definition |
|                 | <b>Phase B</b><br>Preliminary Design and Technology Completion  | To define the project in enough detail to establish an initial baseline capable of meeting mission needs. Develop system structure end product (and enabling product) requirements and generate a preliminary design for each system structure end product.                                                                  | End products in the form of mock-ups, trade study results, specification and interface documents, and prototypes            |
| Implementation  | <b>Phase C</b><br>Final Design and Fabrication                  | To complete the detailed design of the system (and its associated subsystems, including its operations systems), fabricate hardware, and code software. Generate final designs for each system structure end product.                                                                                                        | End product detailed designs, end product component fabrication, and software development                                   |
|                 | <b>Phase D</b><br>System Assembly, Integration and Test, Launch | To assemble and integrate the system (hardware, software, and humans), meanwhile developing confidence that it is able to meet the system requirements. Launch and prepare for operations. Perform system end product implementation, assembly, integration and test, and transition to use.                                 | Operations-ready system end product with supporting related enabling products                                               |
|                 | <b>Phase E</b><br>Operations and Sustainment                    | To conduct the mission and meet the initially identified need and maintain support for that need. Implement the mission operations plan.                                                                                                                                                                                     | Desired system                                                                                                              |
|                 | <b>Phase F</b><br>Closeout                                      | To implement the systems decommissioning/disposal plan developed in Phase E and perform analyses of the returned data and any returned samples.                                                                                                                                                                              | Product closeout                                                                                                            |



# Design trade studies

## THE SYSTEMS ENGINEER'S DILEMMA

At each cost-effective solution:

- To reduce cost at constant risk, performance must be reduced.
- To reduce risk at constant cost, performance must be reduced.
- To reduce cost at constant performance, higher risks must be accepted.
- To reduce risk at constant performance, higher costs must be accepted.

In this context, time in the schedule is often a critical resource, so that *schedule* behaves like a kind of cost.

Key product attributes:  
performance, cost

Time is money

This is crucial in defining the **simulation strategy: runtime vs accuracy**

Computational engineering often targets reduction in engineering time

As part of the **risk management**, it is very important to **mitigate failures**

# NASA system engineering competency model

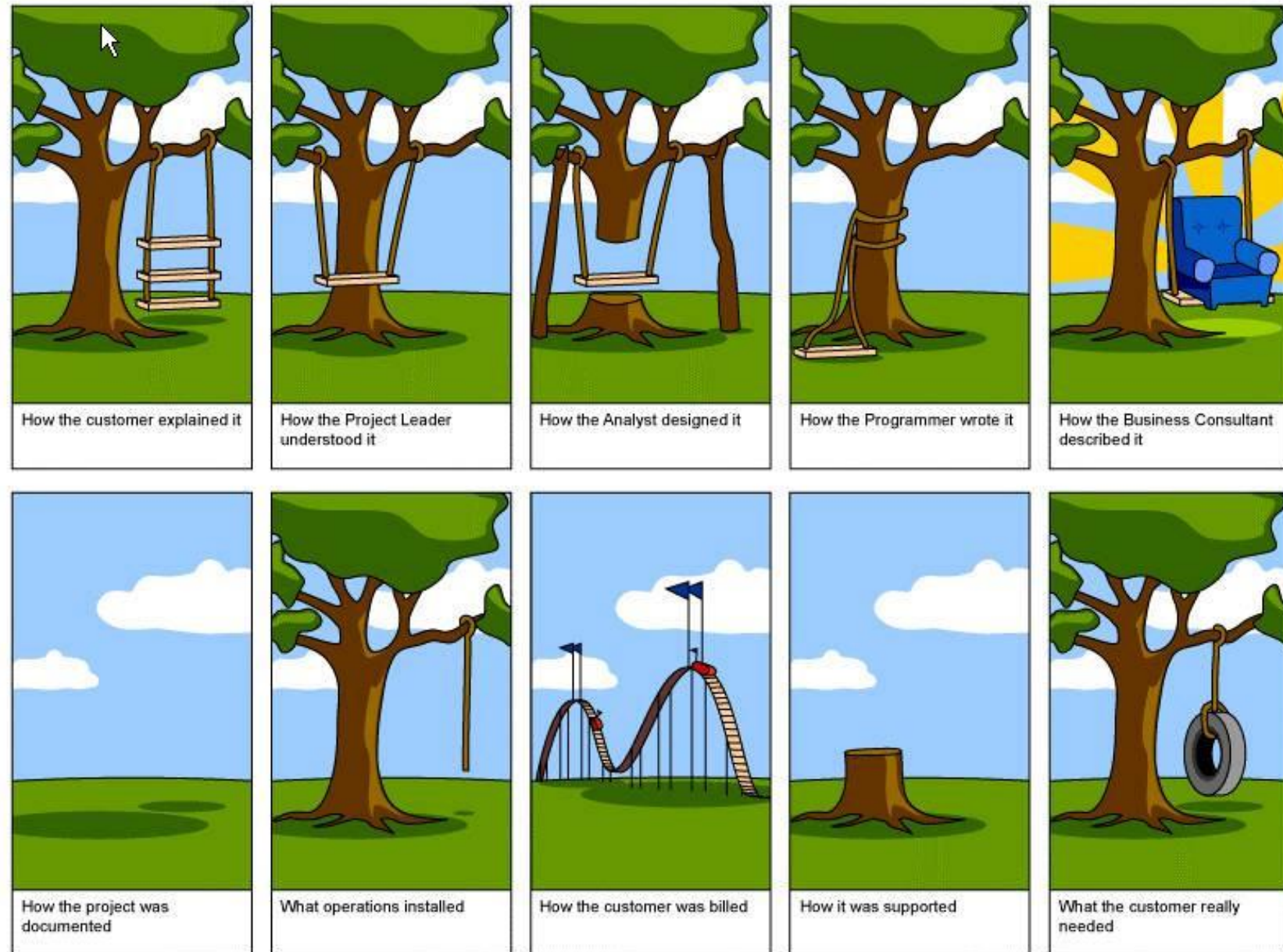
A comprehensive list of **skills and competencies for engineering design tasks**

Table 2.7-1 pages 14, 15, 16

e.g. stakeholder management, technical requirements, design solution, product implementation, product integration, product verification, product validation, interface management, technical risk management, configuration management, technical data management, decision analysis

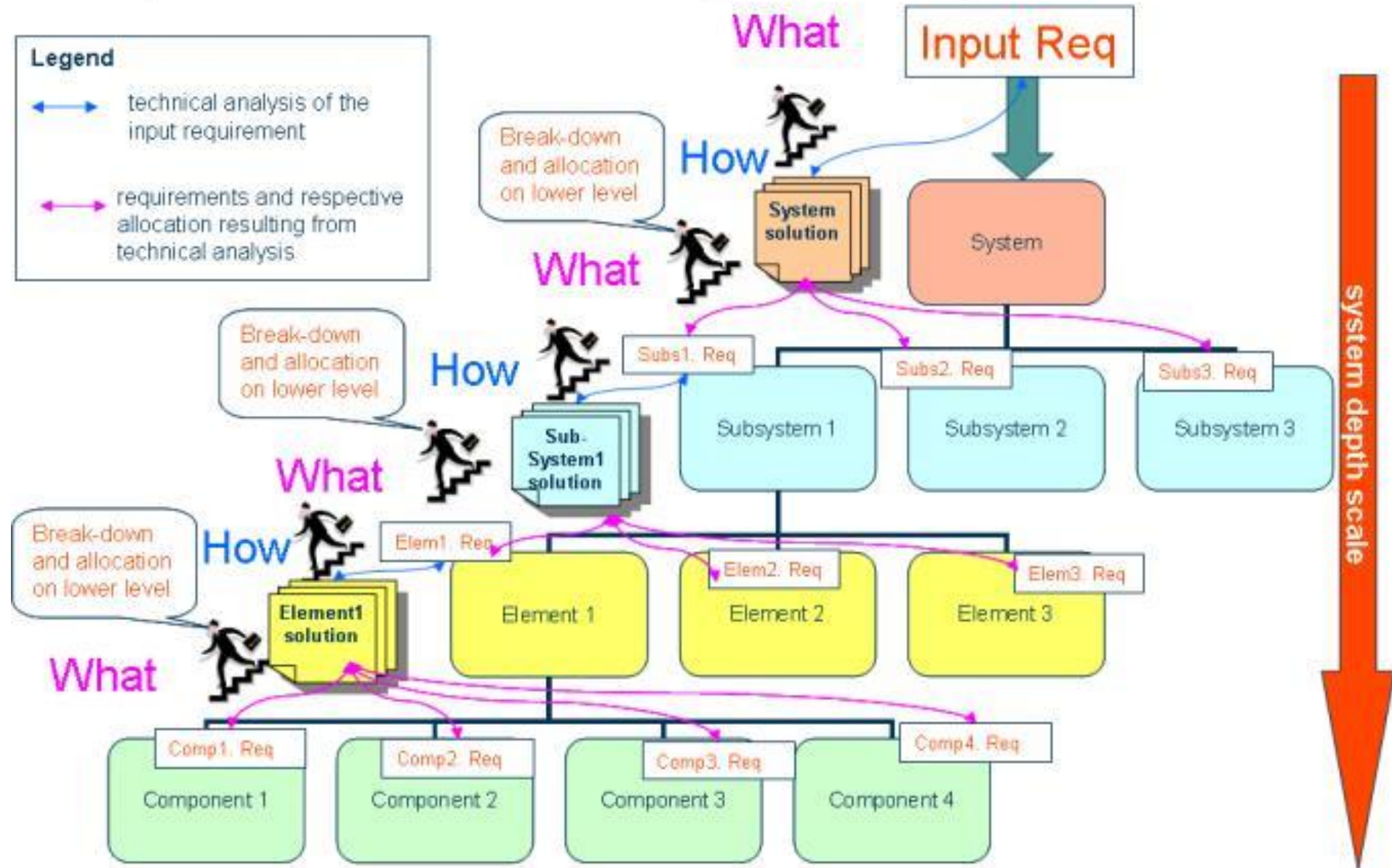


# Requirements



# Flowdown

## Requirement break-down through system layers solutions







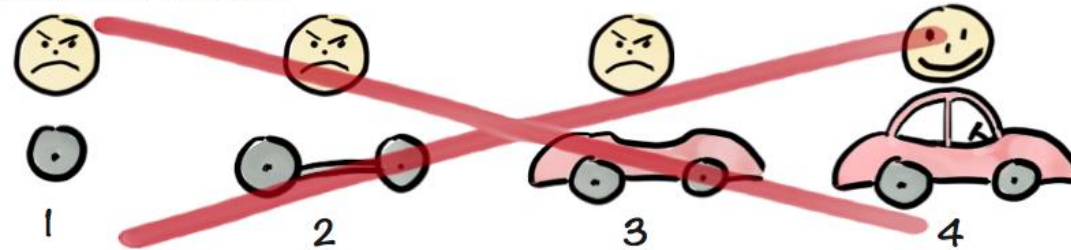
# Agile software development manifesto

We are uncovering better ways of developing software...

Individuals and interactions over processes and tools  
Working software over comprehensive documentation  
Customer collaboration over contract negotiation  
Responding to change over following a plan

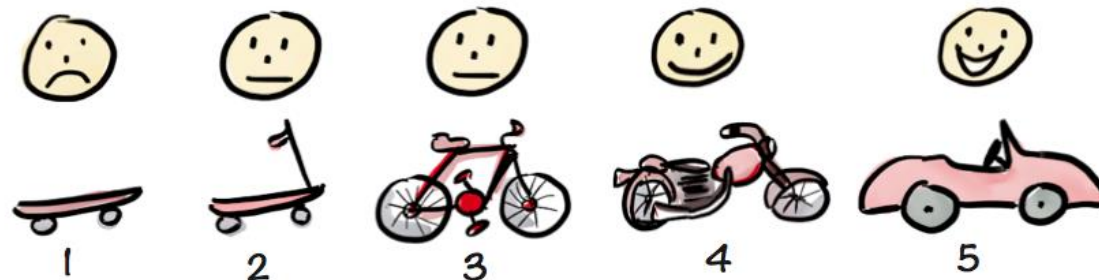
That is, while there is value in the items on the right, we value the items on the left more

**Not like this....**



*Requirements vs  
user stories*

**Like this!**



# Technical data tasks

Table 6.6-1 pages 156, 157

Technical data capture, Technical data maintenance, Technical data/information distribution, Data management system maintenance

**PLM** – product lifecycle management

PLM software usually supports the management of the design of complex engineering systems (e.g. Teamcenter – Siemens Software)

In the context of the digital twin, also **simulation data management** is key to make sure simulation data can be version controlled and re-used whenever needed

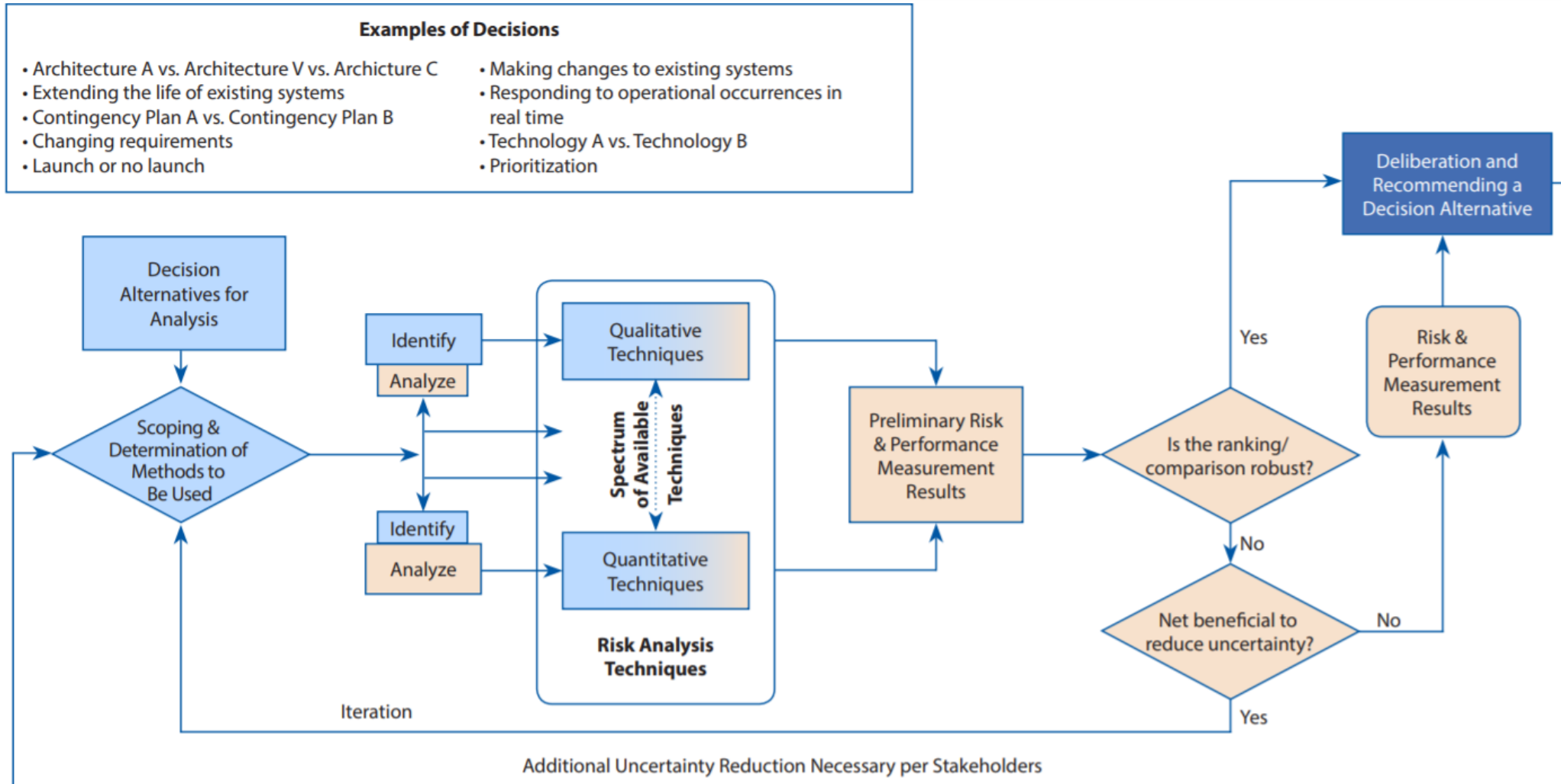
**SPDM** – simulation process data management

[chapter 6.

<https://www.nasa.gov/connect/ebooks/nasa-systems-engineering-handbook>]



# Risk management and engineering decisions

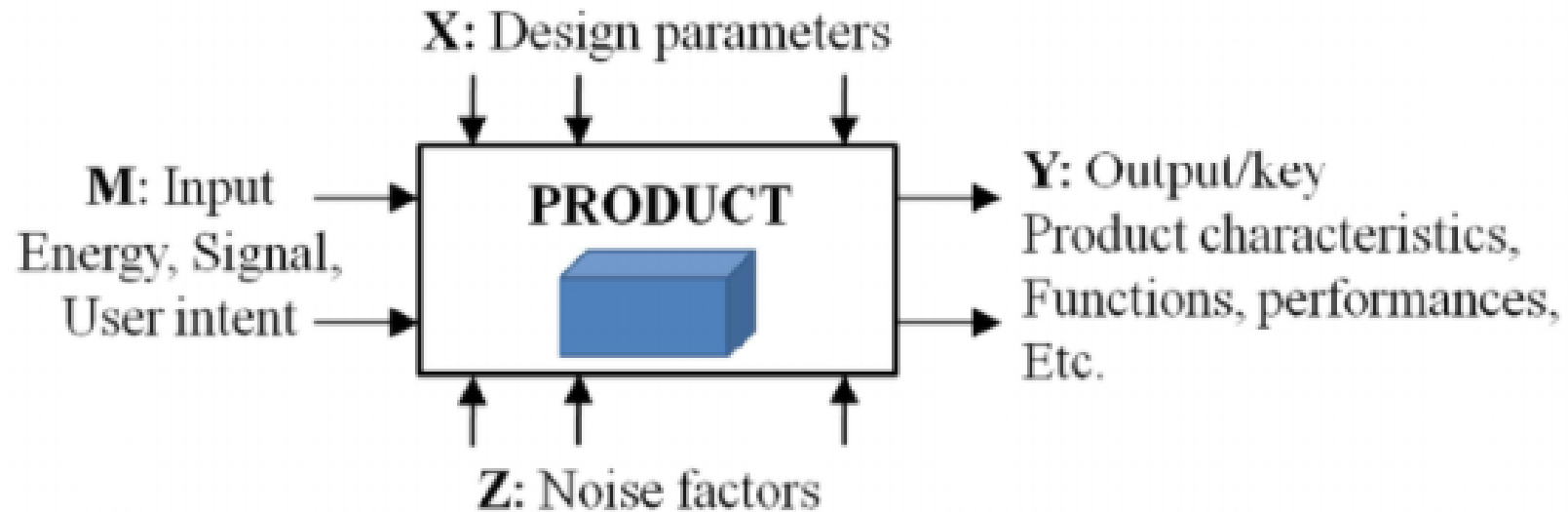


# Systems Engineering Tools

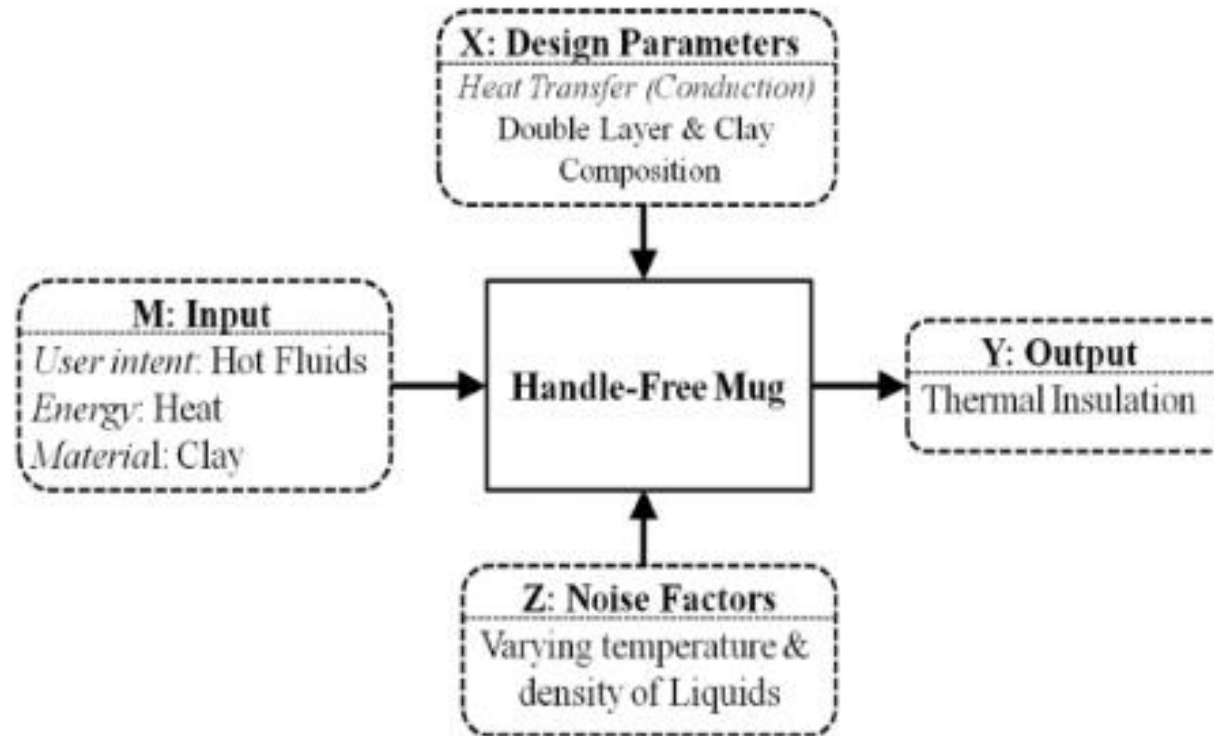
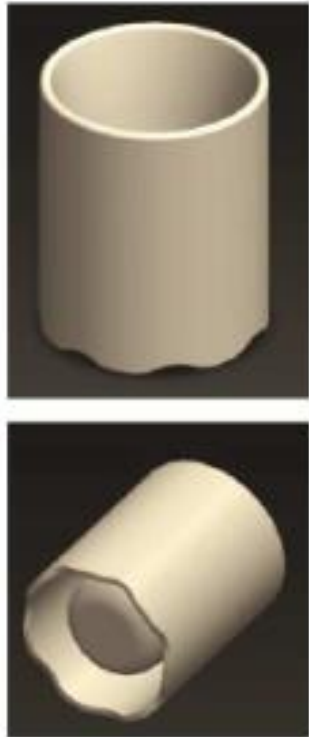
# 10

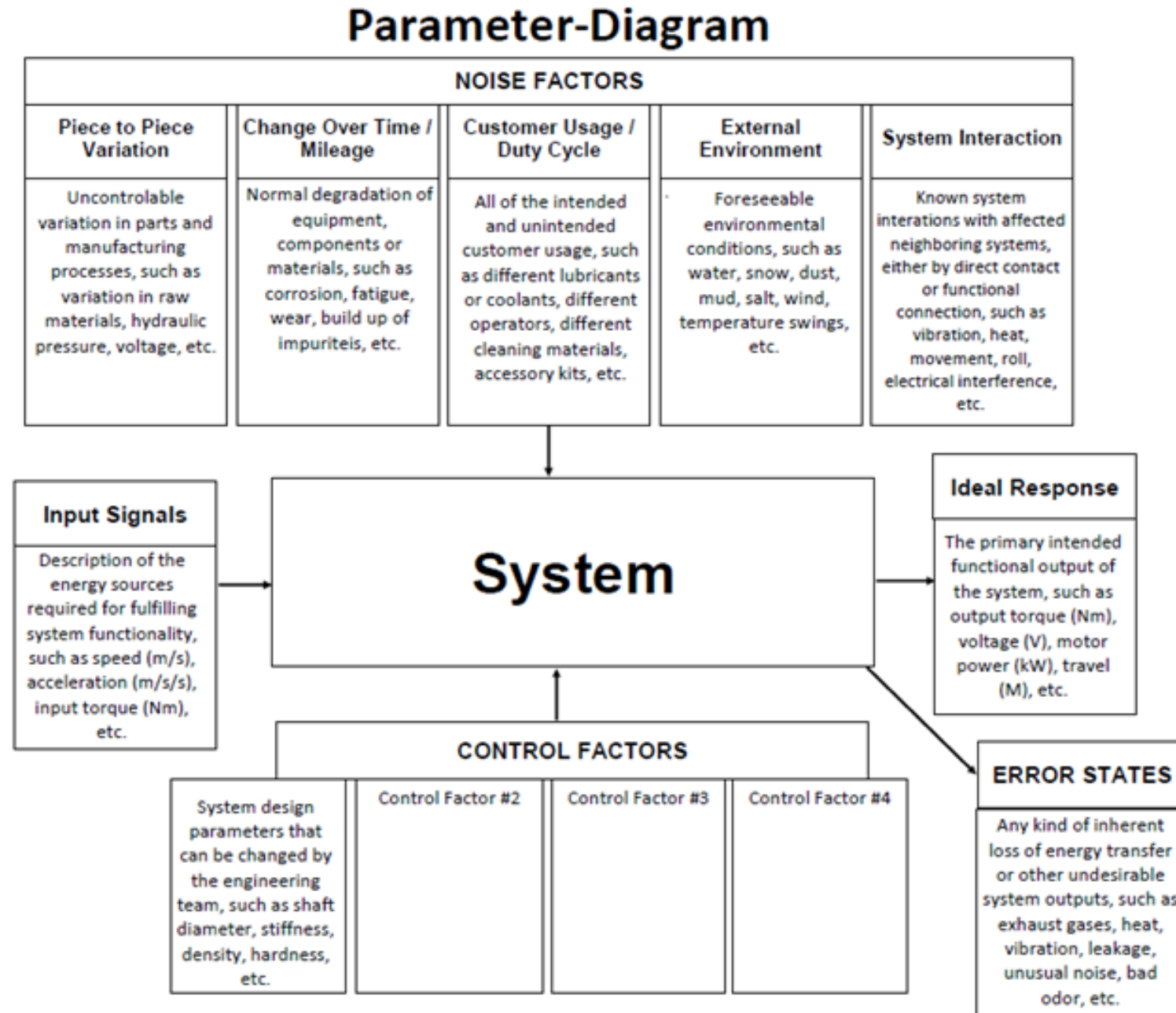
# P-diagram (Parameter diagram)

Understanding a system as an input-output relationship



# P-diagram Example





# Functional Modelling

**Functional modelling** is a powerful technique that allows to capture diagrammatically the key functionalities of a system and their interactions. It is particularly used in the analysis of **complex engineering products** and can also be used for **simulation software**.

Although very powerful, functional modelling is an abstraction of the real problem and does not always reflect the actual product.

For example, for software systems, functional modelling does not correspond to the software architecture or the implementation (software design) but it only captures the main functionalities

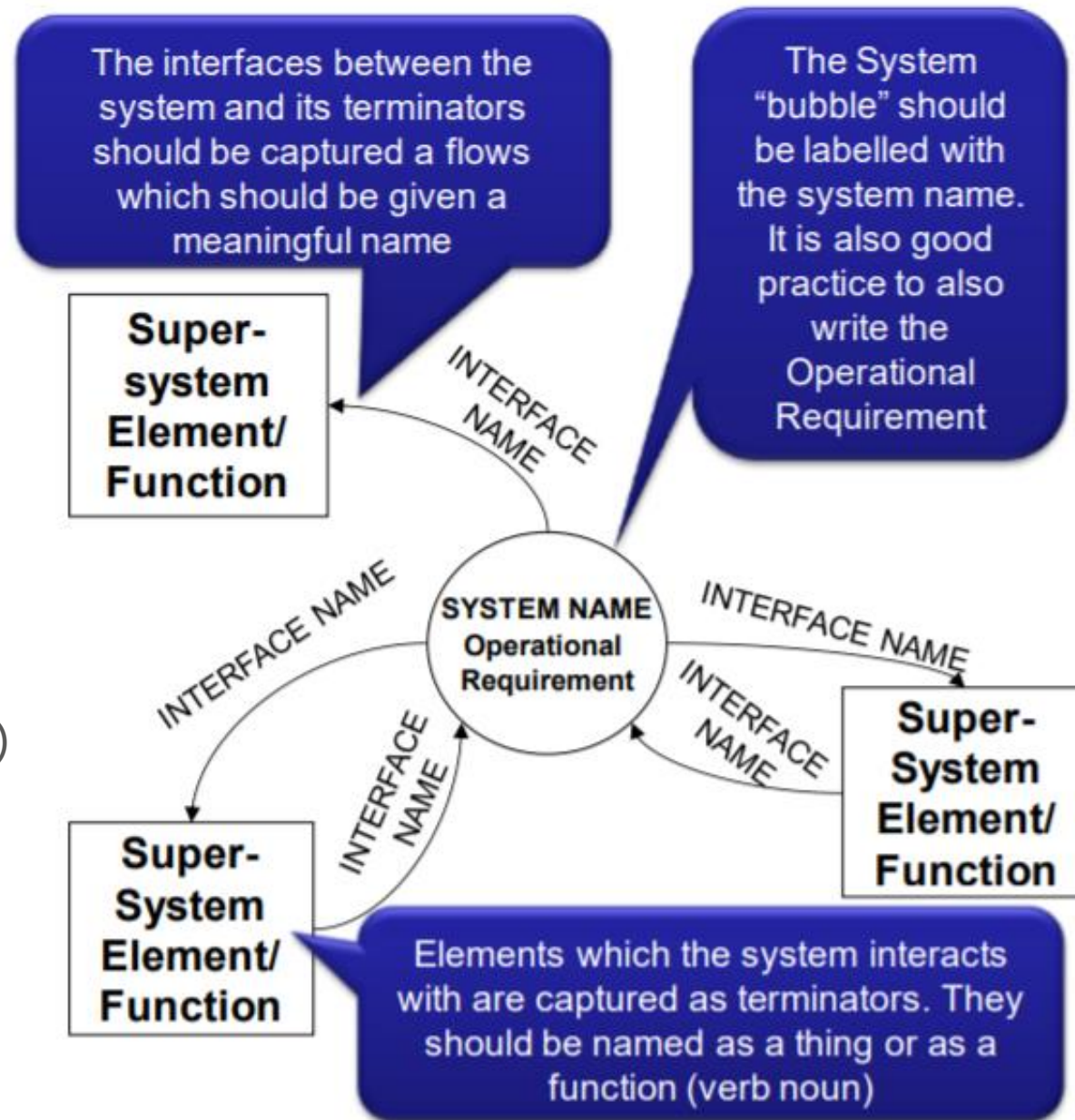
## Further reading:

The systems engineering toolbox paper:

*<https://www.burgehugheswalsh.co.uk/Uploaded/1/Documents/Functional-Modelling-Tool-Draft.pdf>*

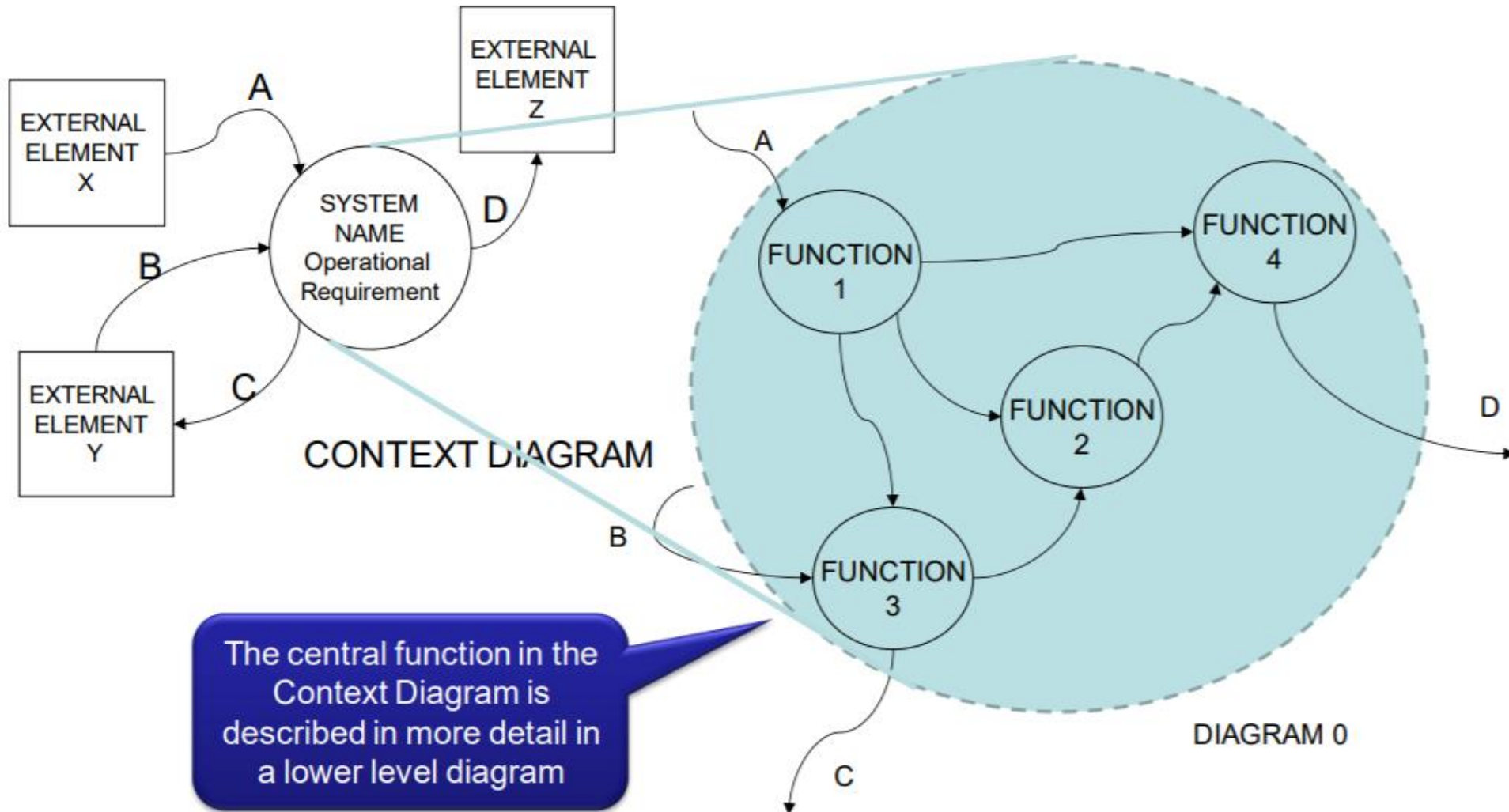
# Context diagram

- Operational requirement
- Functional and non-functional requirements
- Understand scope and boundaries (in and out of scope)





# Functional Flow Diagram





# Pugh Matrix

Decision-making based on quantitative data

| Evaluation Criteria              | Current Vendor | Vendor 1 | Vendor 2 | Vendor 3 | Synthesized Vendor | Importance Rating |
|----------------------------------|----------------|----------|----------|----------|--------------------|-------------------|
| Cost per unit                    |                | (-)      | (-)      | s        | Vendor 3           | 0.12              |
| Availability of onhand stock     |                | (-)      | (+)      | (-)      | Vendor 2           | 0.05              |
| National distributor             |                | (-)      | s        | s        | Vendor 3           | 0.15              |
| American made                    |                | (+)      | (-)      | (+)      | Vendor 3           | 0.20              |
| Flexible payment terms           |                | (-)      | (-)      | (+)      | Vendor 3           | 0.10              |
| Increase revenue                 |                | (-)      | (+)      | (+)      | Vendor 3           | 0.10              |
| Quality of product               |                | s        | (+)      | (-)      | Vendor 2           | 0.20              |
| Local distribution center        |                | (+)      | (+)      | (+)      | Vendor 3           | 0.08              |
| TOTAL +                          |                | 2        | 4        | 4        |                    | 1.00              |
| TOTAL -                          |                | 5        | 3        | 2        |                    |                   |
| Net value                        |                | -3       | 1        | 2        |                    |                   |
| Weighted Total +                 |                | 0.28     | 0.35     | 0.48     |                    |                   |
| Weighted Total -                 |                | 0.52     | 0.42     | 0.25     |                    |                   |
| al Weighted Value of Improvement |                | -0.24    | -0.07    | 0.23     |                    |                   |

Evaluation criteria and associated weights

# FMEA Failure Mode and Effect Analysis

| Process Step / Input                                    | Potential Failure Mode                    | Potential Failure Effects                                               | S<br>E<br>V<br>E<br>R<br>I<br>T<br>Y | Potential Causes                       | O<br>C<br>C<br>U<br>R<br>R<br>E<br>N<br>C<br>E | Current Controls                                                                                                       | D<br>E<br>T<br>E<br>C<br>T<br>I<br>O<br>N | R<br>P<br>N |
|---------------------------------------------------------|-------------------------------------------|-------------------------------------------------------------------------|--------------------------------------|----------------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|-------------|
| What is the process step and Input under investigation? | In what ways does the Key Input go wrong? | What is the impact on the Key Output Variables (Customer Requirements)? |                                      | What causes the Key Input to go wrong? |                                                | What are the existing controls and procedures (inspection and test) that prevent either the cause or the Failure Mode? |                                           |             |
|                                                         |                                           |                                                                         |                                      |                                        |                                                |                                                                                                                        |                                           | 0           |
|                                                         |                                           |                                                                         |                                      |                                        |                                                |                                                                                                                        |                                           | 0           |
|                                                         |                                           |                                                                         |                                      |                                        |                                                |                                                                                                                        |                                           | 0           |

## FMECA (Failure Mode, Effects & Criticality Analysis)

# FMEA Example

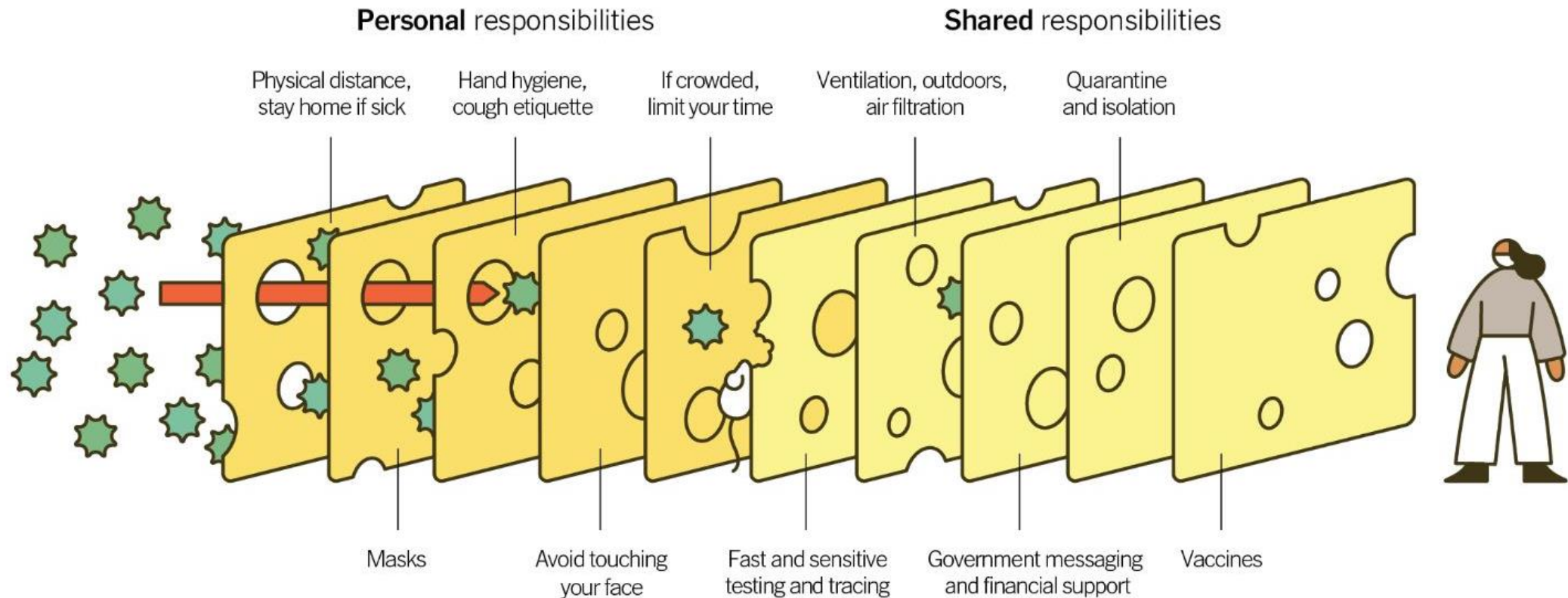
| Process Step           | Potential Failure Mode                | Potential Failure Effect                                                                                               | SEV <sup>1</sup>                          | Potential Causes                                                                                                   | OCC <sup>2</sup>                             | Current Process Controls                                                                                         | DET <sup>3</sup>                                            | RPN <sup>4</sup>                                   | Action Recommended                                                                                                                                                 |
|------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What is the step?      | In what ways can the step go wrong?   | What is the impact on the customer if the failure mode is not prevented or corrected?                                  | How severe is the effect on the customer? | What causes the step to go wrong (i.e., how could the failure mode occur)?                                         | How frequently is the cause likely to occur? | What are the existing controls that either prevent the failure mode from occurring or detect it should it occur? | How probable is detection of the failure mode or its cause? | Risk priority number calculated as SEV x OCC x DET | What are the actions for reducing the occurrence of the cause or for improving its detection? Provide actions on all high RPNs and on severity ratings of 9 or 10. |
| ATM Pin Authentication | Unauthorized access                   | <ul style="list-style-type: none"> <li>• Unauthorized cash withdrawal</li> <li>• Very dissatisfied customer</li> </ul> | 8                                         | Lost or stolen ATM card                                                                                            | 3                                            | Block ATM card after three failed authentication attempts                                                        | 3                                                           | 72                                                 |                                                                                                                                                                    |
|                        | Authentication failure                | Annoyed customer                                                                                                       | 3                                         | Network failure                                                                                                    | 5                                            | Install load balancer to distribute work-load across network links                                               | 5                                                           | 75                                                 |                                                                                                                                                                    |
| Dispense Cash          | Cash not disbursed                    | Dissatisfied customer                                                                                                  | 7                                         | ATM out of cash                                                                                                    | 7                                            | Internal alert of low cash in ATM                                                                                | 4                                                           | 196                                                | Increase minimum cash threshold limit of heavily used ATMs to prevent out-of-cash instances                                                                        |
|                        | Account debited but no cash disbursed | Very dissatisfied customer                                                                                             | 8                                         | <ul style="list-style-type: none"> <li>• Transaction failure</li> <li>• Network issue</li> </ul>                   | 3                                            | Install load balancer to distribute work-load across network links                                               | 4                                                           | 96                                                 |                                                                                                                                                                    |
|                        | Extra cash dispensed                  | Bank loses money                                                                                                       | 8                                         | <ul style="list-style-type: none"> <li>• Bills stuck to each other</li> <li>• Bills stacked incorrectly</li> </ul> | 2                                            | Verification while loading cash in ATM                                                                           | 3                                                           | 48                                                 |                                                                                                                                                                    |

- Severity:** Severity of impact of failure event. It is scored on a scale of 1 to 10. A high score is assigned to high-impact events while a low score is assigned to low-impact events.
- Occurrence:** Frequency of occurrence of failure event. It is scored on a scale of 1 to 10. A high score is assigned to frequently occurring events while events with low occurrence are assigned a low score.
- Detection:** Ability of process control to detect the occurrence of failure events. It is scored on a scale of 1 to 10. A failure event that can be easily detected by the process control is assigned a low score while a high score is assigned to an inconspicuous event.
- Risk priority number:** The overall risk score of an event. It is calculated by multiplying the scores for severity, occurrence and detection. An event with a high RPN demands immediate attention while events with lower RPNs are less risky.

# The Swiss Cheese Model

## Multiple Layers Improve Success

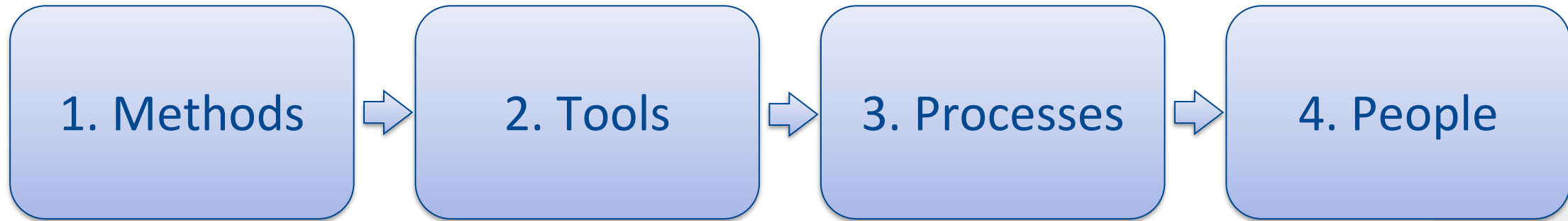
The Swiss Cheese Respiratory Pandemic Defense recognizes that no single intervention is perfect at preventing the spread of the coronavirus. Each intervention (layer) has holes.



# Process Automation in Engineering

# 11

# Modelling and simulation in engineering



1. Modelling and validating the physics
2. Developing the simulation and design tools
3. Standardising and automating the processes
4. Embedding into the working practices



# Simulation software for product design

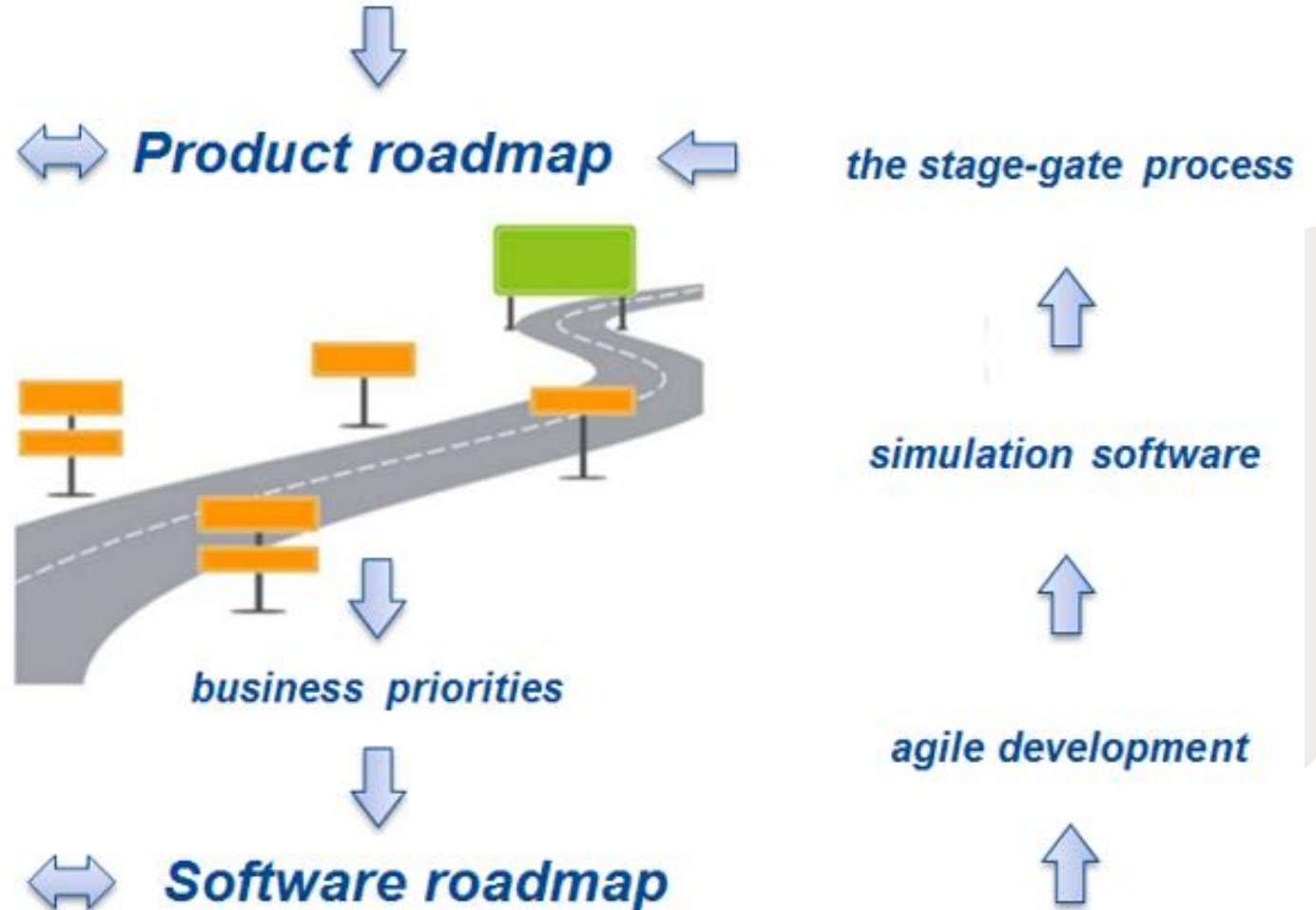
*Internal customers*



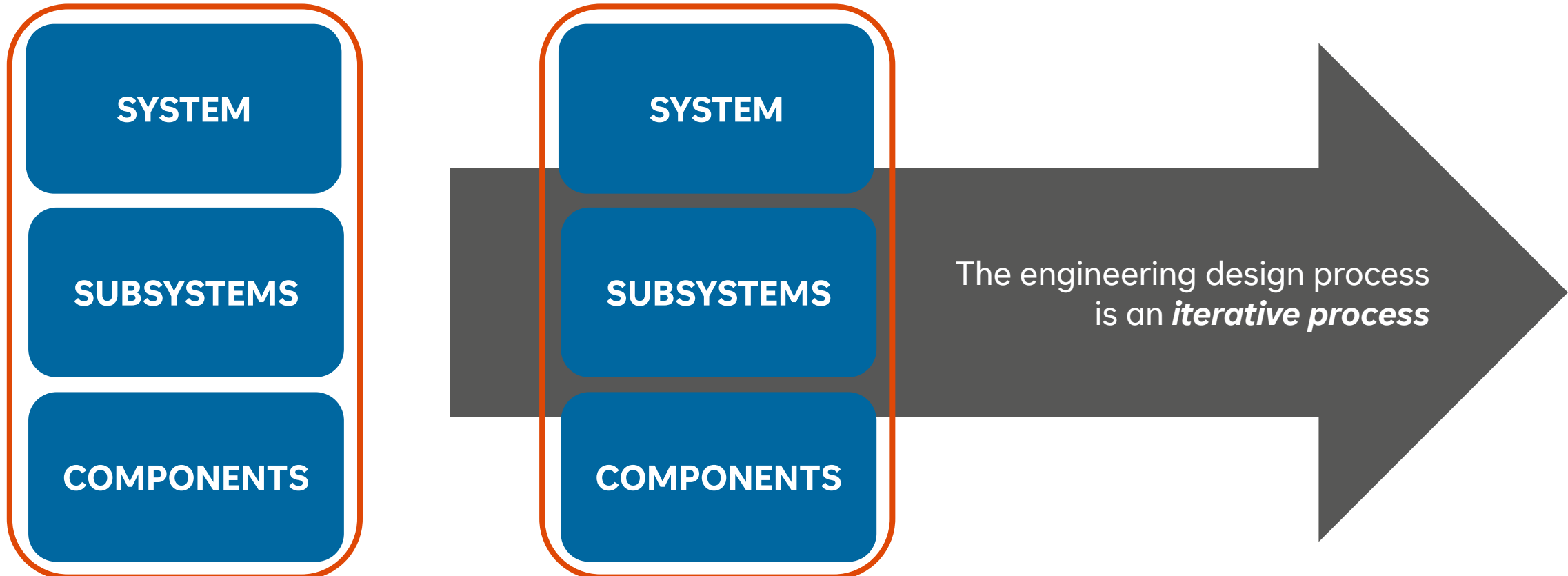
*Simulation software*



*External customers*

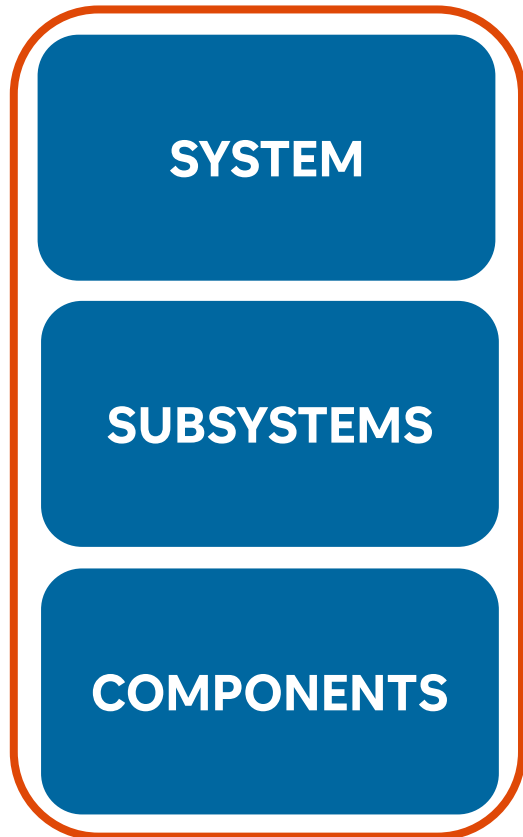


# The engineering design process





# The engineering design process



# Process automation

**Process automation** is the discipline that **automates the engineering design process**, e.g. flow-down of boundary conditions from system to components or exchange of boundary conditions among components, transfer of data from one sub-system design to another (or from one department to another, or from one engineer to another), design iterations, comparisons, trade studies, etc.

- **Simulation** brings into software a physics model of an engineering product or component
- **Process automation** brings into software the design process itself
  - Typically process automation links together multiple simulation tools via automated data transfers to build multi-disciplinary simulation frameworks called *Integrated Design Systems*

## Key benefits:

- Speed up of the process: automation of the manual and time-consuming steps gives more space to use engineering time for interpreting data, thinking and making decisions
- Improvements of product attributes: automating the process allows more comprehensive design space searches and, as a consequence, an improved product design

# Process automation



## The Telegraph

Home Video News World Sport Business Money Comment Culture Travel Life  
Home Aerospace Design Energy Healthcare Innovation @STEMawards2016

HOME » SPONSORED » EDUCATION » STEM AWARDS » AEROSPACE

### 'Computers are not going to replace human engineers'

Rolls-Royce's Design and Validate in the Computer initiative (cunningly code-named DaVinci) is a "virtual engine" that can simulate the attributes of an entire jet engine in software



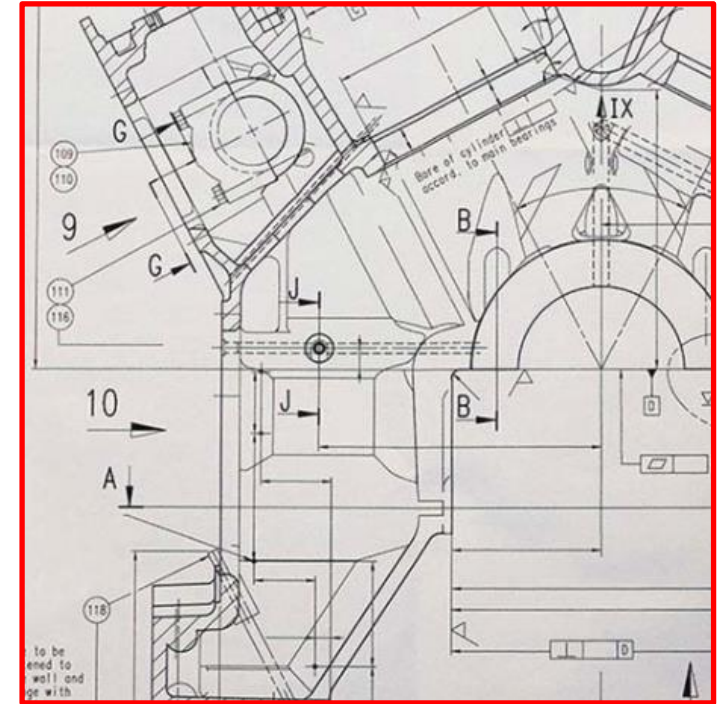
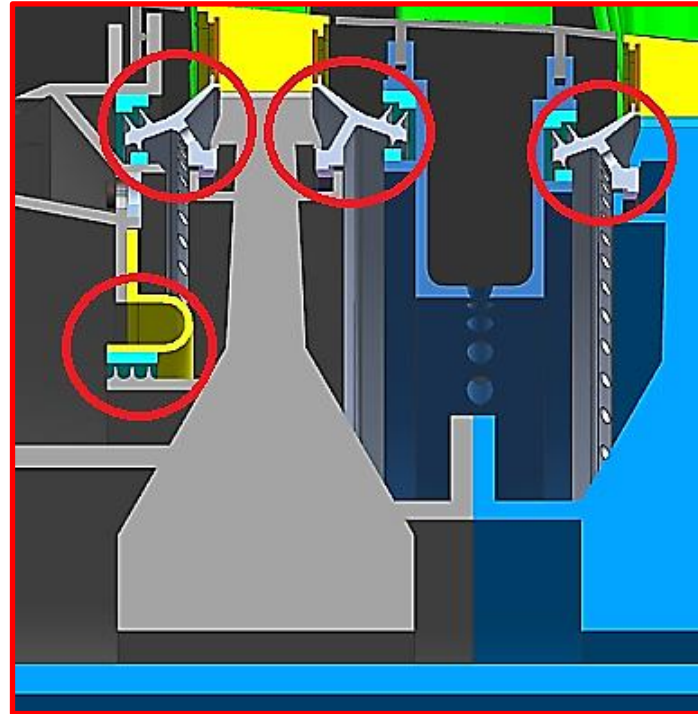
By Paul Bray

5:54PM BST 03 Jun 2016

*The rapidly growing power of high-performance computing is revolutionising **aerospace design**, as computers take on more of the work.*

*Rolls-Royce's Design and Validate in the Computer initiative (cunningly code-named DaVinci) is a "virtual engine" that can simulate the attributes of an entire jet engine in software.*

# Component modelling

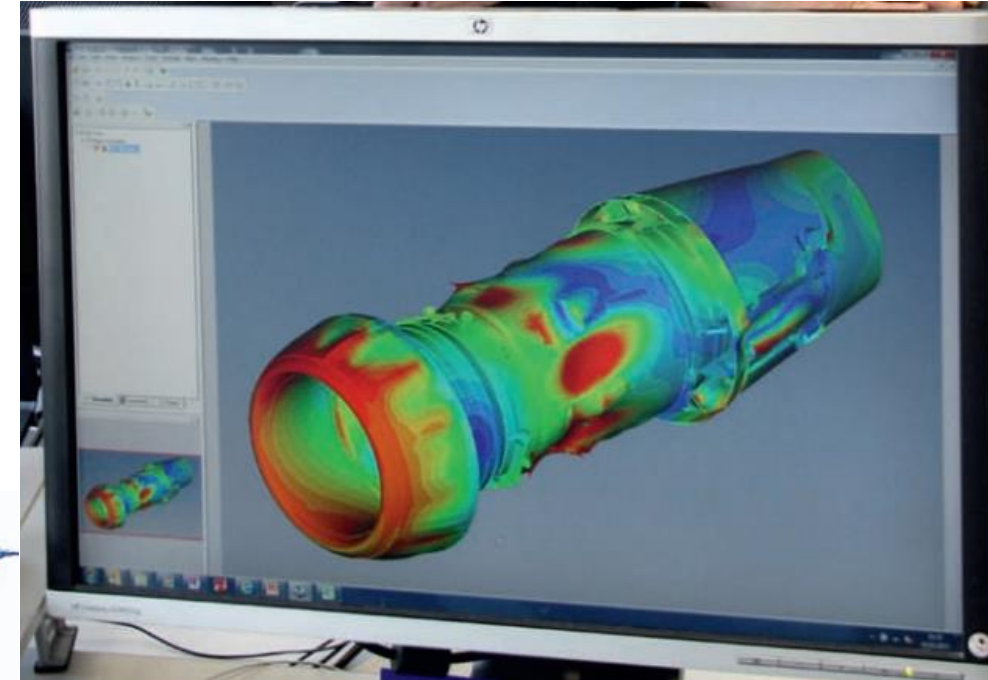
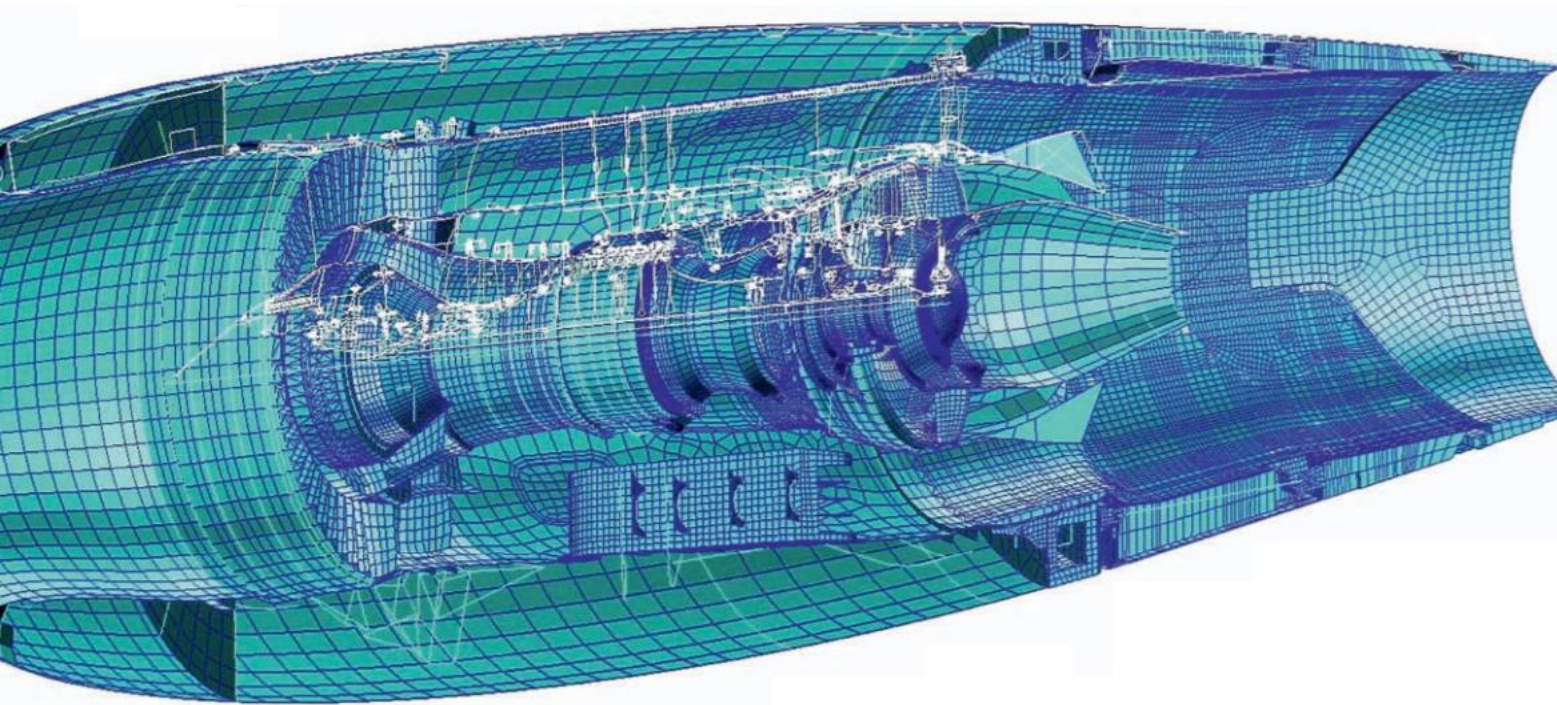




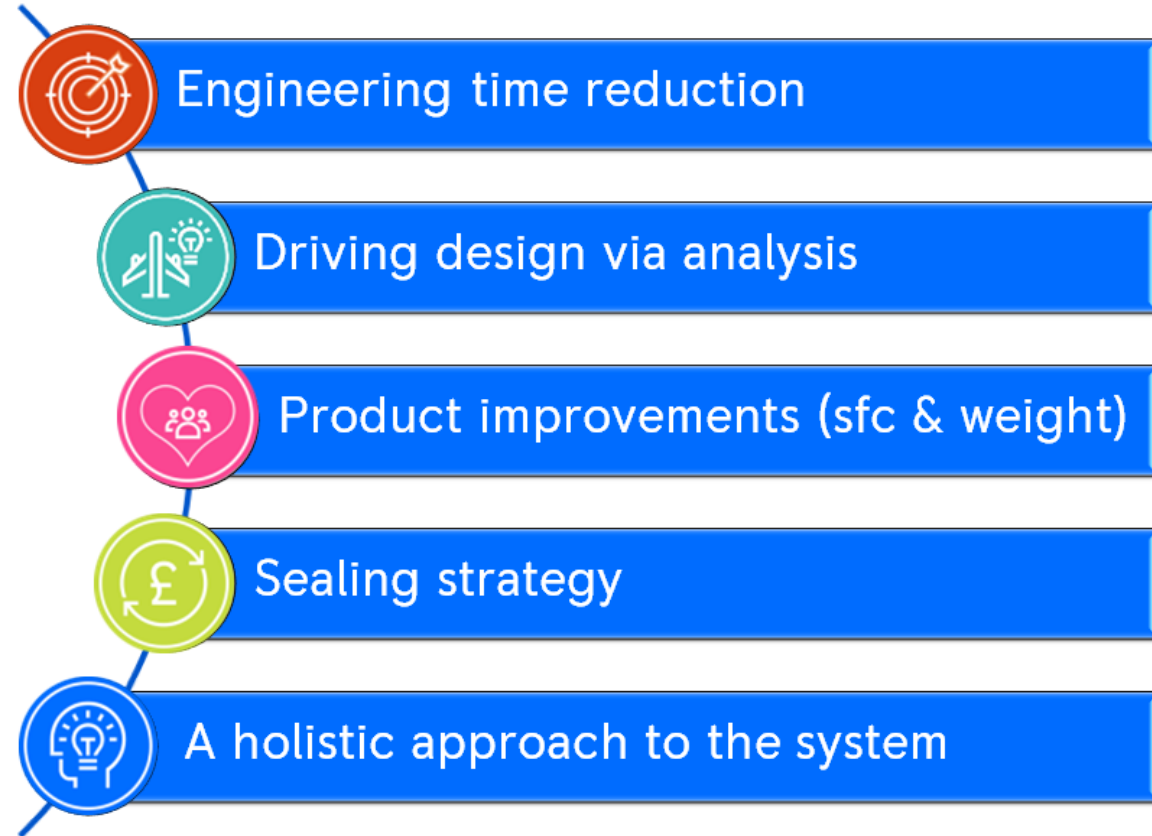
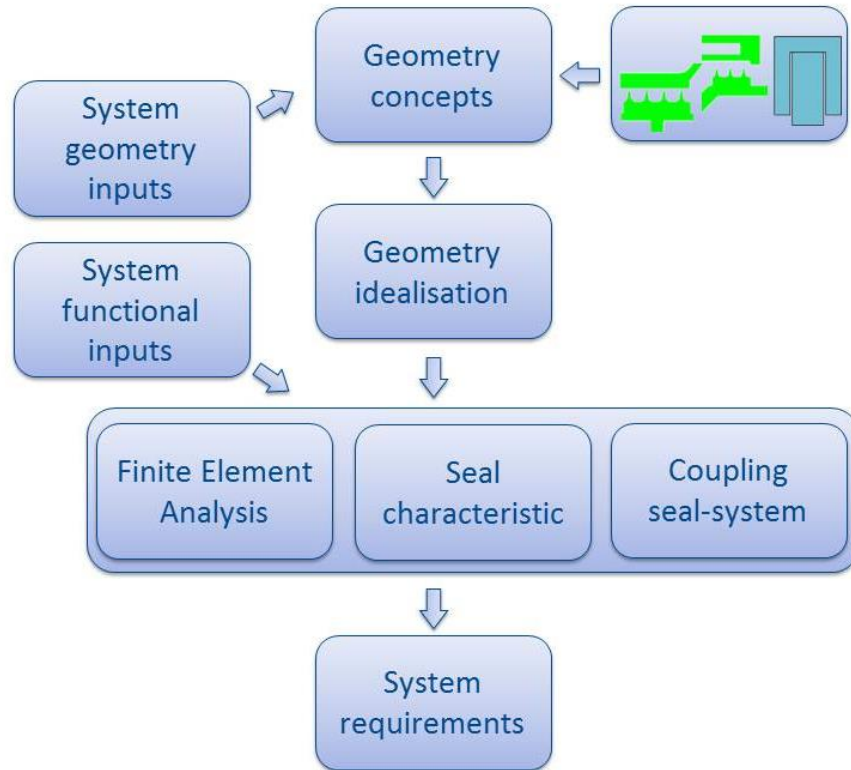
# System-level modelling

Further reading: aero engine Finite Element Analysis models:

<https://www.rolls-royce.com/~media/Files/R/Rolls-Royce/documents/news/featured-stories-archive/paysan-technologist.pdf>

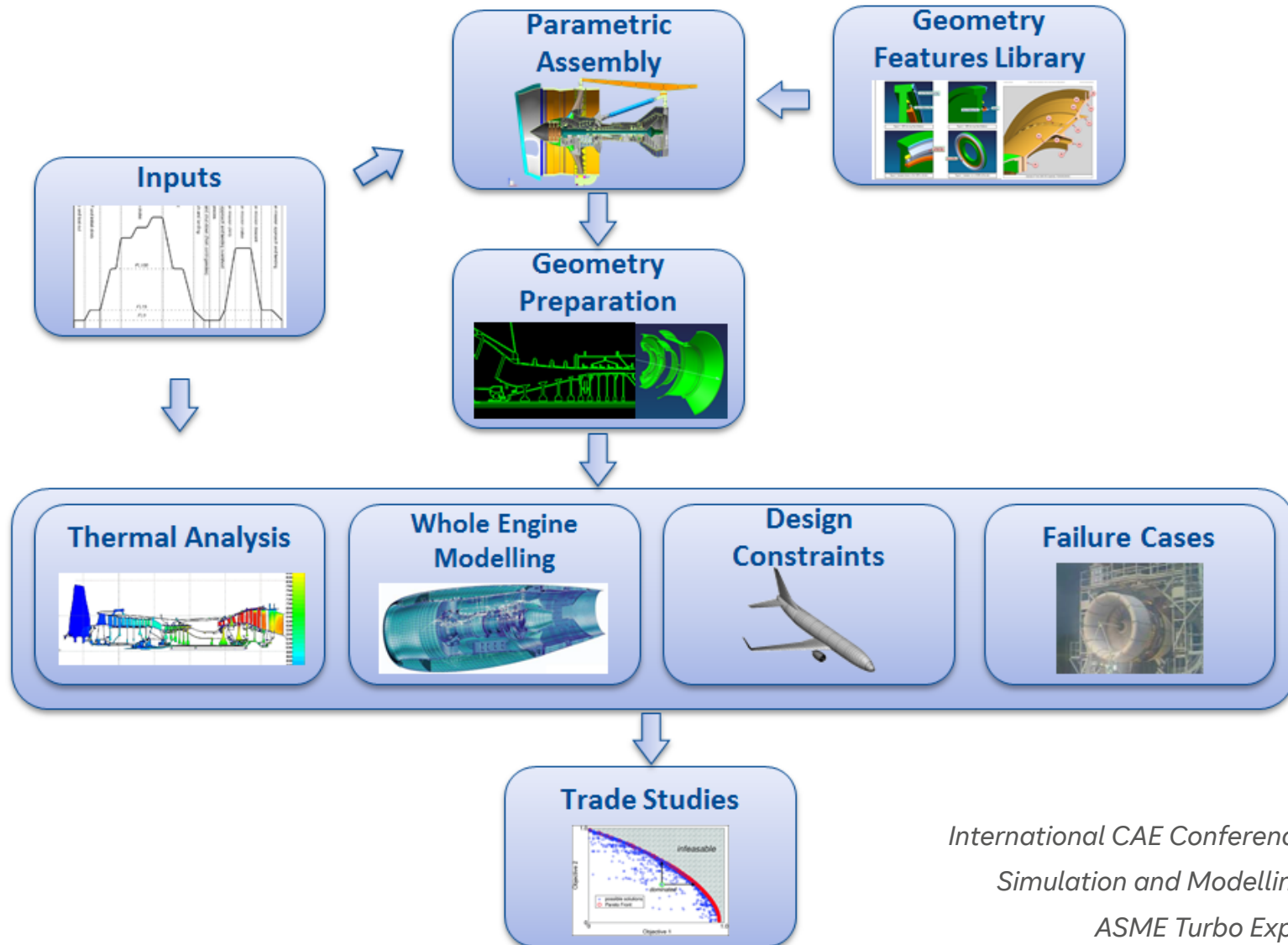


# Process automation in aero engine seal design



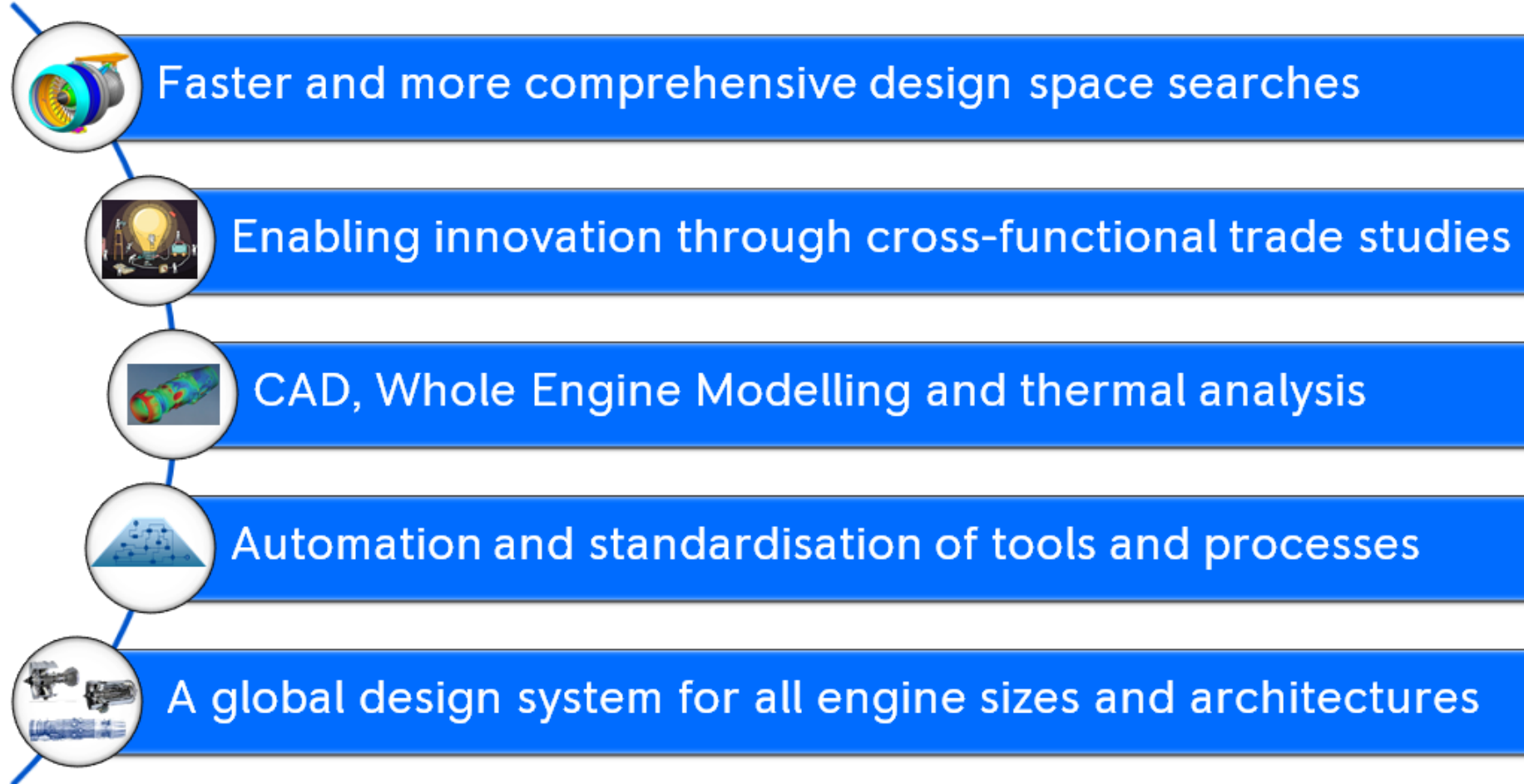
# Process automation in aero engine system design

- Multi-disciplinary analysis
- Robust design and optimisation
- A holistic approach to the system



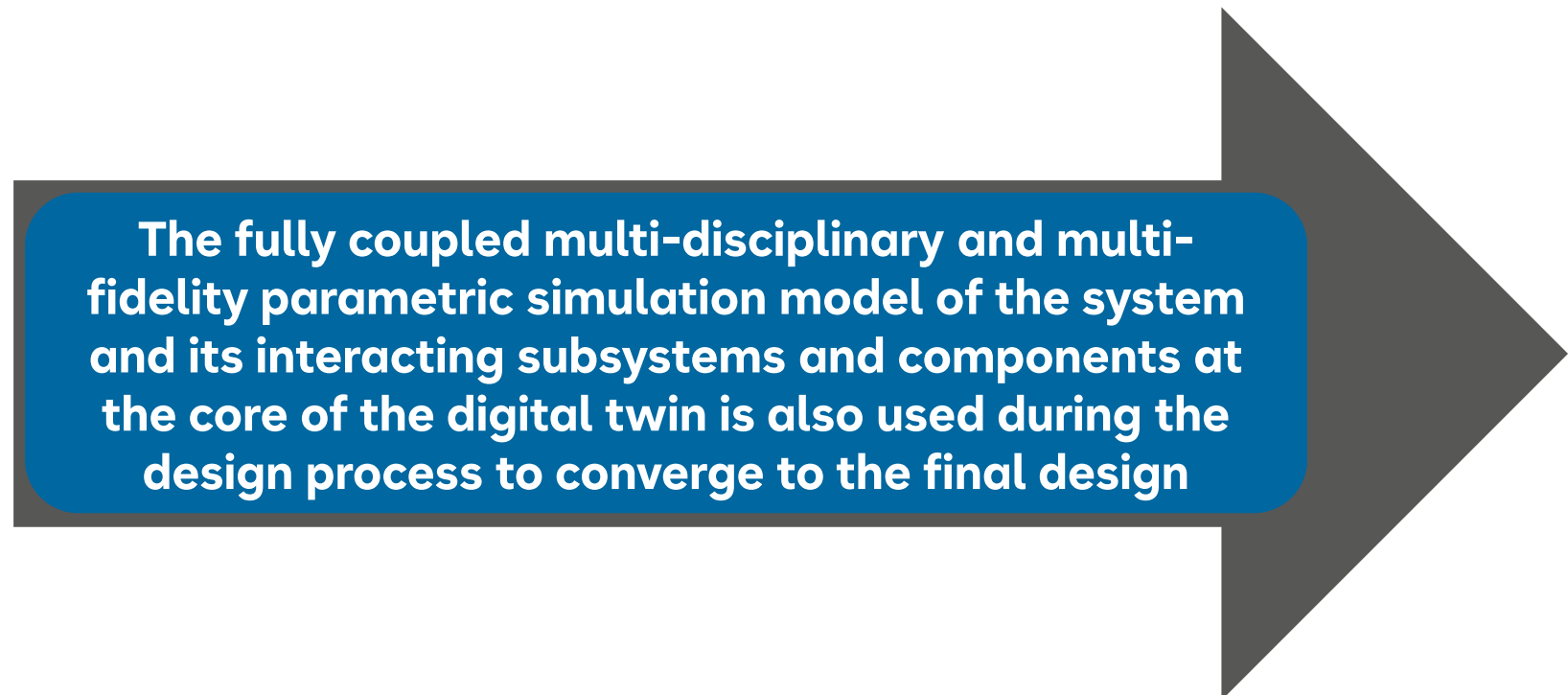
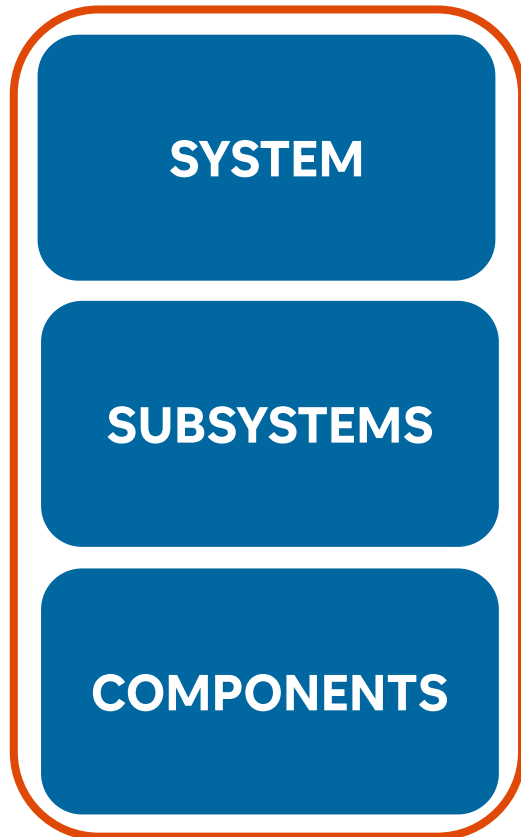
# Integrated Design Systems

*Simulation frameworks for system or component design*





# The engineering design process

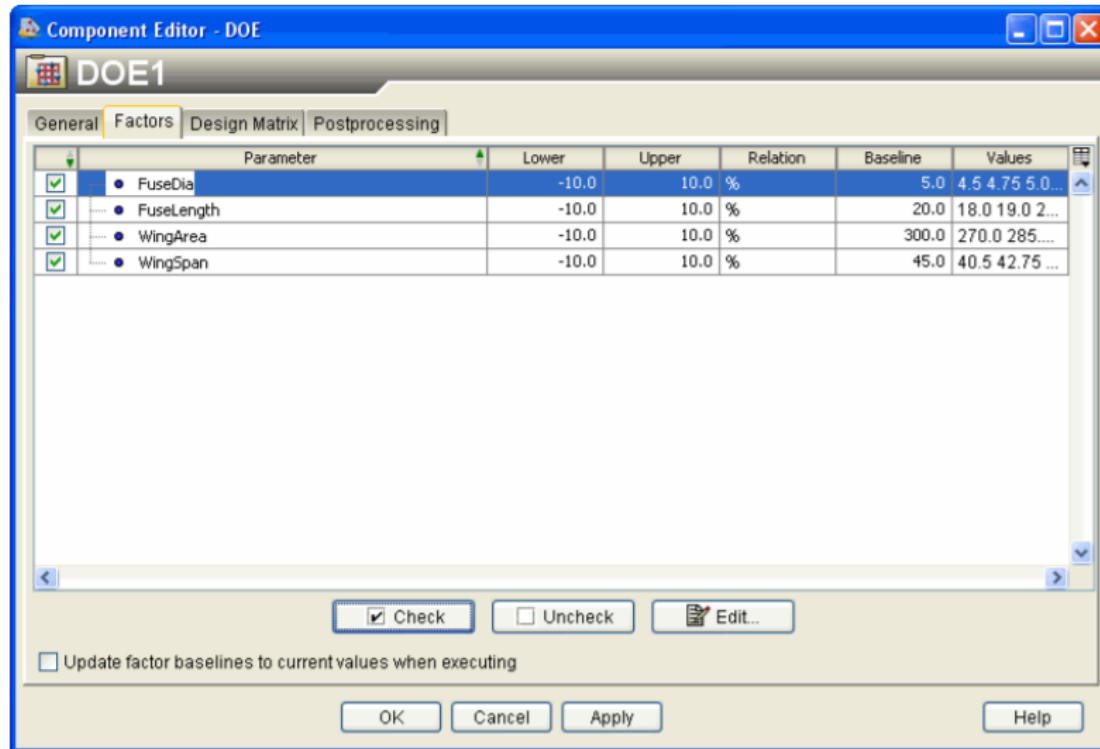


# Robust Design and Optimisation

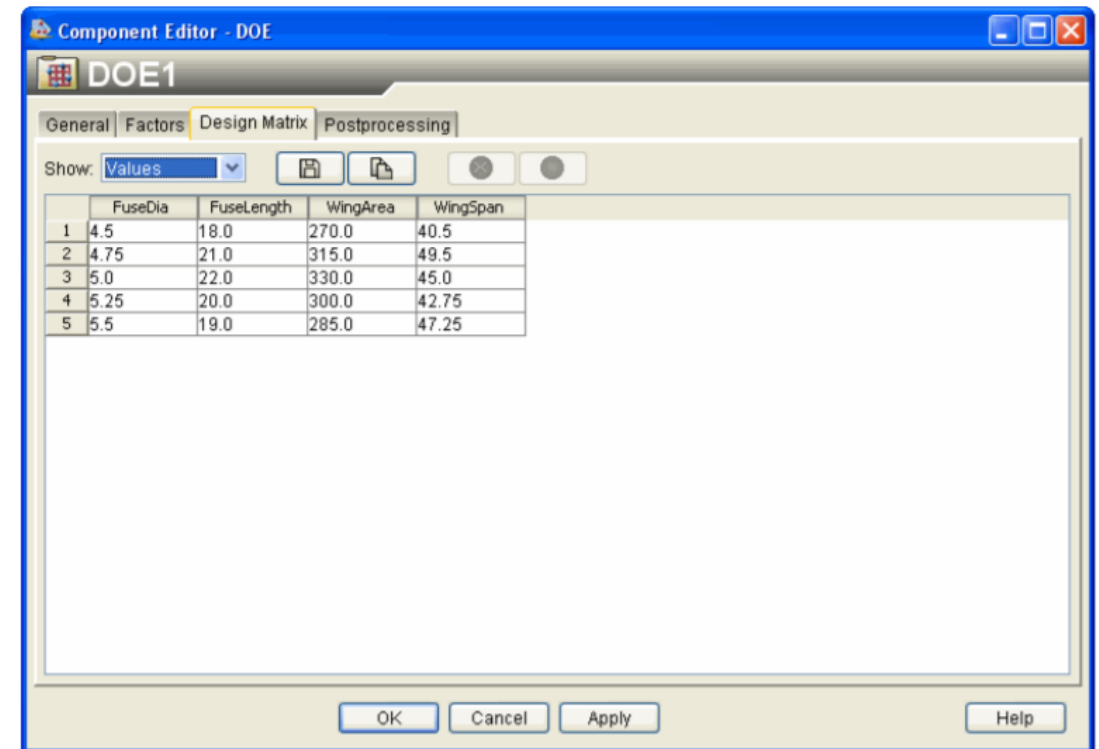
# 12

# Design of Experiments

Design of Experiments (DoEs) are a powerful tool for design space exploration that consists in running multiple configurations of a simulation model, carefully chosen to gather as much information as possible on the outputs and their relationship with the inputs, as well as on the relative importance of the input parameters



Input parameters and ranges of variation



Design matrix

# Types of Design of Experiments (DoEs)

The most common types of DoEs are:

- Full Factorial

If each parameter has a certain number of possible values (also called *levels*), a full factorial DoE will run **all the possible combinations** of all the values for all the parameters. It allows for estimation of all main effects and interactions but it is very computationally expensive

- Fractional Factorial

Less computationally expensive but less information extracted

- Screening Experiments

Few experimental runs. Very low computational cost. It provides information about the key variables

- Latin hypercube

**Optimised choice of values to run.** It includes non-linear effects with a smaller overall number of runs

# Examples of software tools for engineering design studies

- JMP software and the DoE process

[https://www.youtube.com/watch?v=\\_Rgue-7KDww](https://www.youtube.com/watch?v=_Rgue-7KDww)

- Minitab

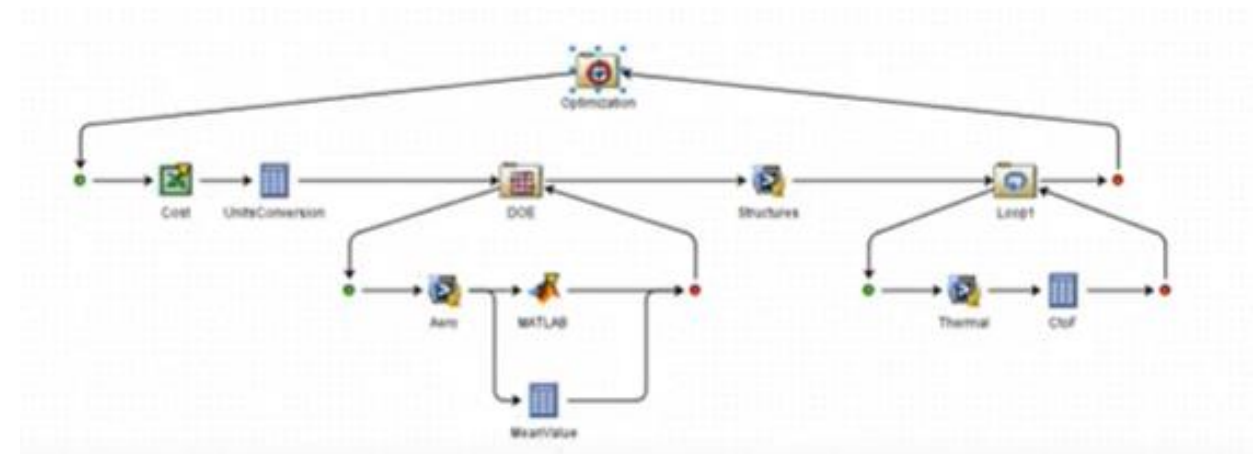
[https://www.youtube.com/watch?v=PD\\_0QjEyQJk](https://www.youtube.com/watch?v=PD_0QjEyQJk)

- Isight

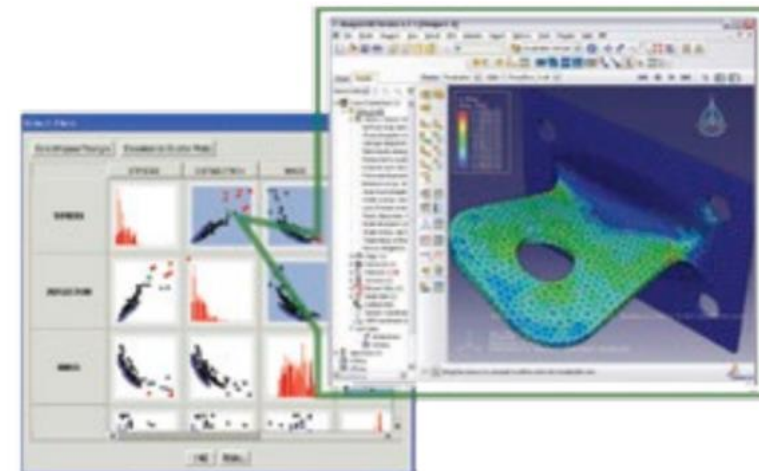
<https://www.youtube.com/watch?v=SwgtZp4Jcjs>

- Heeds

[https://www.youtube.com/watch?v=HvF\\_3Rok8RY](https://www.youtube.com/watch?v=HvF_3Rok8RY)



Process automation



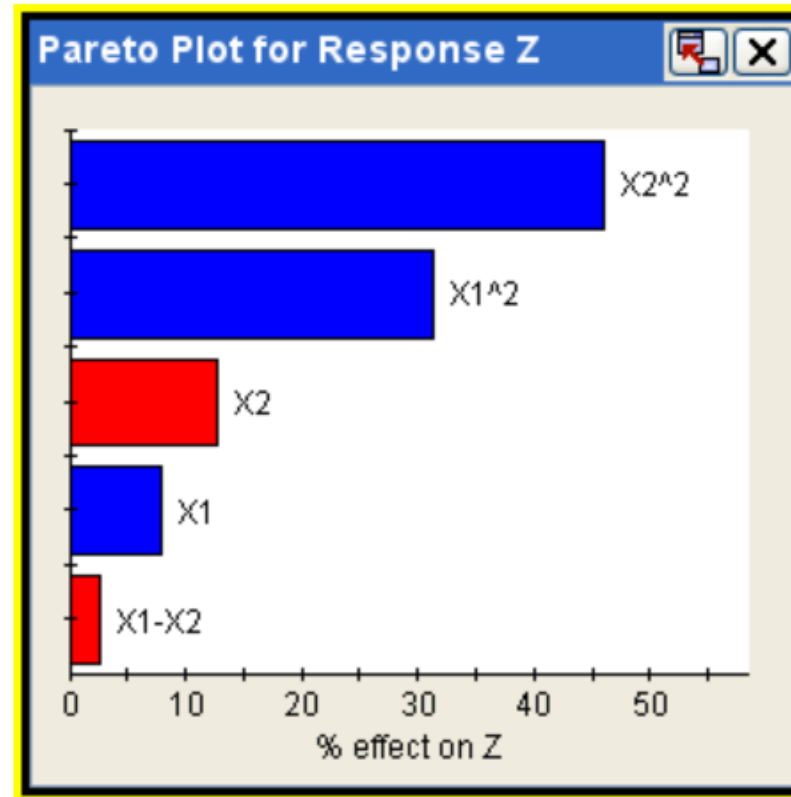
Design studies

# Data post processing and visualisation

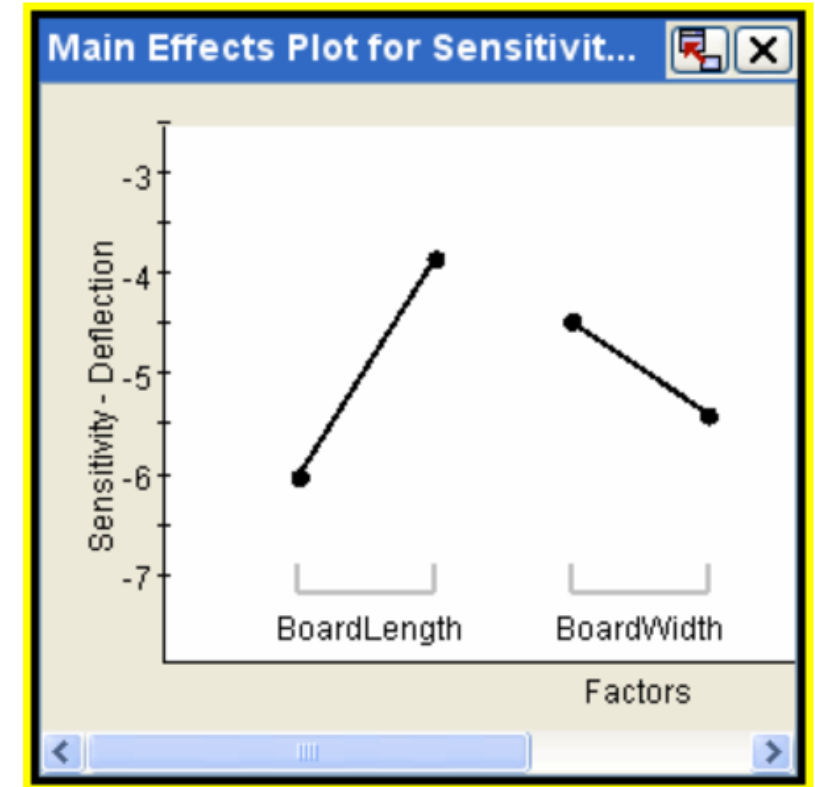
Understanding **sensitivity** and relationships among variables via multiple plots and graphs

- Pareto charts
- Main effects
- Interaction plots
- Parallel axes plots

Understanding **sensitivity**  
is crucial to design  
**robust** products



Pareto chart



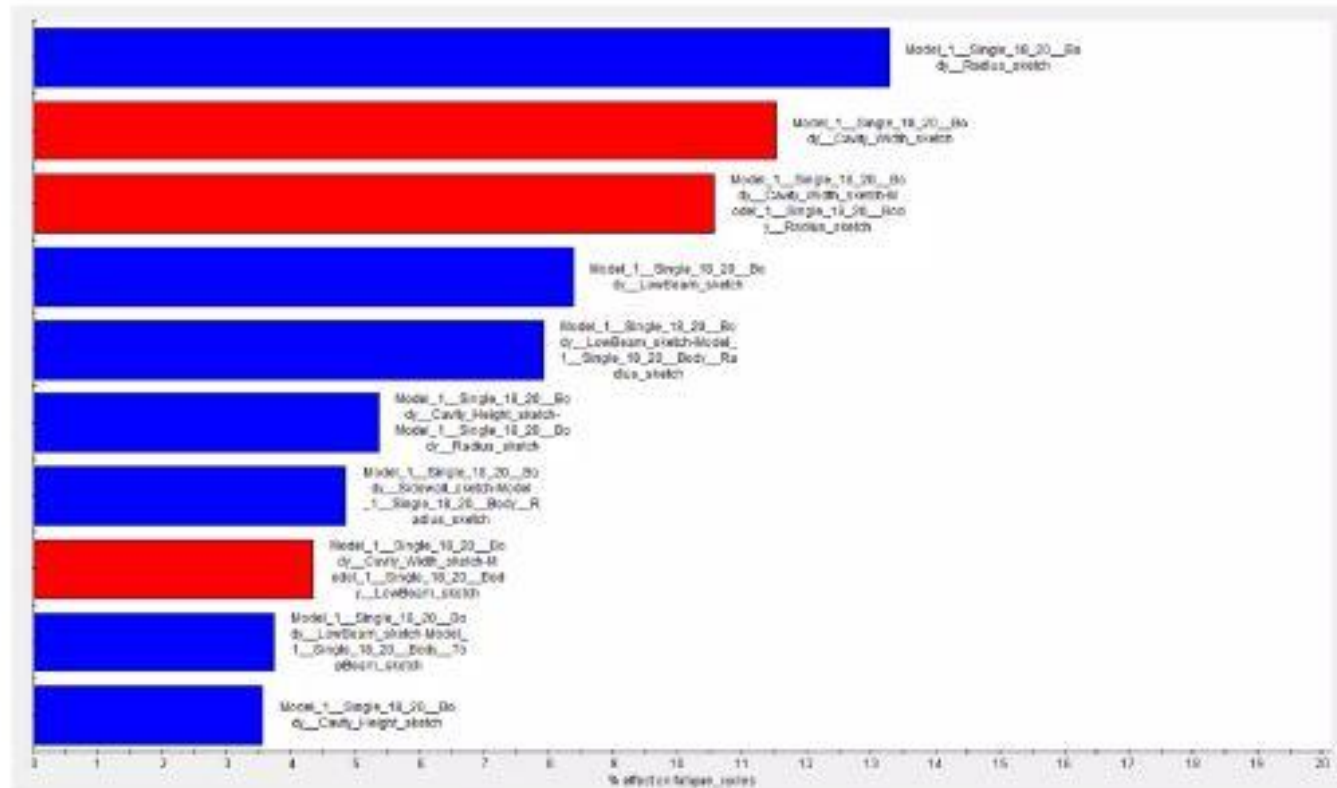
Main effects

# Pareto chart

## Design of Experiments – Step 4 (Isight)



### Pareto Plot



Radius and Cavity Width are the design parameters that have higher effect on the fatigue life of the BOP

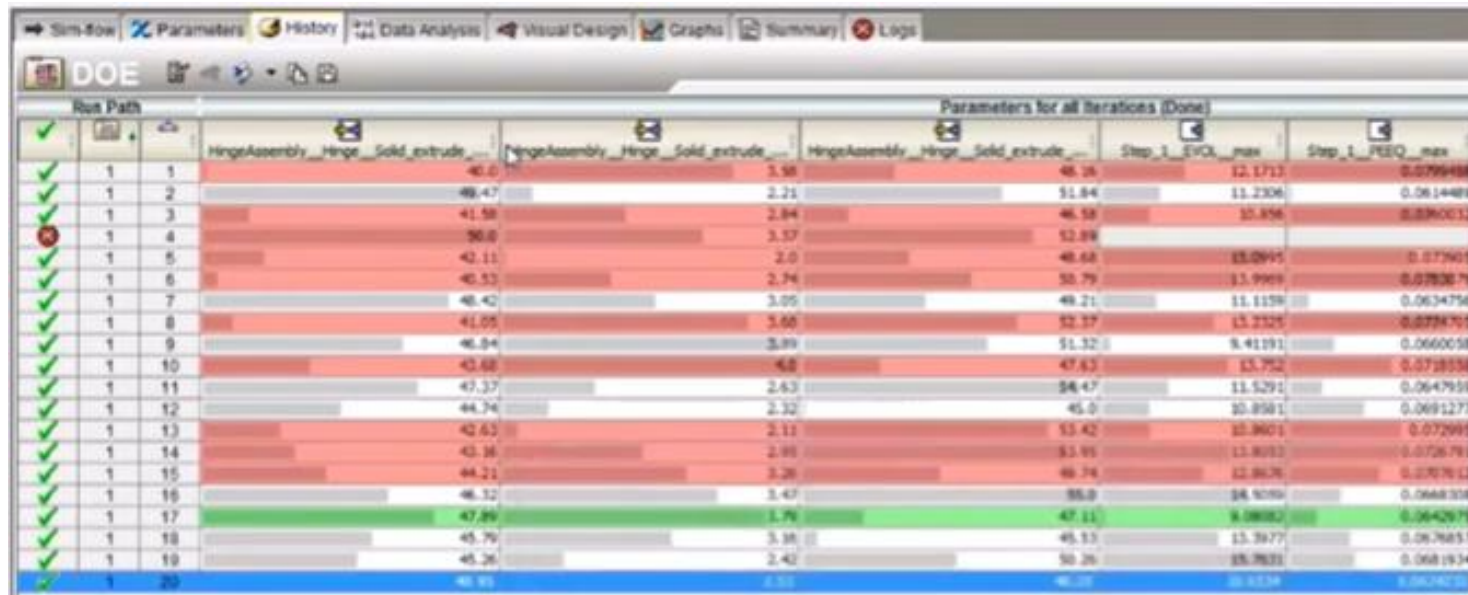
# Design of Experiments and Finite Element Analysis

Design of Experiments can be performed on any predictive model

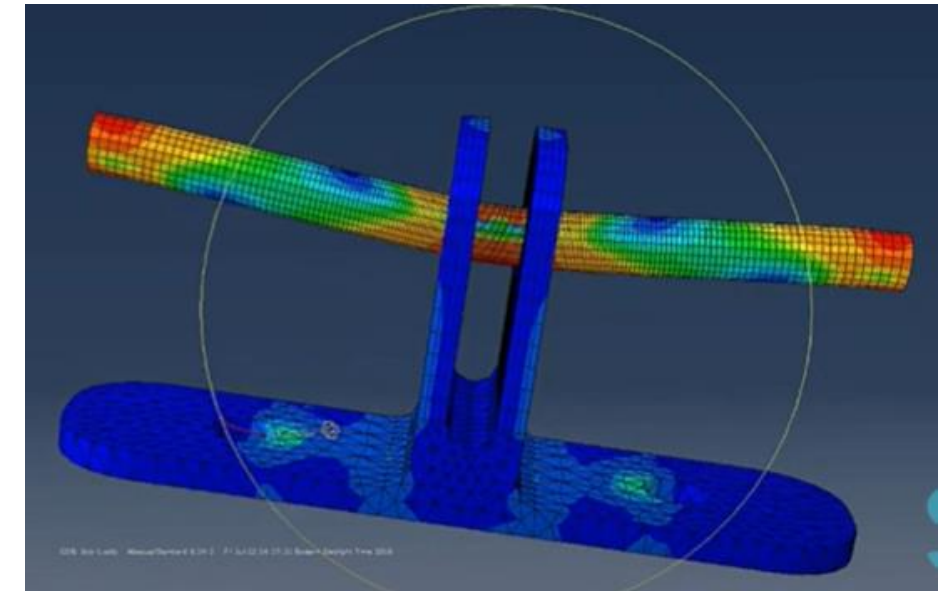
- Example of use of Isight plugin within Abaqus/CAE

[https://www.youtube.com/watch?v=V\\_yaMyLeJ1I](https://www.youtube.com/watch?v=V_yaMyLeJ1I)

<https://www.youtube.com/watch?v=9x9LYvErnwk>



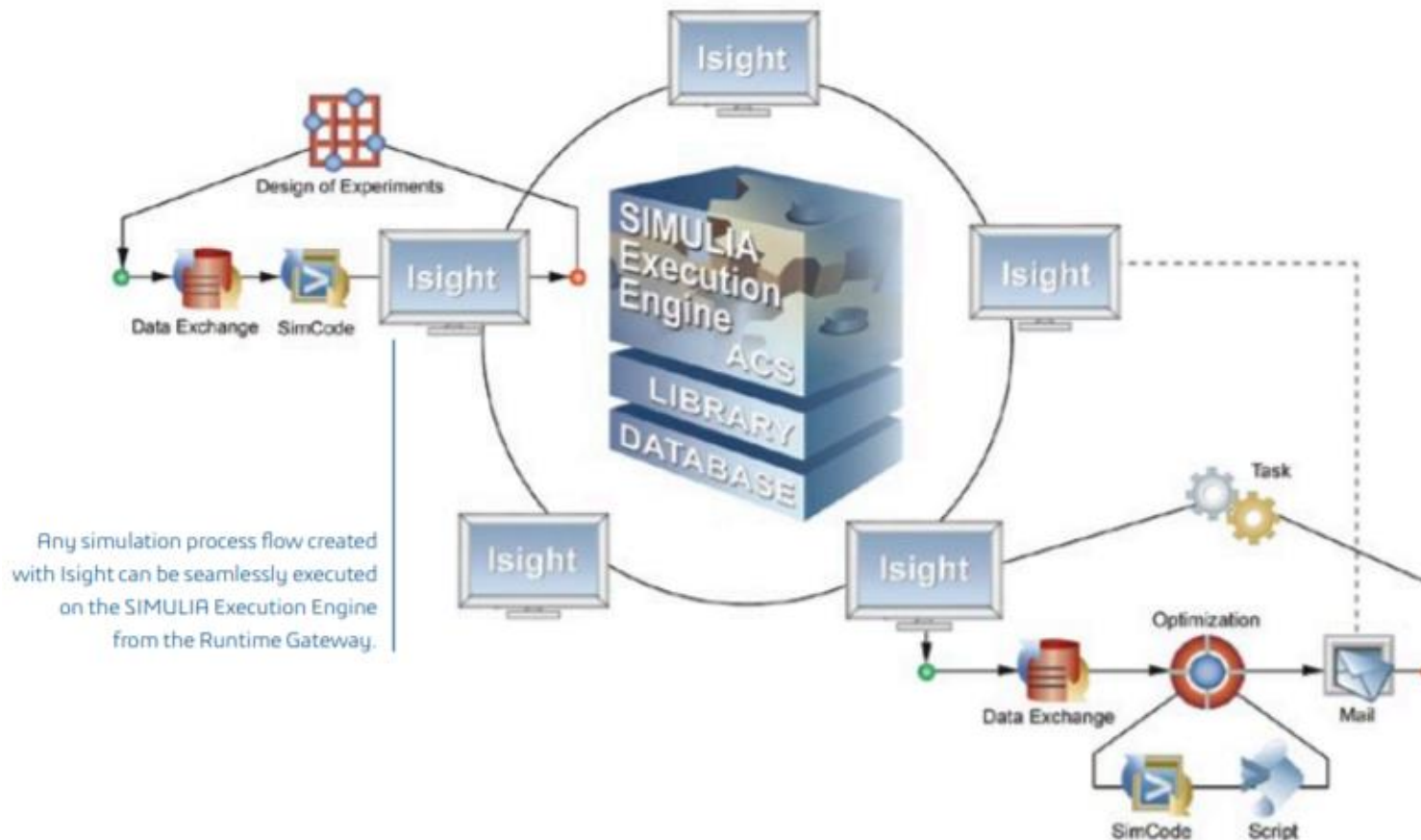
| Run Path |   |    | Parameters for all iterations (Done) |                                      |                                      |                 |                |
|----------|---|----|--------------------------------------|--------------------------------------|--------------------------------------|-----------------|----------------|
|          |   |    | HingeAssembly_Hinge_Solid_extrude... | HingeAssembly_Hinge_Solid_extrude... | HingeAssembly_Hinge_Solid_extrude... | Step_1_Evon_max | Step_1_FEQ_max |
| ✓        | 1 | 1  | 40.0                                 | 3.58                                 | 48.38                                | 12.1713         | 0.0799488      |
| ✓        | 1 | 2  | 48.47                                | 2.21                                 | 51.84                                | 11.2306         | 0.0614489      |
| ✓        | 1 | 3  | 41.58                                | 2.84                                 | 46.58                                | 10.856          | 0.0390012      |
| ✗        | 1 | 4  | 50.0                                 | 3.57                                 | 52.88                                |                 |                |
| ✓        | 1 | 5  | 42.11                                | 2.0                                  | 48.68                                | 10.0945         | 0.077905       |
| ✓        | 1 | 6  | 40.53                                | 2.74                                 | 50.79                                | 13.9969         | 0.0783079      |
| ✓        | 1 | 7  | 40.42                                | 3.05                                 | 49.21                                | 11.1159         | 0.0634756      |
| ✓        | 1 | 8  | 41.05                                | 3.66                                 | 52.37                                | 13.2325         | 0.0794705      |
| ✓        | 1 | 9  | 46.84                                | 3.89                                 | 51.32                                | 8.41191         | 0.0660058      |
| ✓        | 1 | 10 | 43.68                                | 4.8                                  | 47.62                                | 13.752          | 0.0718158      |
| ✓        | 1 | 11 | 47.37                                | 2.63                                 | 54.47                                | 11.5291         | 0.0647959      |
| ✓        | 1 | 12 | 44.74                                | 2.32                                 | 45.0                                 | 10.8581         | 0.0691277      |
| ✓        | 1 | 13 | 42.63                                | 2.11                                 | 53.42                                | 10.3601         | 0.072995       |
| ✓        | 1 | 14 | 43.36                                | 2.95                                 | 53.95                                | 13.8013         | 0.0726791      |
| ✓        | 1 | 15 | 44.21                                | 3.26                                 | 48.74                                | 12.8626         | 0.0707613      |
| ✓        | 1 | 16 | 46.32                                | 3.47                                 | 55.0                                 | 14.9159         | 0.0668338      |
| ✓        | 1 | 17 | 47.89                                | 3.76                                 | 47.11                                | 9.08852         | 0.0642675      |
| ✓        | 1 | 18 | 45.79                                | 3.38                                 | 45.13                                | 15.7977         | 0.0676857      |
| ✓        | 1 | 19 | 45.26                                | 2.42                                 | 50.26                                | 15.7631         | 0.0681934      |
| ✓        | 1 | 20 | 46.95                                | 2.53                                 | 46.25                                | 16.9334         | 0.06274235     |





# Simulation strategies

From **Design of Experiments** to **Optimisation**



# Industry use case example

Further reading:

<https://www.slideshare.net/ArindamChakrabortyPh1/design-optimization-of-bop-for-fatigue-and-strength-in-hpht-environment-using-isight-and-fesafe>

# Design Optimisation

**Design optimisation** is a powerful computational technique that consists in running an iterative process to search the design space for the best design (the *optimum* solution), given one or more design **objectives** (typically the target behaviour for the design) and **constraints**.

Examples:

- I want to design the seal with lowest leakage with a constraint that the mass should not be above  $x$  kg
- I want to design the mug that gives most insulation with a certain constraint on manufacturing cost
- I want to design the engine with minimum specific fuel consumption within all the safety margins

Optimisation can be at **component** level or **system** level (at system level the model is typically more complex)

Optimisation can have more objectives (typically conflicting): **multi-objective optimisation**

Example:

- I want to design the leaf seal with lowest leakage and with minimum heat generation at the interface

In an **optimization problem** the objective is to *optimize* (*maximize* or *minimize*) some function  $f$ . This function  $f$  is called the **objective function**. It is the focal point or goal of our optimization problem.

For example, an objective function  $f$  to be *maximized* may be the revenue in a production of TV sets, the rate of return of a financial portfolio, the yield per minute in a chemical process, the mileage per gallon of a certain type of car, the hourly number of customers served in a bank, the hardness of steel, or the tensile strength of a rope.

Similarly, we may want to *minimize*  $f$  if  $f$  is the cost per unit of producing certain cameras, the operating cost of some power plant, the daily loss of heat in a heating system, CO<sub>2</sub> emissions from a fleet of trucks for freight transport, the idling time of some lathe, or the time needed to produce a fender.

In most optimization problems the objective function  $f$  depends on several variables

$$x_1, \dots, x_n.$$

These are called **control variables** because we can “control” them, that is, choose their values.

For example, the yield of a chemical process may depend on pressure  $x_1$  and temperature  $x_2$ . The efficiency of a certain air-conditioning system may depend on temperature  $x_1$ , air pressure  $x_2$ , moisture content  $x_3$ , cross-sectional area of outlet  $x_4$ , and so on.

Optimization theory develops methods for optimal choices of  $x_1, \dots, x_n$ , which maximize (or minimize) the objective function  $f$ , that is, methods for finding optimal values of  $x_1, \dots, x_n$ .

In many problems the choice of values of  $x_1, \dots, x_n$  is not entirely free but is subject to some **constraints**, that is, additional restrictions arising from the nature of the problem and the variables.

For example, if  $x_1$  is production cost, then  $x_1 \geq 0$ , and there are many other variables (time, weight, distance traveled by a salesman, etc.) that can take nonnegative values only. Constraints can also have the form of equations (instead of inequalities).

Further reading:

# The method of steepest descent (or gradient method)

The idea of this method is to find a minimum of  $f(\mathbf{x})$  by repeatedly computing minima of a function  $g(t)$  of a single variable  $t$ , as follows. Suppose that  $f$  has a minimum at  $\mathbf{X}_0$  and we start at a point  $\mathbf{x}$ . Then we look for a minimum of  $f$  closest to  $\mathbf{x}$  along the straight line in the direction of  $-\nabla f(\mathbf{x})$ , which is the direction of steepest descent (= direction of maximum decrease) of  $f$  at  $\mathbf{x}$ . That is, we determine the value of  $t$  and the corresponding point

$$(2) \quad \mathbf{z}(t) = \mathbf{x} - t\nabla f(\mathbf{x})$$

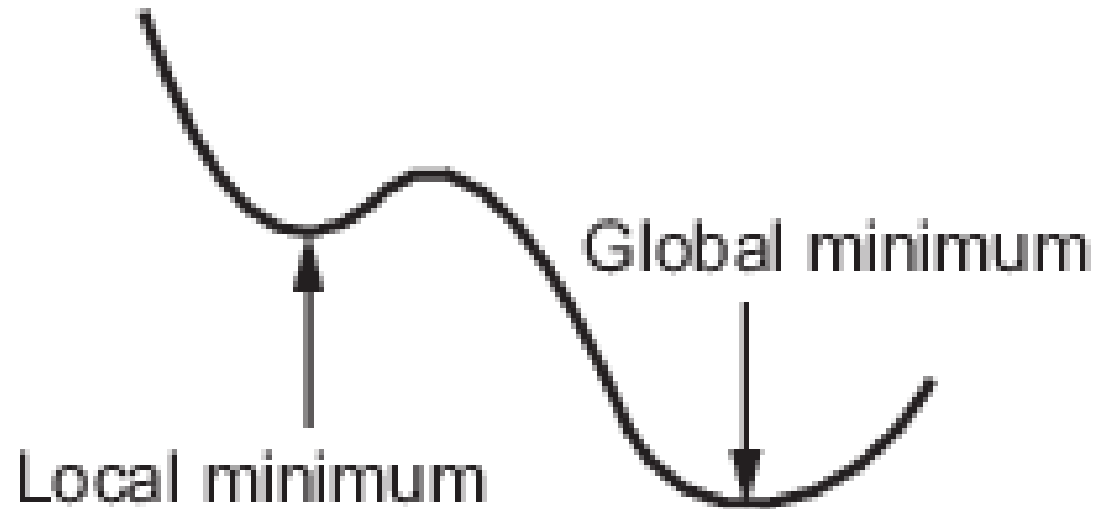
at which the function

$$(3) \quad g(t) = f(\mathbf{z}(t))$$

has a minimum. We take this  $\mathbf{z}(t)$  as our next approximation to  $\mathbf{X}_0$ .

# Global minima vs local minima

The challenge of optimisation is to find the global minimum, or best design



In some engineering problems a local minimum may be '*good enough*'

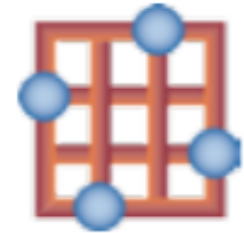


# Design Methods. Example software: Isight

1/2

## Design of Experiments

The DOE component enables engineers to quickly assess the impact of various design variables based on a set of objectives and identify significant interactions. The design data produced by DOE runs can also be used with approximation models for use with optimization methods.



## Optimization

Isight provides a comprehensive selection of parallelized optimization techniques that can be applied to a variety of problems. It also includes techniques that can handle multi-objective optimization problems.



## Data Matching

This process enables simulation models to be calibrated by minimizing any variety of different error measures using optimization techniques. Target data can be imported as experimental or simulation results.

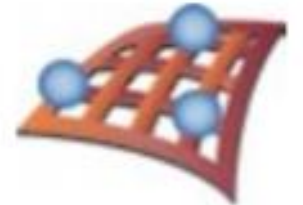


# Design Methods. Example software: Isight

2/2

## Approximations and the Visual Design Driver

Isight offers powerful real-time tools to interpolate results of computationally intensive realistic simulations. Approximation models are automatically cross-validated to ensure accurate predictions. The Visual Design Driver allows users to see their approximation models from many different views and “surf” the design space graphically and interactively.



## Quality Methods

Isight provides stochastic methods that account for variation in product designs and their operating environment. The Monte Carlo Simulation (MCS) component offers an accurate method to address uncertainty and randomness in the design process. It allows users to sample the design space, assess the impact of known uncertainties in input variables on the system responses, and characterize the statistical nature (mean, variance, range, distribution, etc.) of the responses of interest.





# Uncertainty Quantification

**Uncertainty Quantification** is the discipline that allows identifying and understanding sources of uncertainty in an engineering problem and estimating and quantifying the uncertainties in the inputs as well as how they propagate to the uncertainties on the outputs. It allows making predictions of quantities that are not exactly known (all engineering quantities 'in the real world' are not 'exactly' known, hence uncertainty quantification is crucial in engineering)

Understanding and characterising uncertainties is the first step to try and minimise the uncertainties on the outputs of a design (or a simulation), in order to achieve a more **robust** final design

A design is robust if it shows **low sensitivity** (small variation in the outputs) to a variation of input parameters

Examples of sources of uncertainty in engineering:

- Manufacturing tolerances, uncertainties in input operating conditions (e.g. ambient temperature known only within a range), model uncertainties (unknown in model predictions due to limited modelling capability)

Types of uncertainty: statistical uncertainty (e.g. random noise) and systematic uncertainty (e.g. a *bias*, 'offset', introduced by an experimental test setup)

# Uncertainty

Uncertainty Propagation from inputs to outputs and sensitivity

$$(x_1, \dots, x_N) \rightarrow R(x_1, \dots, x_N)$$

$$x_1 \pm \delta x_1 \rightarrow |R(x_1 + \delta x_1, x_2, \dots, x_N) - R(x_1 - \delta x_1, x_2, \dots, x_N)|$$

...

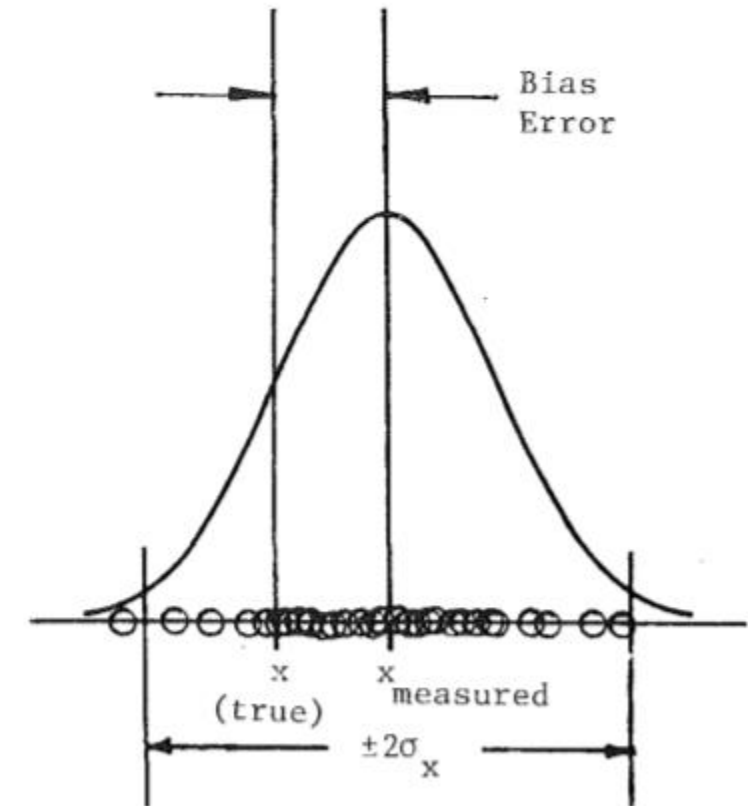
$$x_N \pm \delta x_N \rightarrow |R(x_1, x_2, \dots, x_N + \delta x_N) - R(x_1, x_2, \dots, x_N - \delta x_N)|$$

Combination of uncertainties: root sum square

$$\delta R = \left\{ \left( \frac{\partial R}{\partial x_1} \delta x_1 \right)^2 + \left( \frac{\partial R}{\partial x_2} \delta x_2 \right)^2 + \dots + \left( \frac{\partial R}{\partial x_N} \delta x_N \right)^2 \right\}^{1/2}$$

*The assumption is a normal distribution for each parameter and all parameters independent.*

**Tolerance stack up** calculations typically use a root sum square



# Monte Carlo

Monte Carlo methods are computational techniques often used in engineering that use **random sampling** in order to make predictions on **complex deterministic problems** (for example, **multi-objective optimisation**).

They are particularly useful in solving simulation problems with **highly coupled physics** or/and with **many uncertainties in the inputs**. Hence they are crucial in the modelling of complex engineering systems.

At each **input parameter** is associated a **probability distribution** in relation to its uncertainty (e.g. *Gaussian distribution* or *rectangular distribution*), and randomness is used to select the specific value of that parameter within the probability distribution.

This generates a **stochastic set of inputs** for a series of *runs* of the deterministic problem. Repeating the procedure within optimisation algorithms, allows solving optimisation problems when the inputs have uncertainty (and a probability distribution associated to them).

## Further reading

**Book: Computational Approaches for Aerospace Design –  
Computational Methods and Aerospace Design Use Cases**

[https://books.google.de/books?hl=en&lr=&id=dMeitxe-ilgC&oi=fnd&pg=PR7&dq=andy+keane&ots=ey8qsHEQGr&sig=pQqsnkTrFR2xHutWKD9HgN-9Tig&redir\\_esc=y#v=onepage&q=andy%20keane&f=false](https://books.google.de/books?hl=en&lr=&id=dMeitxe-ilgC&oi=fnd&pg=PR7&dq=andy+keane&ots=ey8qsHEQGr&sig=pQqsnkTrFR2xHutWKD9HgN-9Tig&redir_esc=y#v=onepage&q=andy%20keane&f=false)

# Example of component optimisation

- ASME GT2020-15623: Process Automation for Aero Engines Secondary Air System Seal Design, Dr Adele Nasti, Rolls-Royce Deutschland Ltd & Co KG, presented at ASME Turbo Expo 2020:  
<https://asmedigitalcollection.asme.org/GT/proceedings-abstract/GT2020/84188/V07CT14A023/1095072>

# 13

© 2021 Rolls-Royce plc

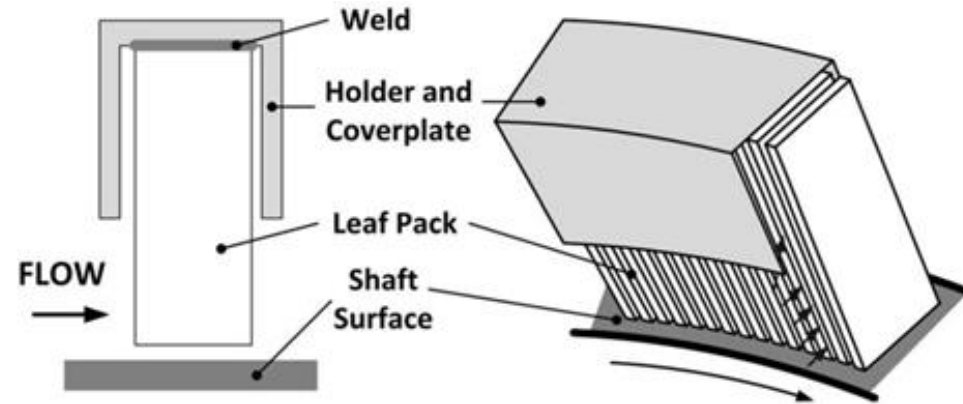
The information in this document is the property of Rolls-Royce plc and may not be copied or communicated to a third party, or used for any purpose other than that for which it is supplied without the express written consent of Rolls-Royce plc.

This information is given in good faith based upon the latest information available to Rolls-Royce plc, no warranty or representation is given concerning such information, which must not be taken as establishing any contractual or other commitment binding upon Rolls-Royce plc or any of its subsidiary or associated companies.

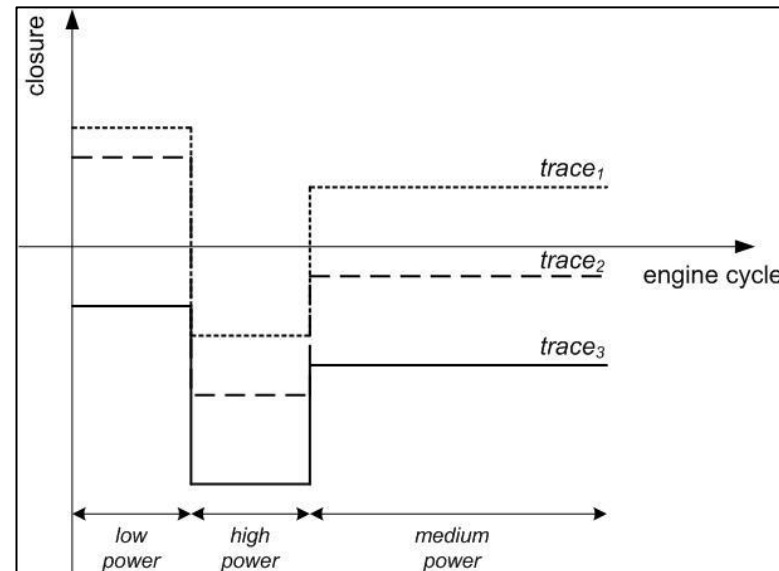
TotalCare®, SelectCare®, Life®, Term®, Flex®, Trent® and UltraFan® are registered trade marks of Rolls-Royce plc.

Non-confidential | Not subject to Export Control | ©Rolls-Royce

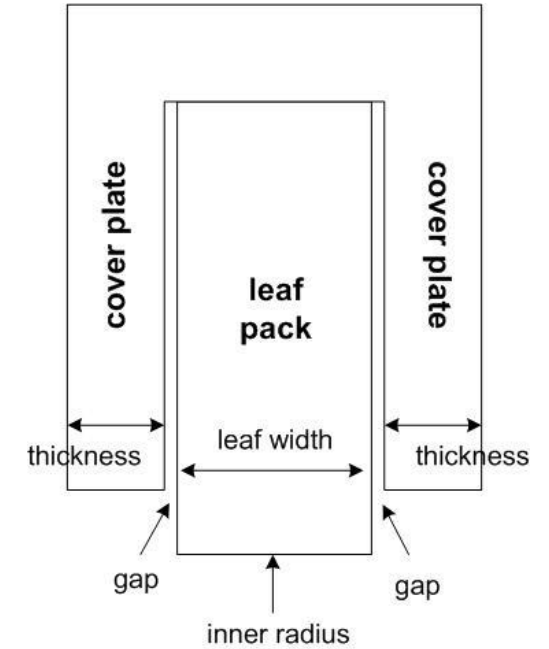
# Optimizing seal design to system behaviour



Leaf seal



Steady-state closure traces over engine cycle



Key geometric parameters



## Examples

### Leaf seal optimized at cruise

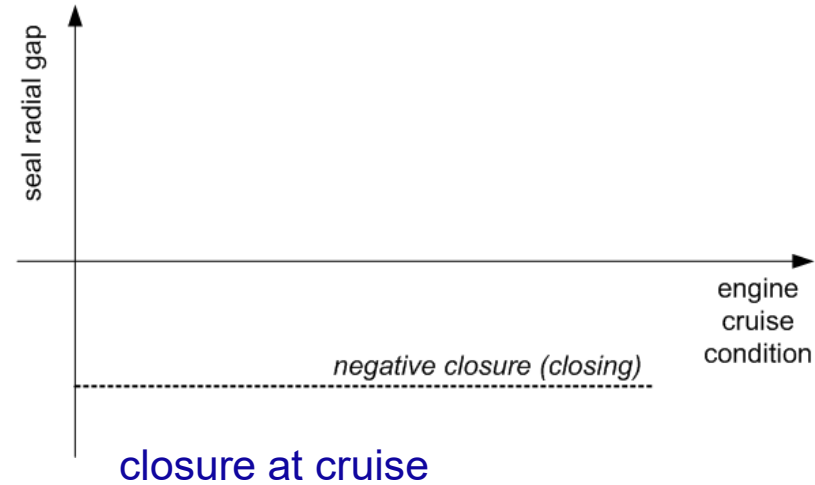
- Key requirement: achieving optimum performance at cruise operating condition, where the seal spends most of its life





## Engine behavior

### Leaf seal optimized at cruise



- If system closure ( $\Delta s - \Delta r$ )  $< 0$  at cruise condition, *closing* system

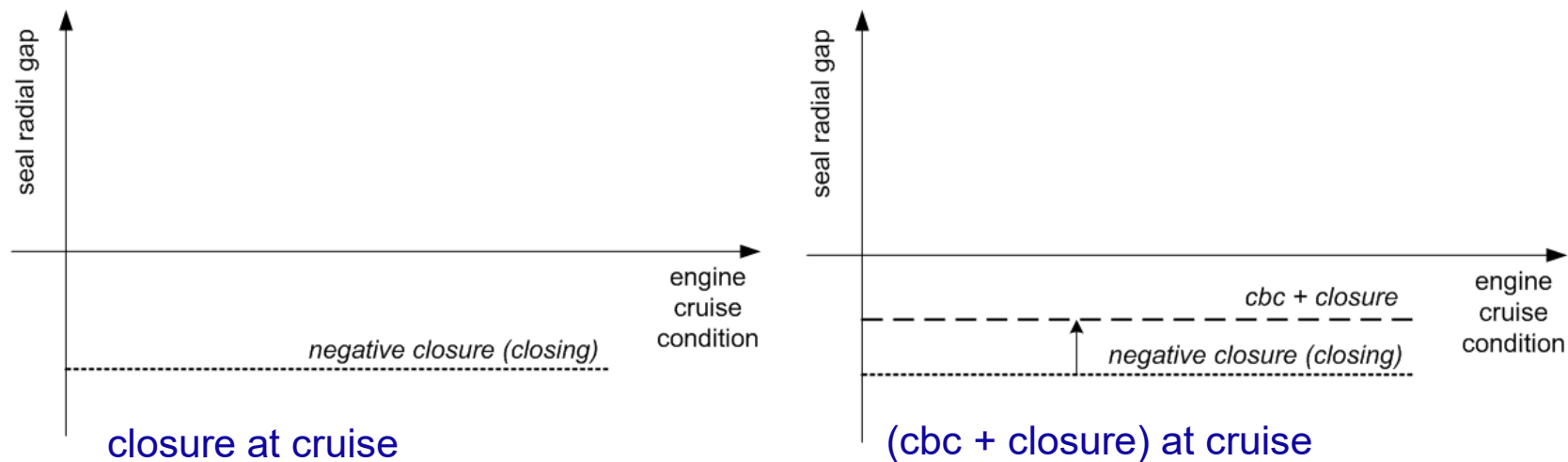


Engine behavior

+

Cold build clearance

## Leaf seal optimized at cruise



- If system closure ( $\Delta s - \Delta r$ )  $< 0$  at cruise condition, *closing* system
- And physical gap = (closure + cold build clearance)  $< 0$  at cruise

## Lift-up leaf seal optimized at cruise

Engine behavior

+

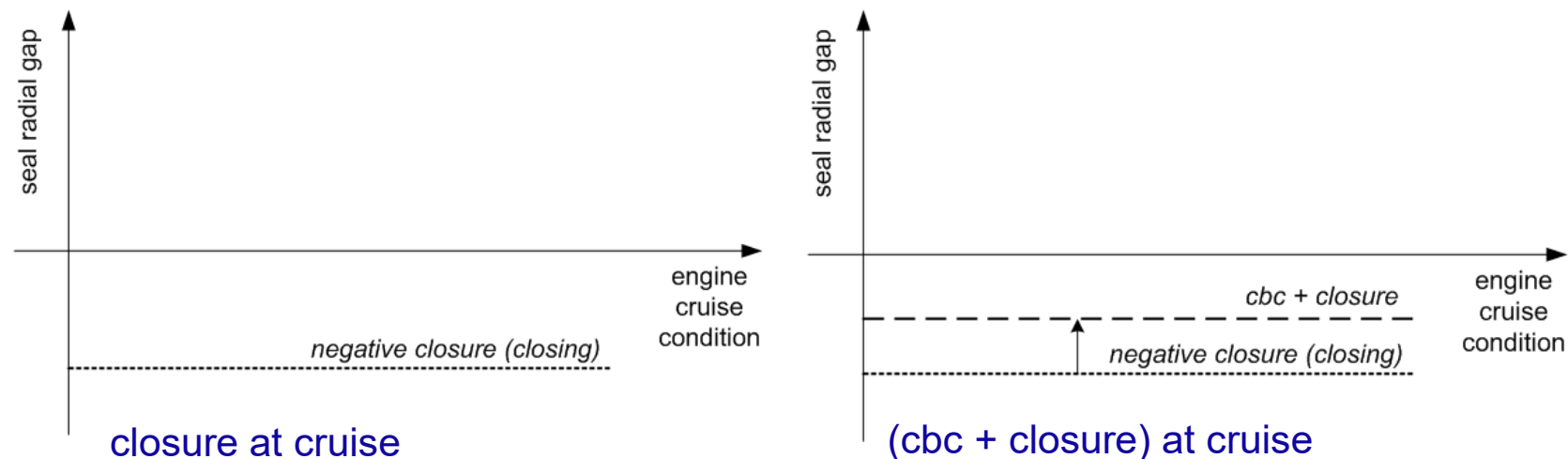
Cold build clearance

+

Lift-up effect

=

Optimum sealing



- If system closure ( $\Delta s - \Delta r$ )  $< 0$  at cruise condition, *closing* system
- And physical gap = (closure + cold build clearance)  $< 0$  at cruise
- Then, in selecting the seal geometric parameters, target a ***lift-up*** at cruise that counterbalances the combined effect of closure and cold build clearance:

$$gap_{liftup} \sim - (cbc + closure)$$



Engine behavior

+

Cold build clearance

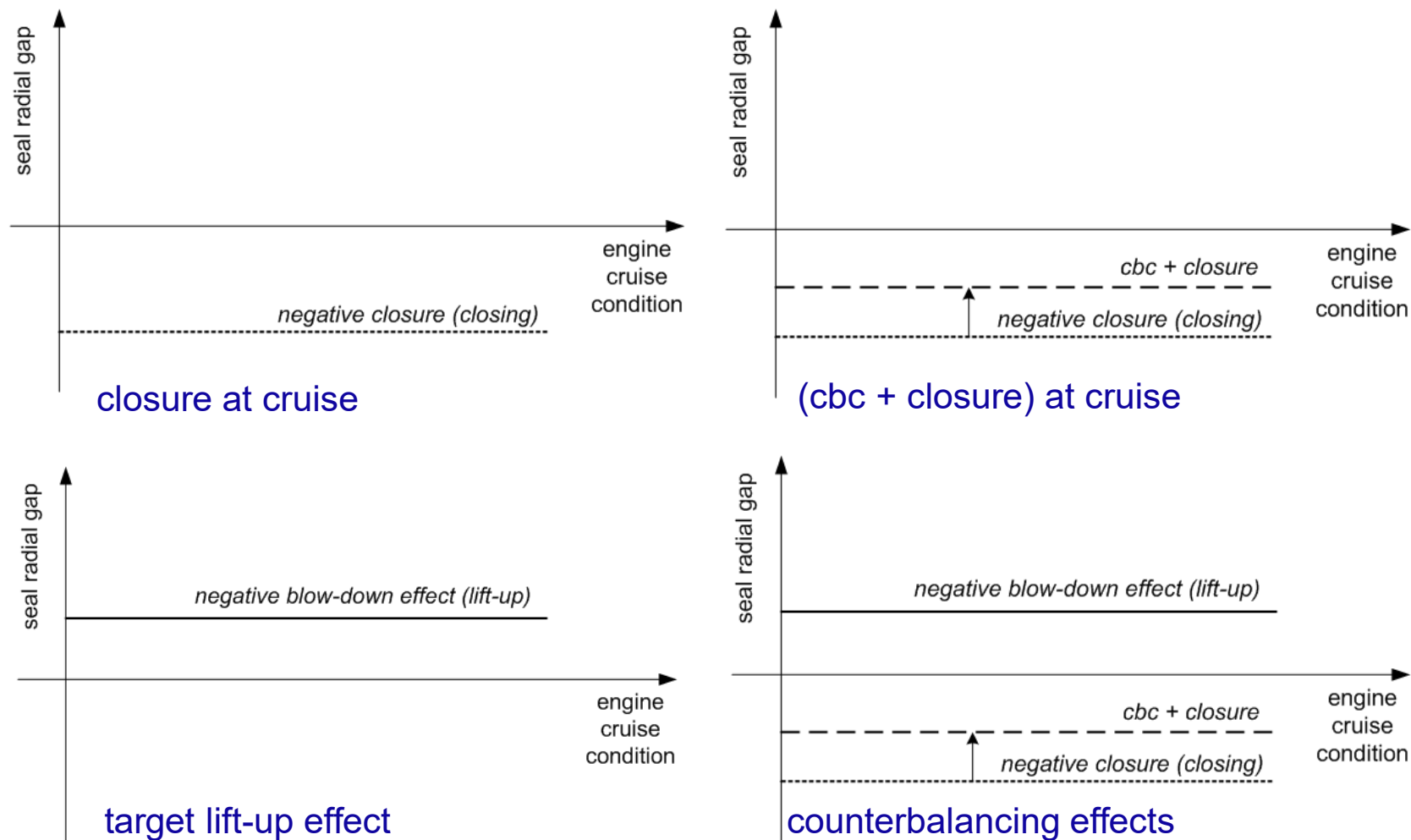
+

Lift-up effect

=

Optimum sealing

## Lift-up leaf seal optimized at cruise

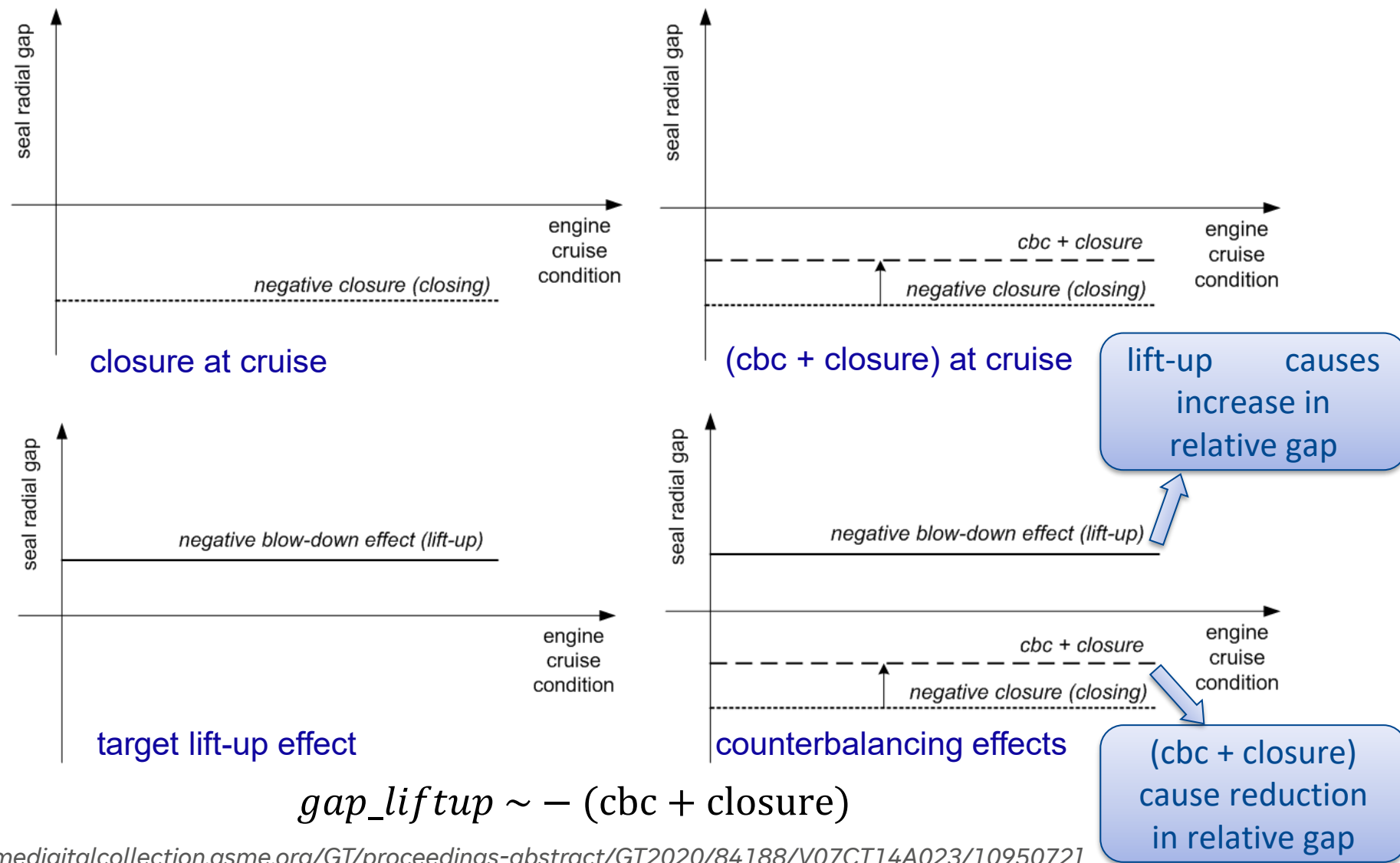


$$gap_{liftup} \sim -(cbc + closure)$$

[<https://asmedigitalcollection.asme.org/GT/proceedings-abstract/GT2020/84188/V07CT14A023/1095072>]

# Lift-up leaf seal optimized at cruise

Engine behavior  
+  
Cold build clearance  
+  
Lift-up effect  
=  
Optimum sealing

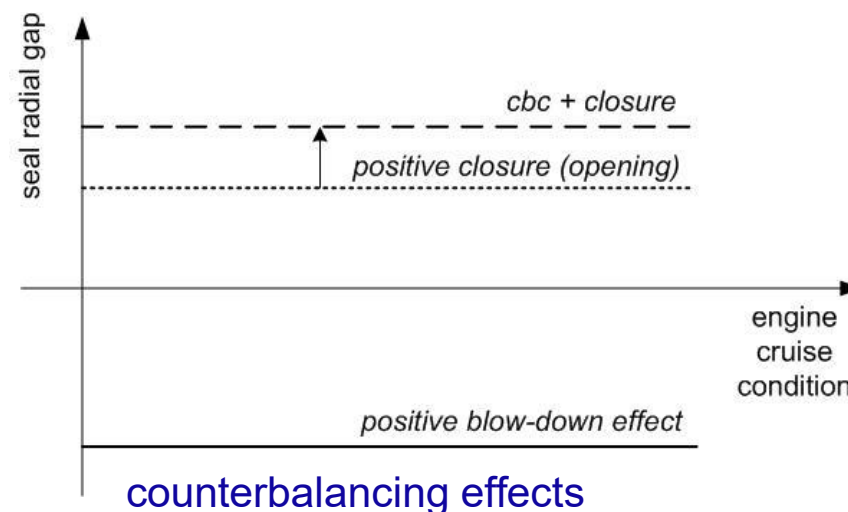
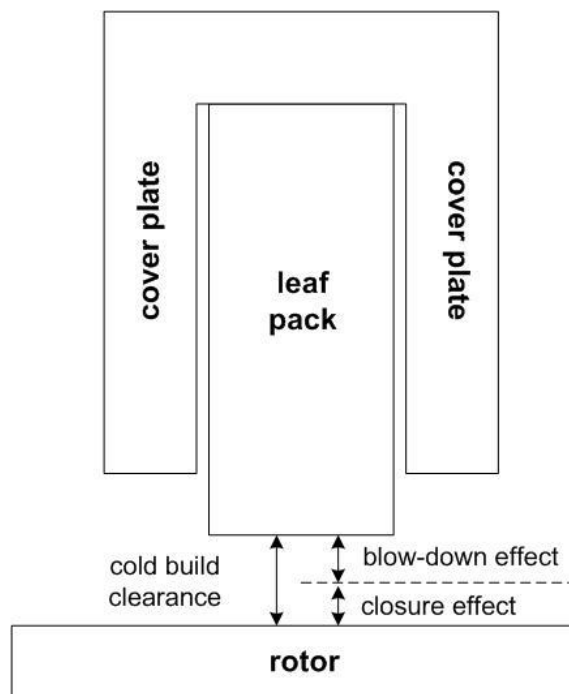


Engine behavior  
+  
Cold build clearance  
+  
Blow-down effect  
=  
Optimum sealing

## Blow-down leaf seal optimized at cruise

- If system closure ( $\Delta s - \Delta r$ )  $> 0$  at cruise condition, *opening* system
- Then, in selecting the seal geometric parameters, target a **blow-down** at cruise that counterbalances the combined effect of closure and cold build clearance:

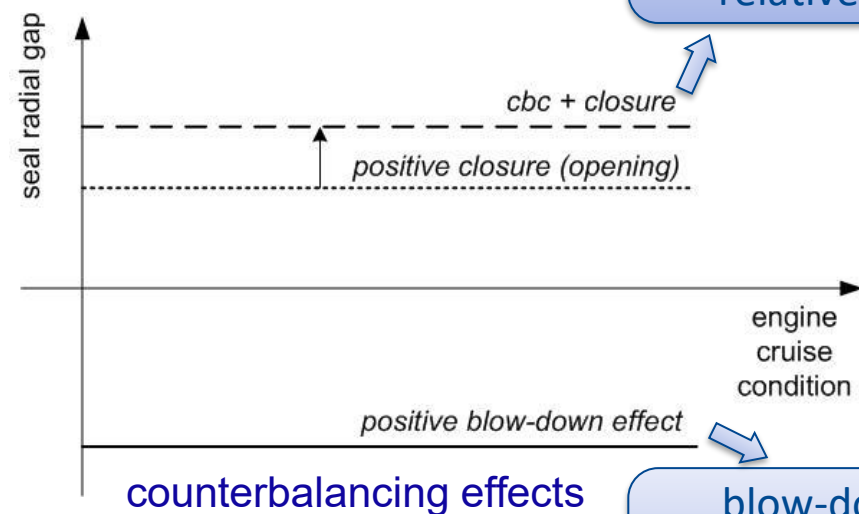
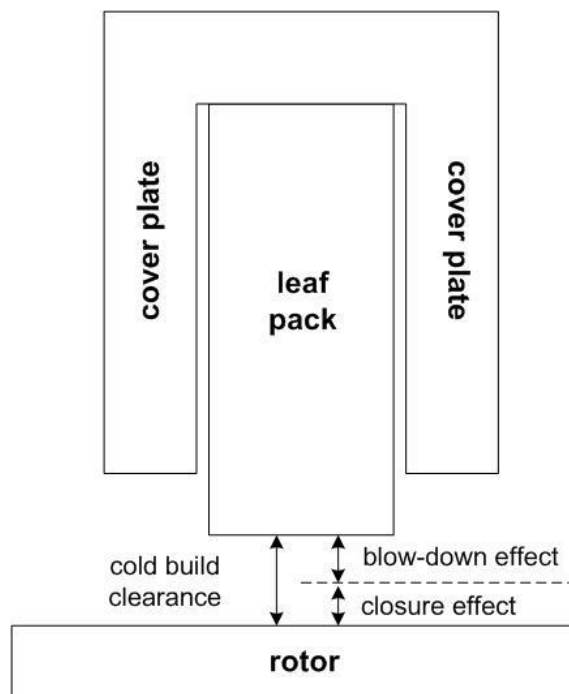
$$gap\_blowdown \sim - (cbc + closure)$$



## Blow-down leaf seal optimized at cruise

- If system closure ( $\Delta s - \Delta r$ )  $> 0$  at cruise condition, *opening* system
- Then, in selecting the seal geometric parameters, target a **blow-down** at cruise that counterbalances the combined effect of closure and cold build clearance:

$$gap\_blowdown \sim - (cbc + closure)$$



Engine behavior

+

Cold build clearance

+

Blow-down effect

=

Optimum sealing

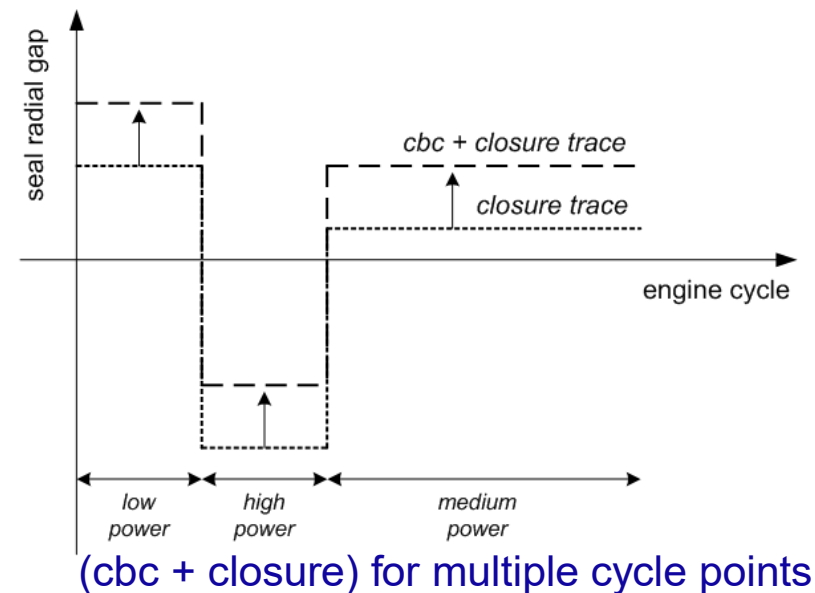
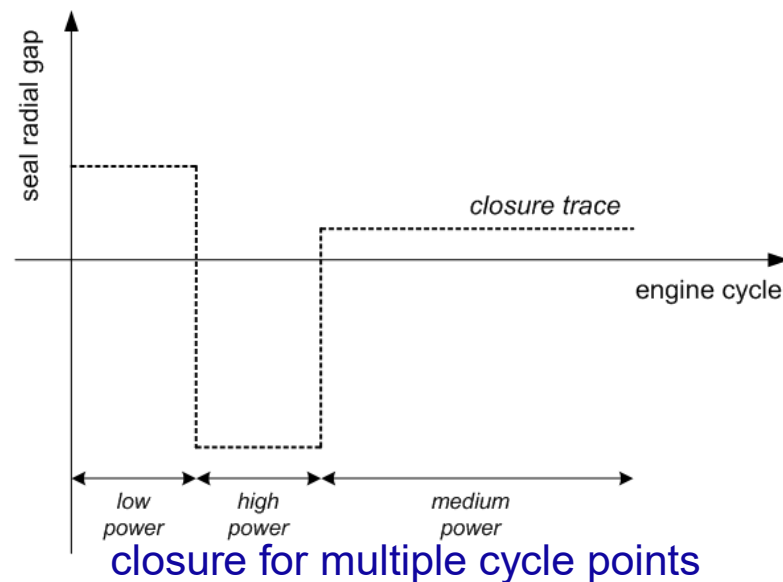


Engine behavior

+

Cold build clearance

## Leaf seal optimized at multiple operating conditions







Engine behavior

+

Cold build clearance

+

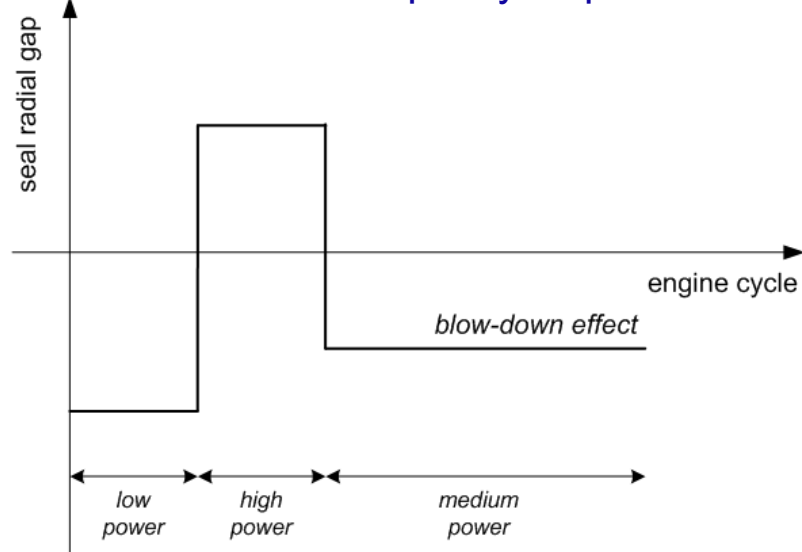
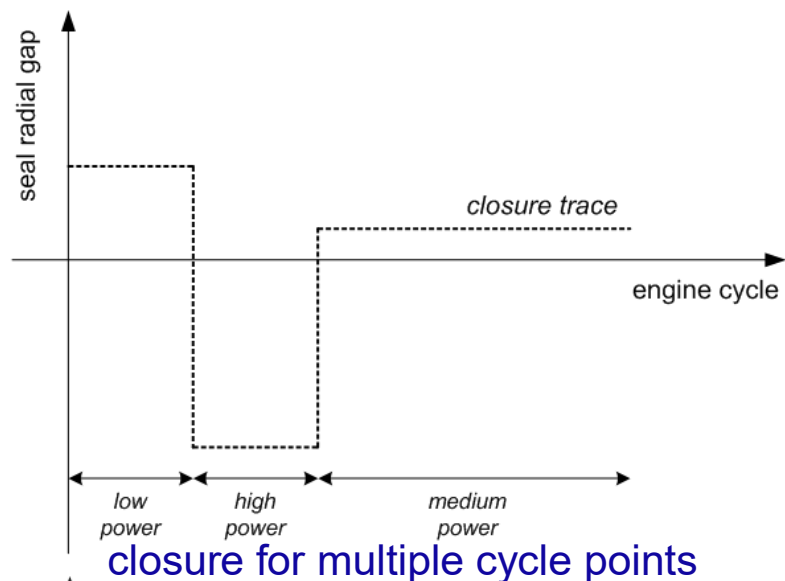
Blow-down effect

=

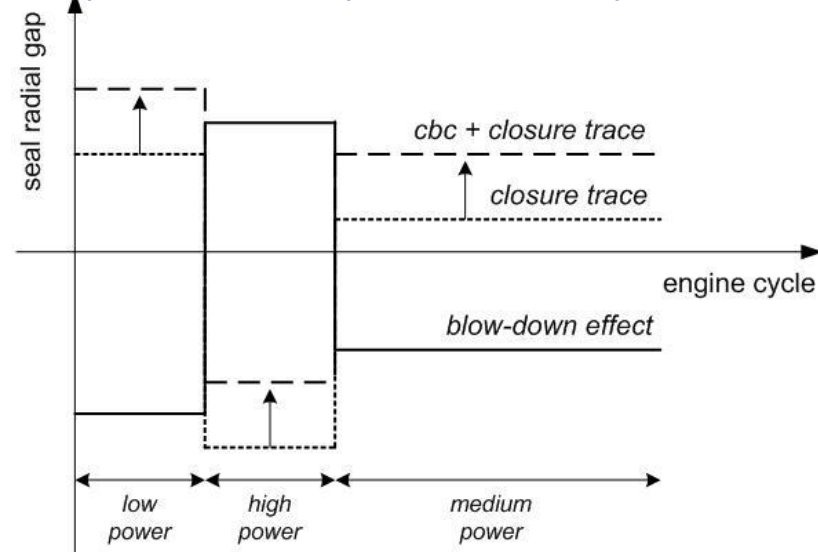
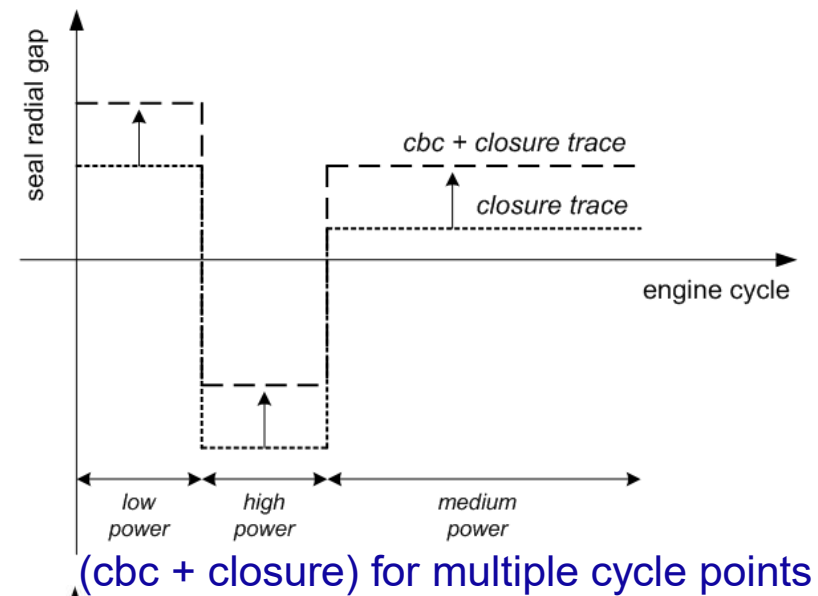
Optimum sealing

$$gap_{blowdown} \sim \\ -(cbc + closure)$$

## Leaf seal optimized at multiple operating conditions



target blow-down/lift-up effect

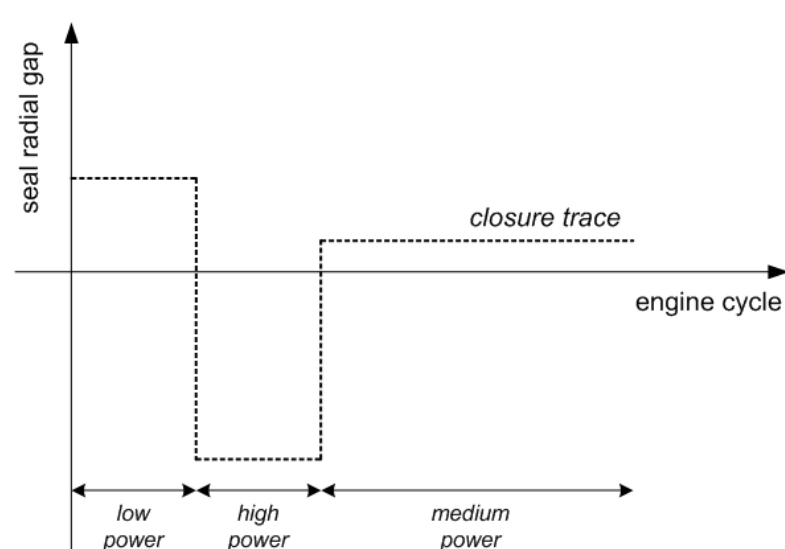


counterbalancing effects

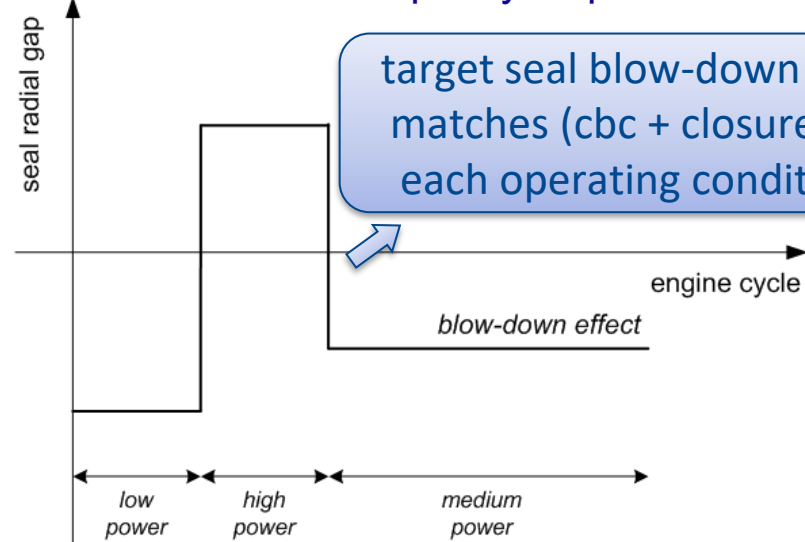
Engine behavior  
+  
Cold build clearance  
+  
Blow-down effect  
=  
Optimum sealing

$$gap_{blowdown} \sim -(cbc + closure)$$

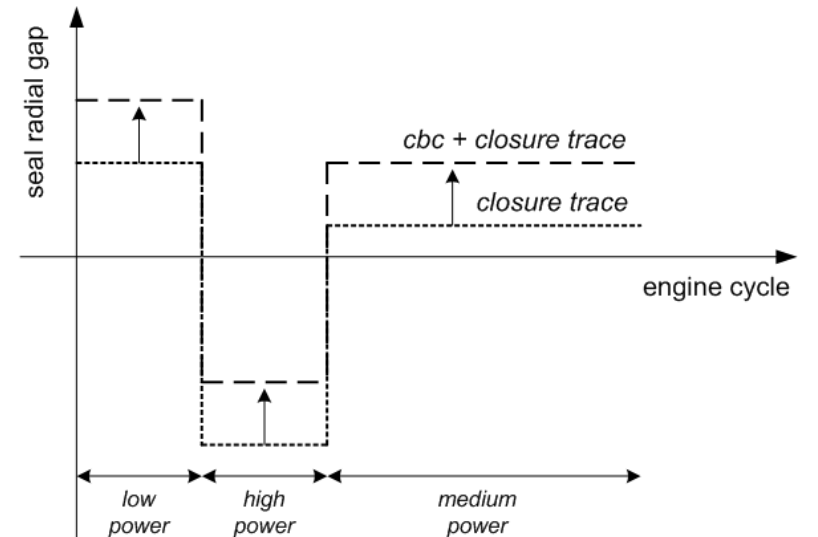
## Leaf seal optimized at multiple operating conditions



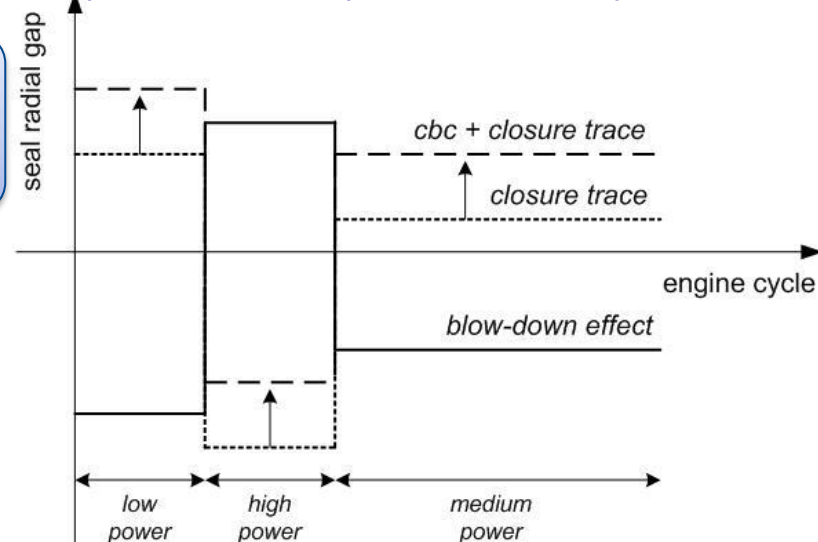
closure for multiple cycle points



target blow-down/lift-up effect



(cbc + closure) for multiple cycle points

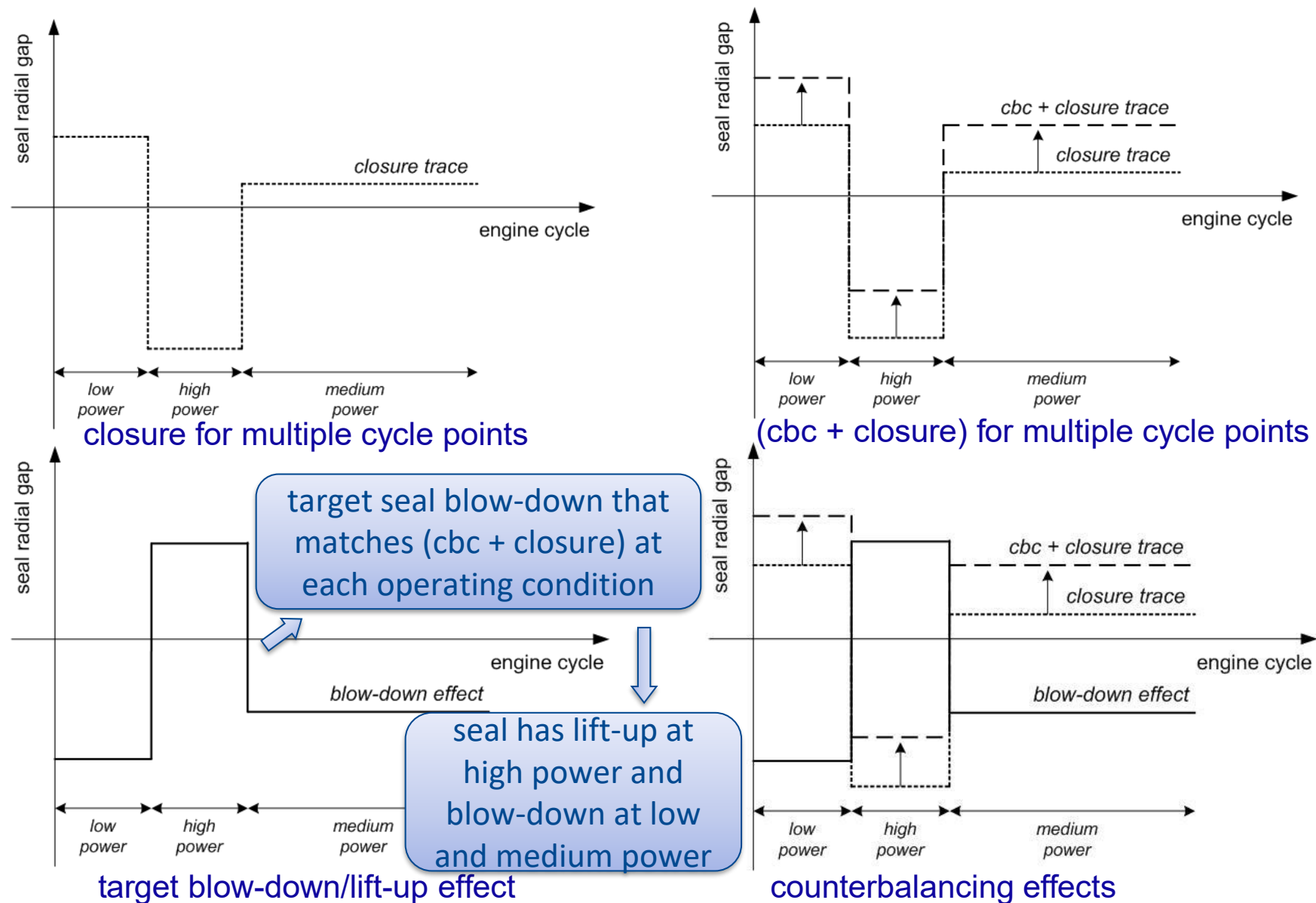


counterbalancing effects

Engine behavior  
+  
Cold build clearance  
+  
Blow-down effect  
=  
Optimum sealing

$$gap_{blowdown} \sim -(cbc + closure)$$

## Leaf seal optimized at multiple operating conditions





## Examples

# Optimization requirements

- Seal optimized at cruise
- Seal optimized at multiple operating conditions
  - Different weights for different operating conditions
- Seal optimized for end of life performance
- Maximize performance retention
  - Minimize performance degradation
- Robust optimization
  - Robustness to all sources of variations
- Prioritize manufacturability, cost or seal re-use in multiple locations
- Optimize a system of more than one seal
- Minimize sensitivity to system inputs
- Holistic air system design

**coupling or de-coupling seal & system to meet design intent**

# Example of system-level DoE

- ASME GT2020-16079: Novel feature based approach for turbomachinery design and analysis, Nasti et al, Rolls-Royce, presented at ASME Turbo Expo 2020:  
<https://asmedigitalcollection.asme.org/GT/proceedings-abstract/GT2020/84089/V02CT35A060/1094573>

# 14

© 2021 Rolls-Royce plc

The information in this document is the property of Rolls-Royce plc and may not be copied or communicated to a third party, or used for any purpose other than that for which it is supplied without the express written consent of Rolls-Royce plc.

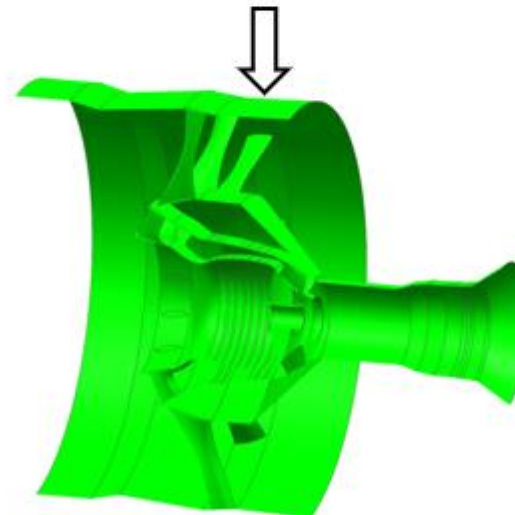
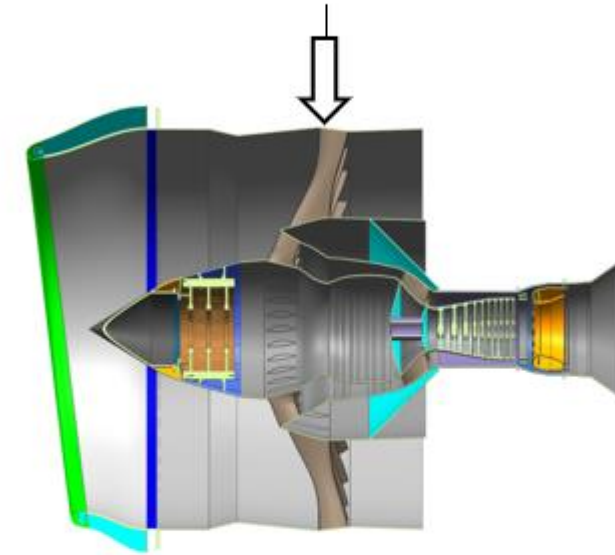
This information is given in good faith based upon the latest information available to Rolls-Royce plc, no warranty or representation is given concerning such information, which must not be taken as establishing any contractual or other commitment binding upon Rolls-Royce plc or any of its subsidiary or associated companies.

TotalCare®, SelectCare®, Life®, Term®, Flex®, Trent® and UltraFan® are registered trade marks of Rolls-Royce plc.

Non-confidential | Not subject to Export Control | ©Rolls-Royce

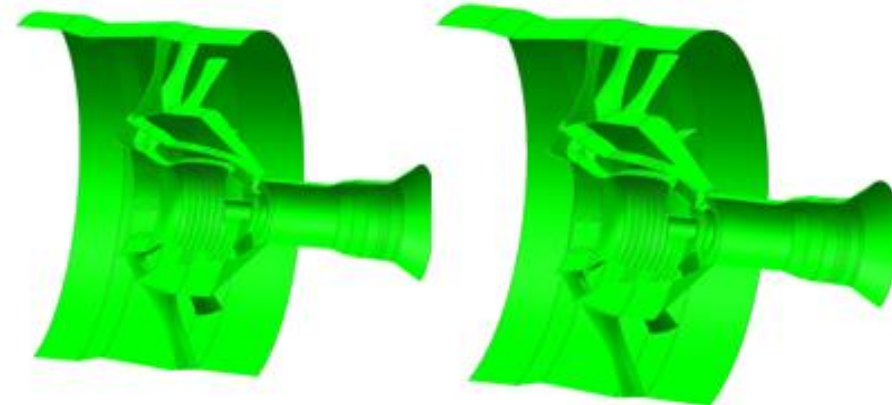
# Whole Engine Design

- Inputs
- Gas path
- Parametric CAD
- Geometry idealisation
- Shell & Beam model
- Finite element analysis
- Post processing and data visualisation
- Trade studies



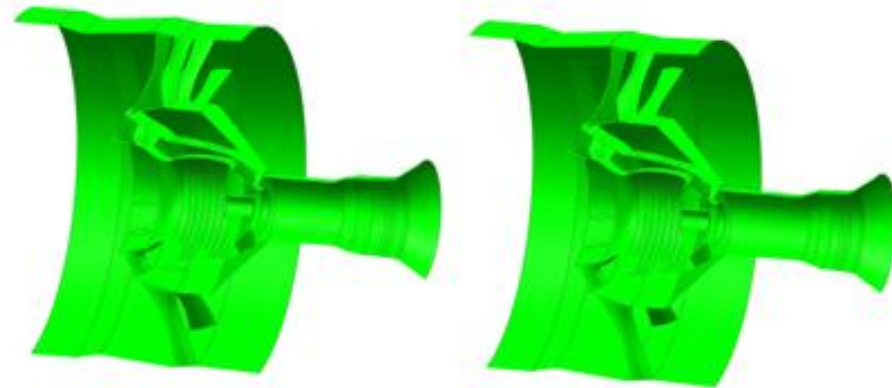
Geometry idealisation

# Design of experiments via parametric geometry



(a) Nominal model

(b) Fan casing thickness increased to 10 times the nominal value

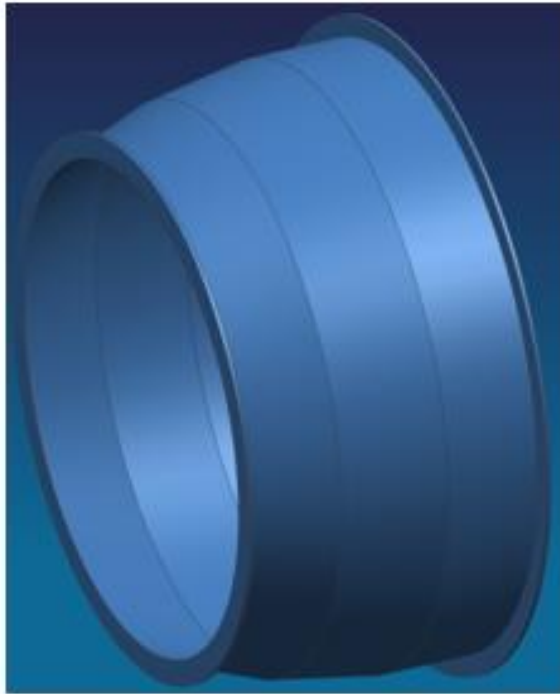


(c) Number of struts increased by 25%

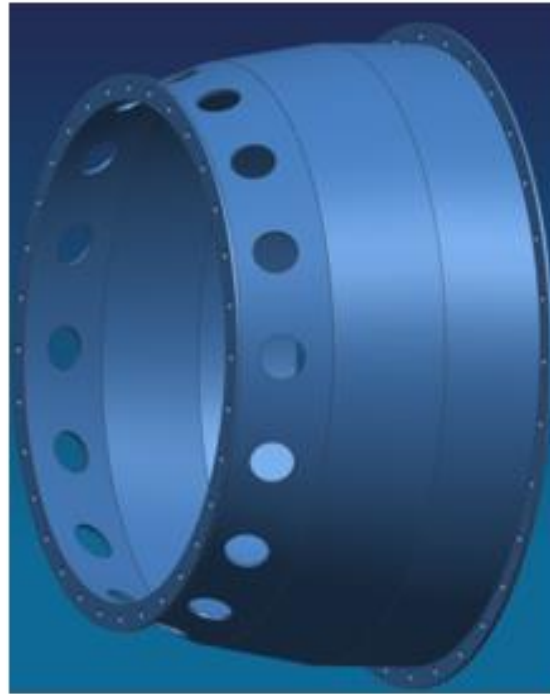
(d) CCOC Front flange moved 20mm downstream



# Geometry fidelity in different design stages



Stage 0 - Conceptual Design  
(a)



Stage 1 - Product Planning  
(b)



Stage 2 - Detailed Design  
(c)



# Finite Element Analysis

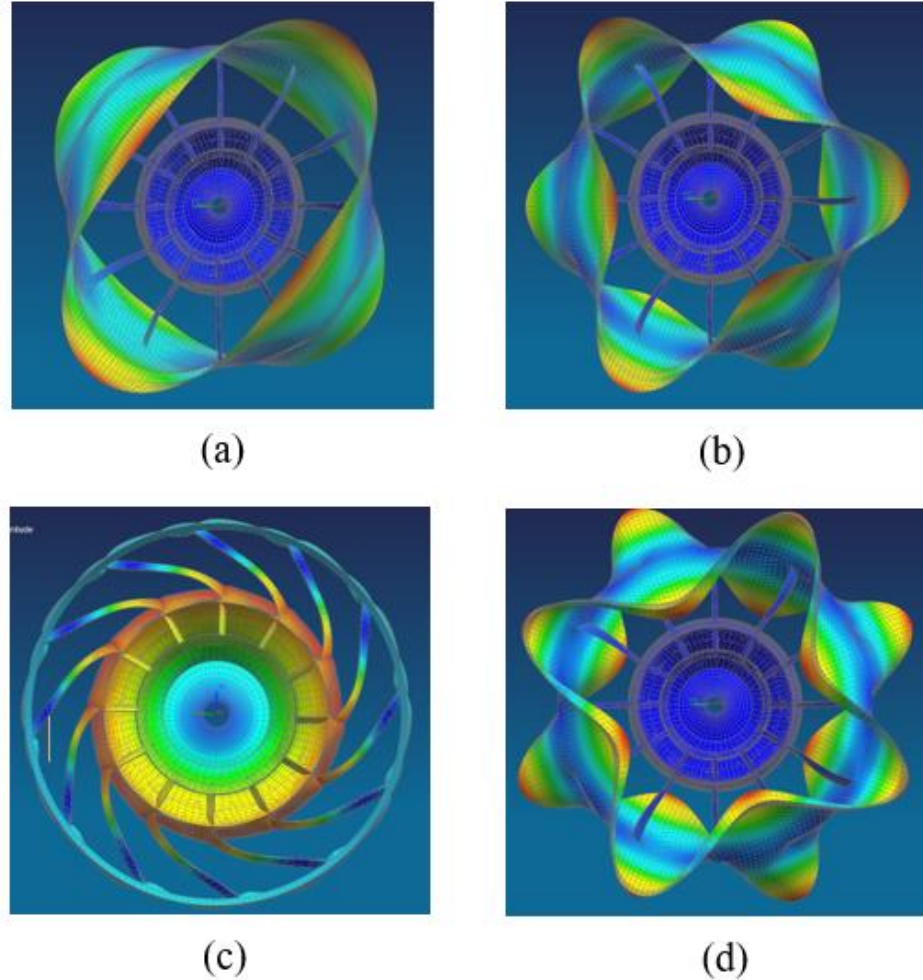


Figure 13. First few Eigen modes of the whole engine model

# Finite Element Analysis

# 15

# Finite Element Analysis

**Finite Element Analysis** is a numerical technique to solve the partial differential equations that regulate the behaviour of a body (a component, a sub-system or a system) under certain conditions and obtain approximate predictions for key physical quantities like temperature, displacement, stress. It is particularly used in **structural analysis**, **heat transfer** and for **fluid flow** predictions.

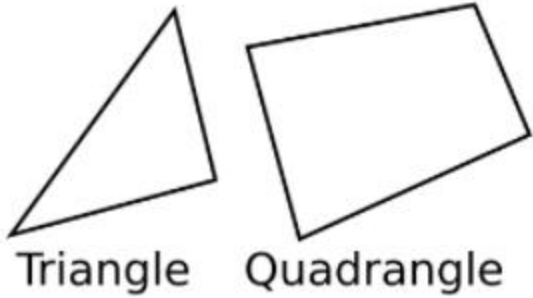
**The Finite Element Method** is based on a discretisation of the body into smaller parts (finite elements), that constitutes the **mesh** of the object. The equations that govern the finite elements constitute a large system of equations, that are typically solved via simulation software via minimisation of an error function, and provide an approximate set of results for the overall problem.

**Steady state** simulation vs **transient** simulation: stabilised solution vs behaviour over time

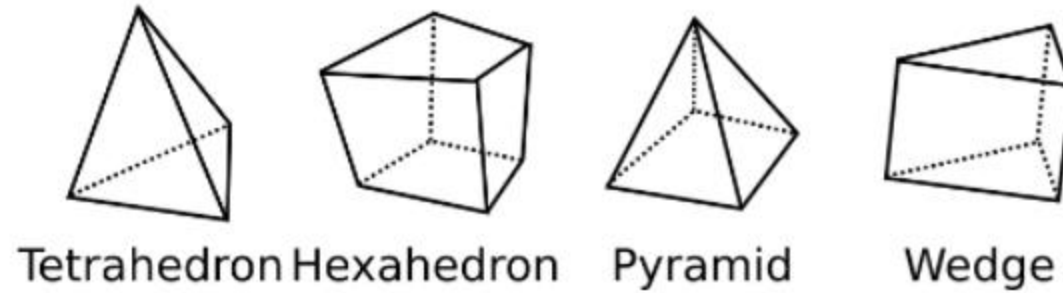
# Meshing

Common types of meshes

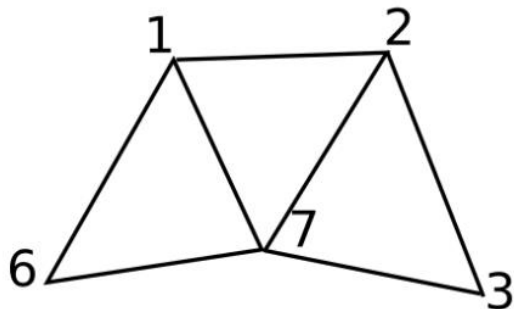
## 2-Dimensional



## 3-Dimensional

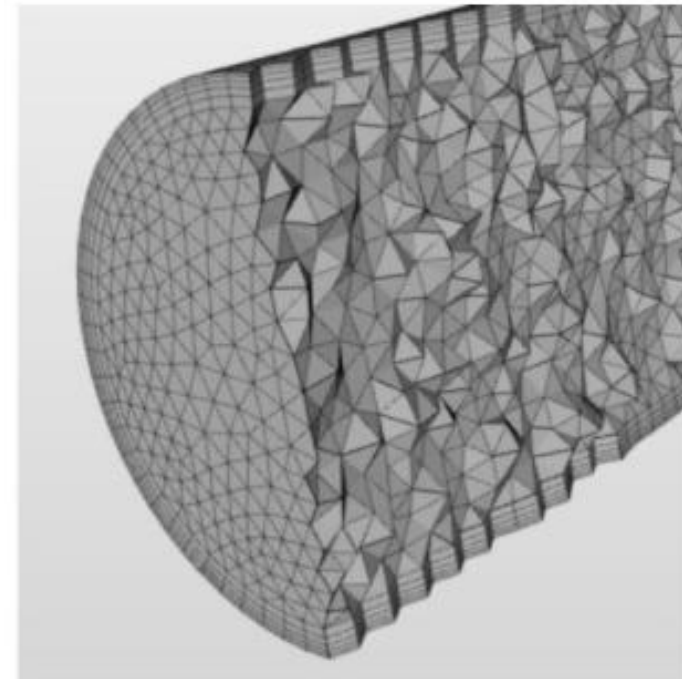
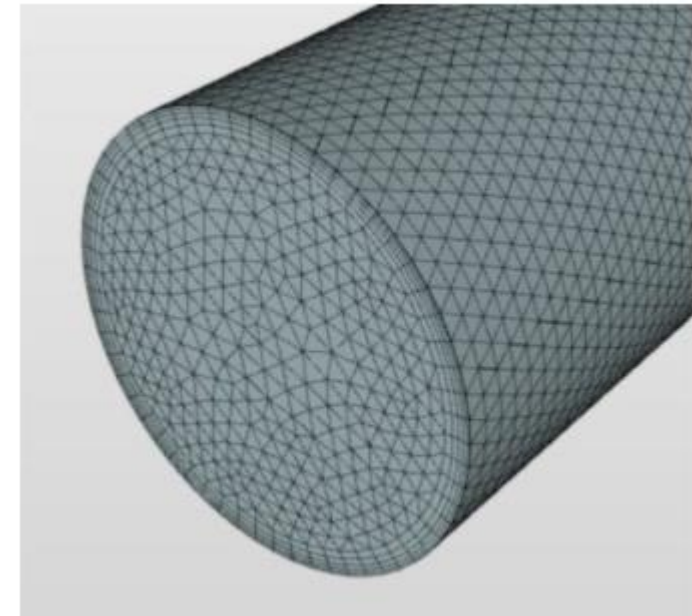
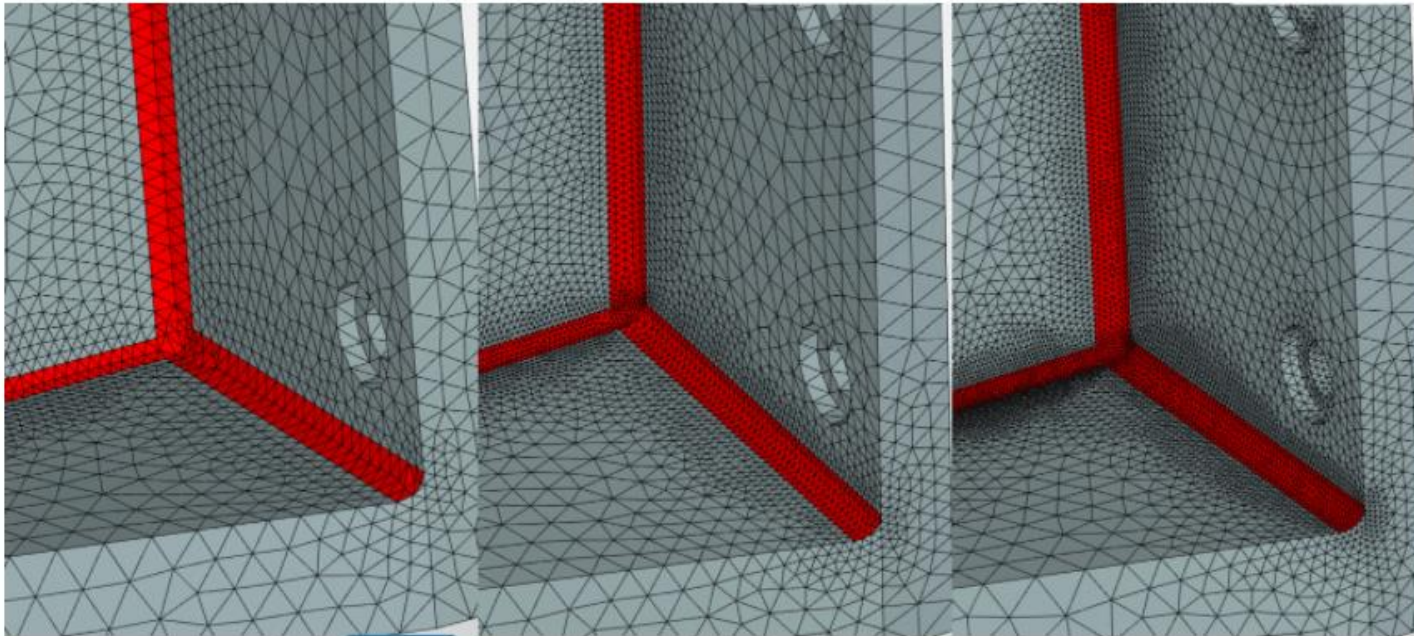


Nodes



# Meshing

## Mesh refinement



# Common Nastran solution types

- 101 - Linear Static
- 103 - Modal
- 105 - Buckling
- 106 - Non-Linear Static
- 107 - Direct Complex Eigenvalue
- 108 - Direct Frequency Response
- 109 - Direct Transient Response
- 110 - Modal Complex Eigenvalue
- 111 - Modal Frequency Response
- 112 - Modal Transient Response
- 129 - Nonlinear Transient
- 144 - Static Aeroelastic Analysis
- 145 - Flutter / Aeroservoelastic analysis
- 146 - Dynamic Aeroelastic Analysis
- 153 - Non-Linear static coupled with heat transfer
- 159 - Nonlinear Transient coupled with Heat transfer

## Further reading

NASA FEA Book: <https://etd.gsfc.nasa.gov/wp-content/uploads/2025/04/FEMCI-The-Book.pdf>

Key topics:

### **Finite Element Model Checking**

<https://ntrs.nasa.gov/api/citations/19950020950/downloads/19950020950.pdf>

### **Eigenvalue extraction method in Nastran**

### **Coefficient of thermal expansion**

# Dynamic

Eigenmode analysis:

<https://www.youtube.com/watch?v=9UpMeDO0A04>



# NX and Simcenter 3D

# 16

# NX shortcuts and mouse commands

[https://community.3dcs.com/help\\_manual/nx\\_shortcutkeys.htm](https://community.3dcs.com/help_manual/nx_shortcutkeys.htm)

<https://www.cad-tips.com/siemens-nx/the-mouse-functions/>

<https://community.sw.siemens.com/s/article/nx-shortcut-keys-view-full-list-and-create-custom-keys>

| Buttons                    | Function                                   |
|----------------------------|--------------------------------------------|
| Left Button                | Selection and Selection trap               |
| Center Button/Roller       | Zoom with roller                           |
| Right Button               | On a object loads the right-click menu     |
| Left + Center              | Zoom                                       |
| Right + Center             | Rotate                                     |
| Right button and Shift Key | Pan                                        |
| Hold Center                | Temporarily changes the center of rotation |

From Lectures Exercises:

**Exercise**

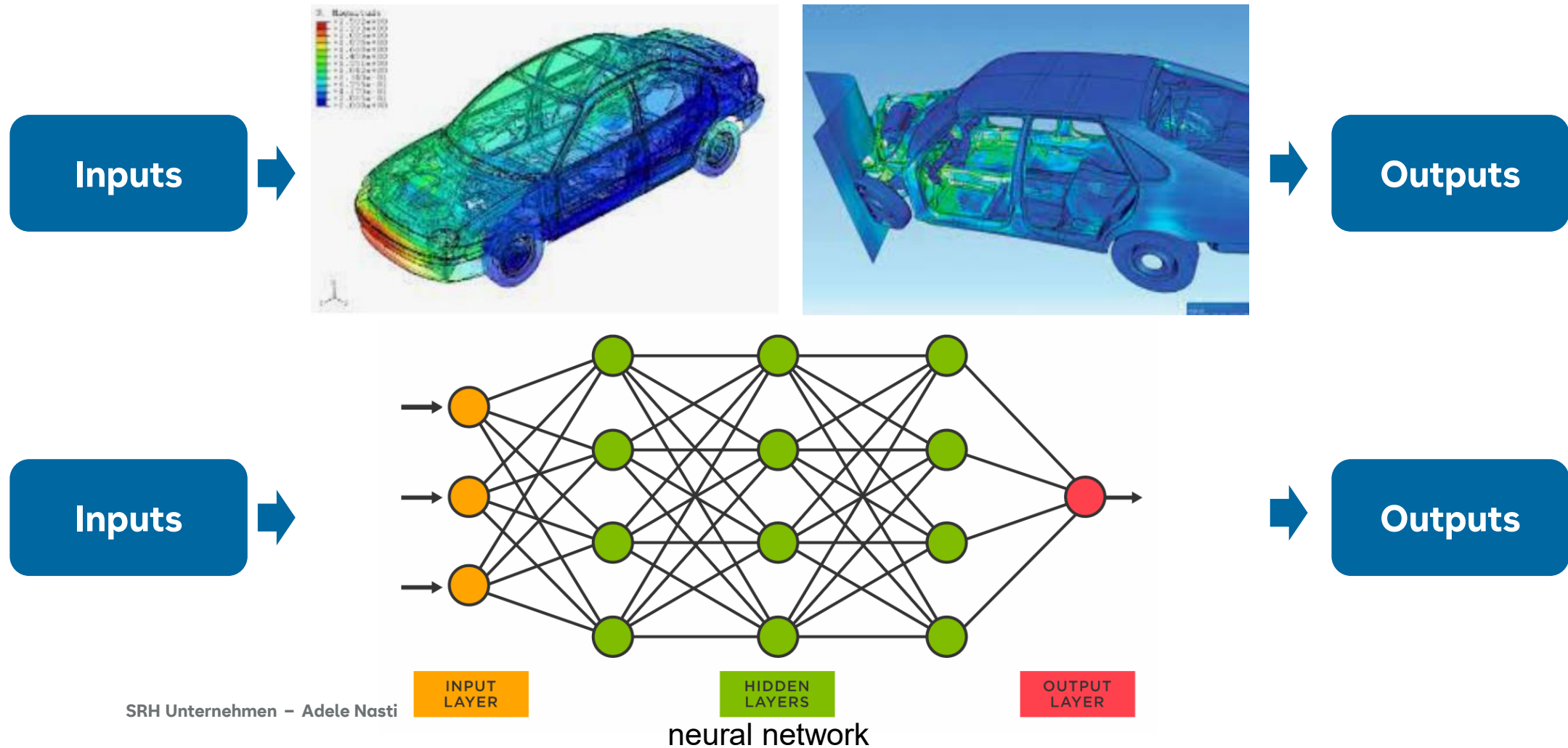
**07 & 08**

# Advanced Computational Engineering

# 17

# Physics-based models and empirical models

Engineering simulation is always about input-output relationships



# Machine Learning vs Deep Learning

## Machine Learning

enables computers to process data, learn from them and build a model that approximates the input-output relationship in the data

Examples of machine learning algorithms: Linear Regression, Logistic Regression, Decision Trees, Support Vector Machine, Naive Bayes, k-nearest neighbours, K-Means, Random Forest, Dimensionality Reduction Algorithms

## Deep Learning

enables computers to continuously train themselves and improve their knowledge as they go along. This is typically achieved by complex neural networks with multiple hidden layers

# Supervised vs unsupervised learning

## Supervised learning

- Labelled datasets (inputs and outputs)
- inputs:  $x_1, x_2, \dots, x_n \rightarrow$  outputs:  $y_1, y_2, \dots, y_m$
- Characterise input-output relationship and make predictions on new outputs (on the basis of new inputs)
  - E.g. regression or classification
  - Potential use: to create a surrogate model of a simulation model  
*component performance = function (temperature, pressure)*

## Unsupervised learning

- Unlabelled datasets. The algorithm works out the inherent structure of the data
- Get insights from the data, uncover patterns and determine what relevant information is hidden in the data
  - E.g. clustering
  - Potential use: experimental validation of engineering components

# Machine Learning in Engineering Simulation

Physics-based models in engineering are nowadays very sophisticated

Datasets available from experiments or product operation are not always *large datasets*

## Can machine learning beat physics-based modelling?

The answer needs to be studied on a case-by-case basis

## What are the most effective applications of machine learning in engineering?

Machine Learning can be used as a **surrogate modelling** technique on simulation datasets, in particular for simulation models with long runtimes

It can also be used on a combination of simulation data and *real* data

Other machine learning techniques like Principal Component Analysis can be used to understand experimental data and extract the relevant features in order to build anomaly detection algorithms

*The potential applications of AI & machine learning in engineering are infinite and yet to be fully discovered*

# Example: Principal Component Analysis

# 18

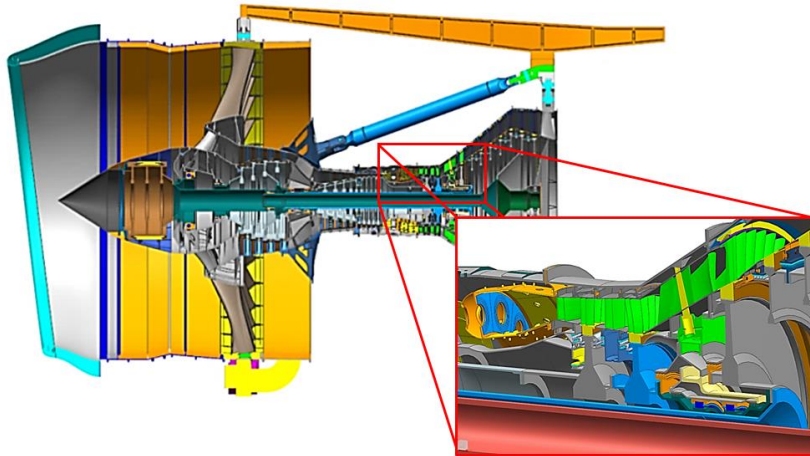


# Identifying patterns in large datasets

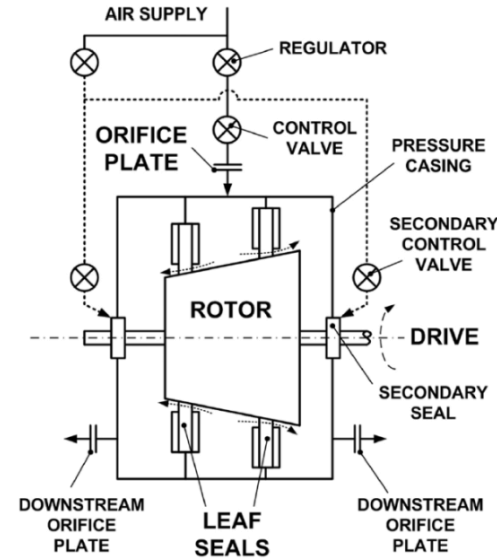
**Principal Component Analysis (PCA)** is an unsupervised linear transformation technique that identifies patterns in datasets based on the **correlations** between features

PCA is extensively used for feature extraction and **dimensionality reduction**, to compress a dataset onto a lower-dimensional feature subspace, with the aim of maintaining the most relevant information

# Experimental validation in engineering



**Jet engine design and component assembly**



**Experimental rig for component testing**

An experimental rig for component testing can measure hundreds of parameters

— E.g. Temperature and pressure measurements via sensors:  $T_1, T_2, \dots, T_h, P_1, P_2, \dots, P_k$

Are they all necessary to characterise the component behaviour?

Are they all independent?

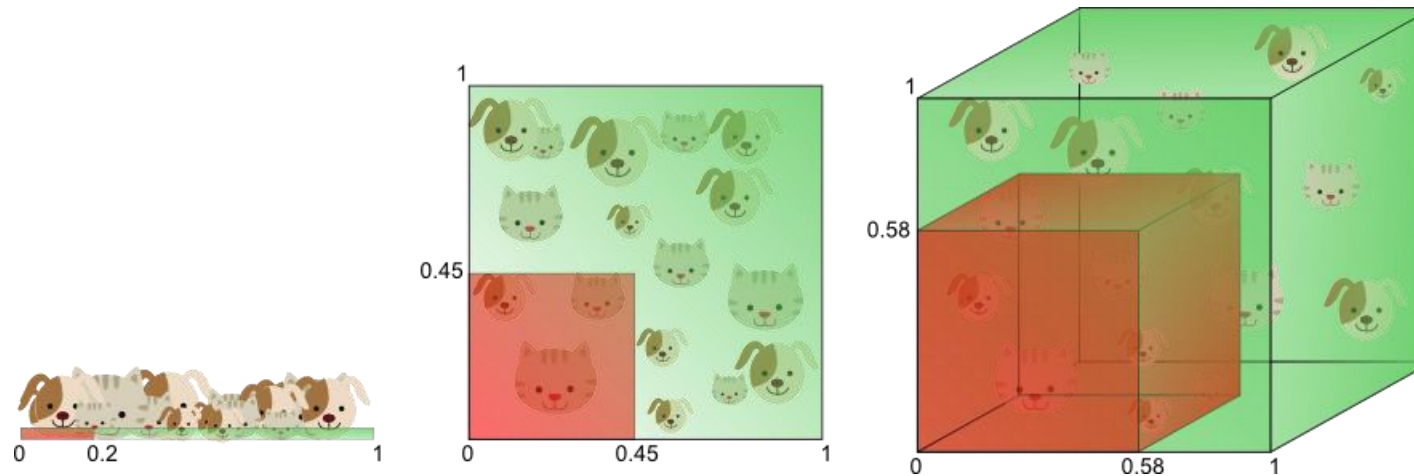
No. Redundancy (e.g. to mitigate the failure of a sensor or to eliminate noise). Some variables can be highly correlated

Can we exploit the correlations between the parameters to reduce the number of parameters to monitor and get a clearer understanding of the component behaviour?

# Too many parameters

## The curse of dimensionality

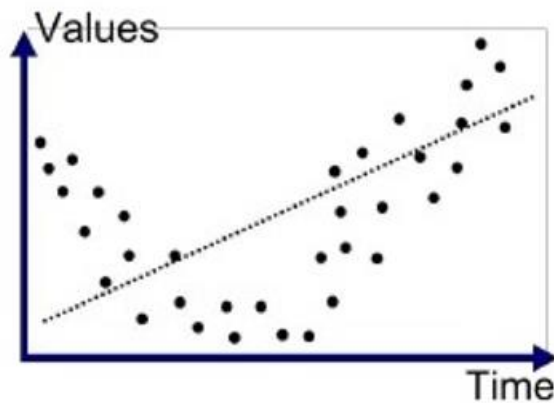
- In many dimensions, the data become sparse
- It is harder to spot patterns
- The amount of data needed grows exponentially with dimensionality
- Too many features can decrease the model accuracy



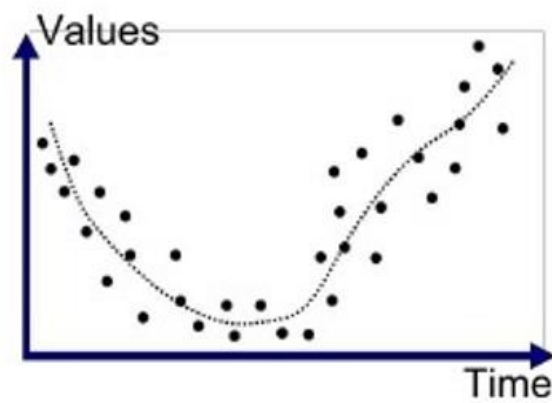
# Too complex models

## Overfitting

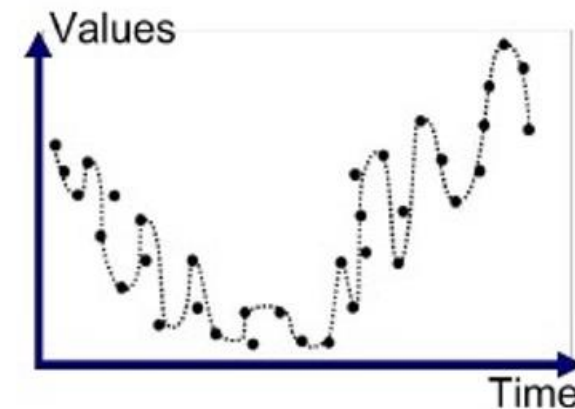
- Some data might represent noise and not a pattern in the data
- If the model is too complex, the solution could be bespoke to the dataset used for the training
- The capability to predict behaviour on new data may become poor



Underfitted



Good Fit/Robust



Overfitted

# How much data is enough data?

*Keep the information that matters, drop what is redundant*

Dimensionality Reduction

## Feature selection

- Down select a subset of the variables
- e.g.  $T_1, T_2, \dots, T_h, P_1, P_2, \dots, P_k \rightarrow T_1, T_2, P_1, P_k$

## Feature extraction

→ PCA

- Extract information from the set of variables and construct a new feature subspace
- e.g.  $T_1, T_2, \dots, T_h, P_1, P_2, \dots, P_k$

→  $T_1 + T_2, T_1 / T_h, T_1 / P_1, P_k - P_2$

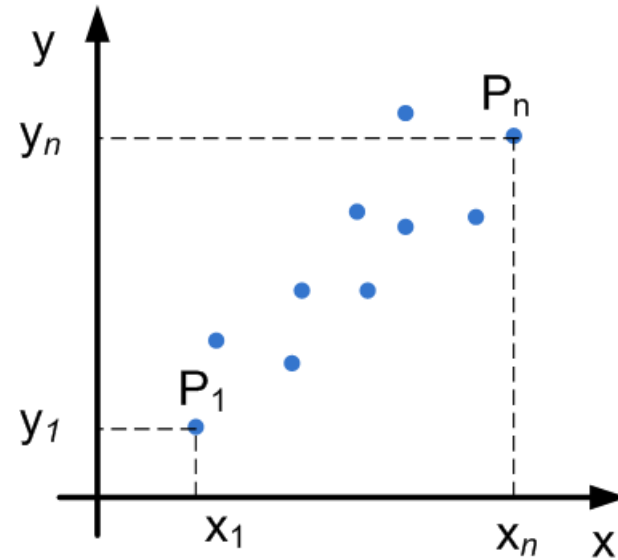
# Principle Component Analysis

PCA identifies the **directions of maximum variance** in high-dimensional data and projects the data onto a new subspace with equal or fewer dimensions

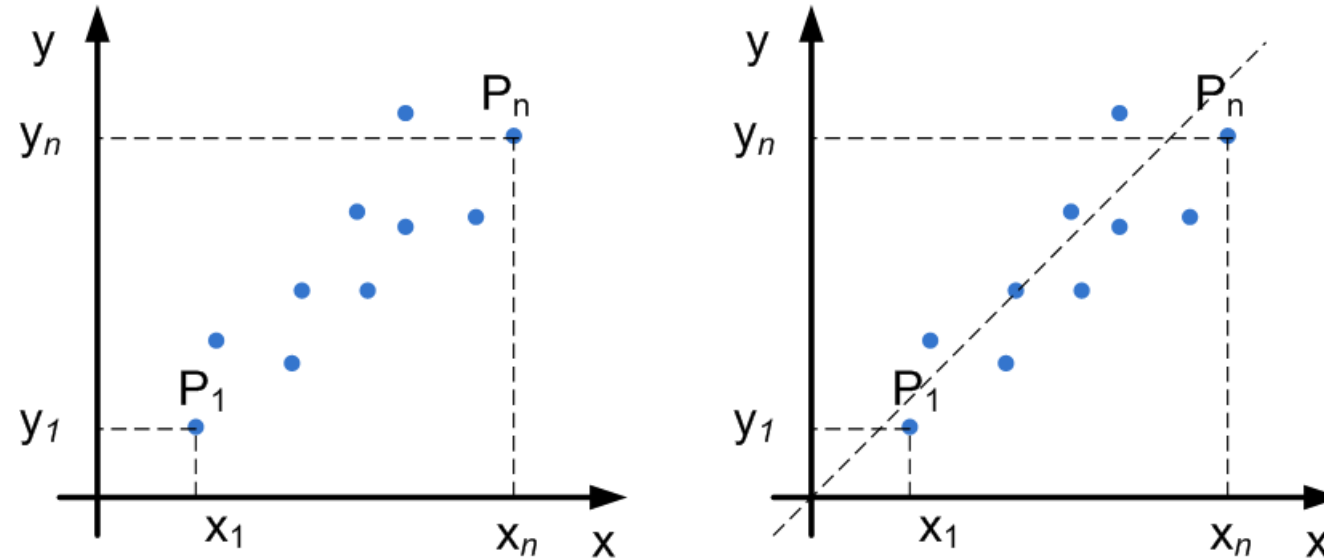
The principal components are the directions that contain **most of the relevant information** of the dataset

**Benefits:** both computational efficiency and predictive capability

# Principal Component Analysis for dimensionality reduction (2-D $\rightarrow$ 1-D)

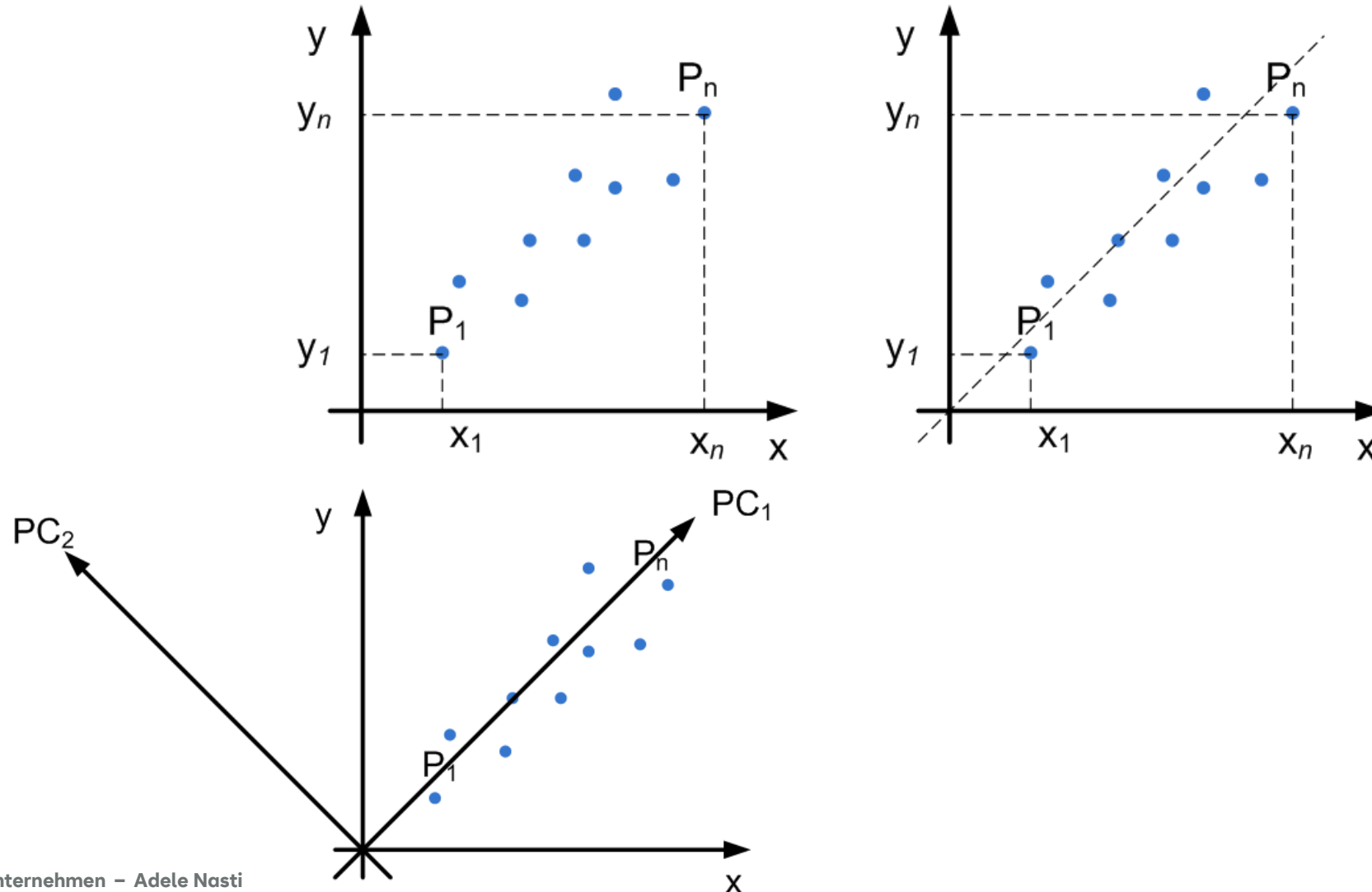


# Principal Component Analysis for dimensionality reduction (2-D $\rightarrow$ 1-D)

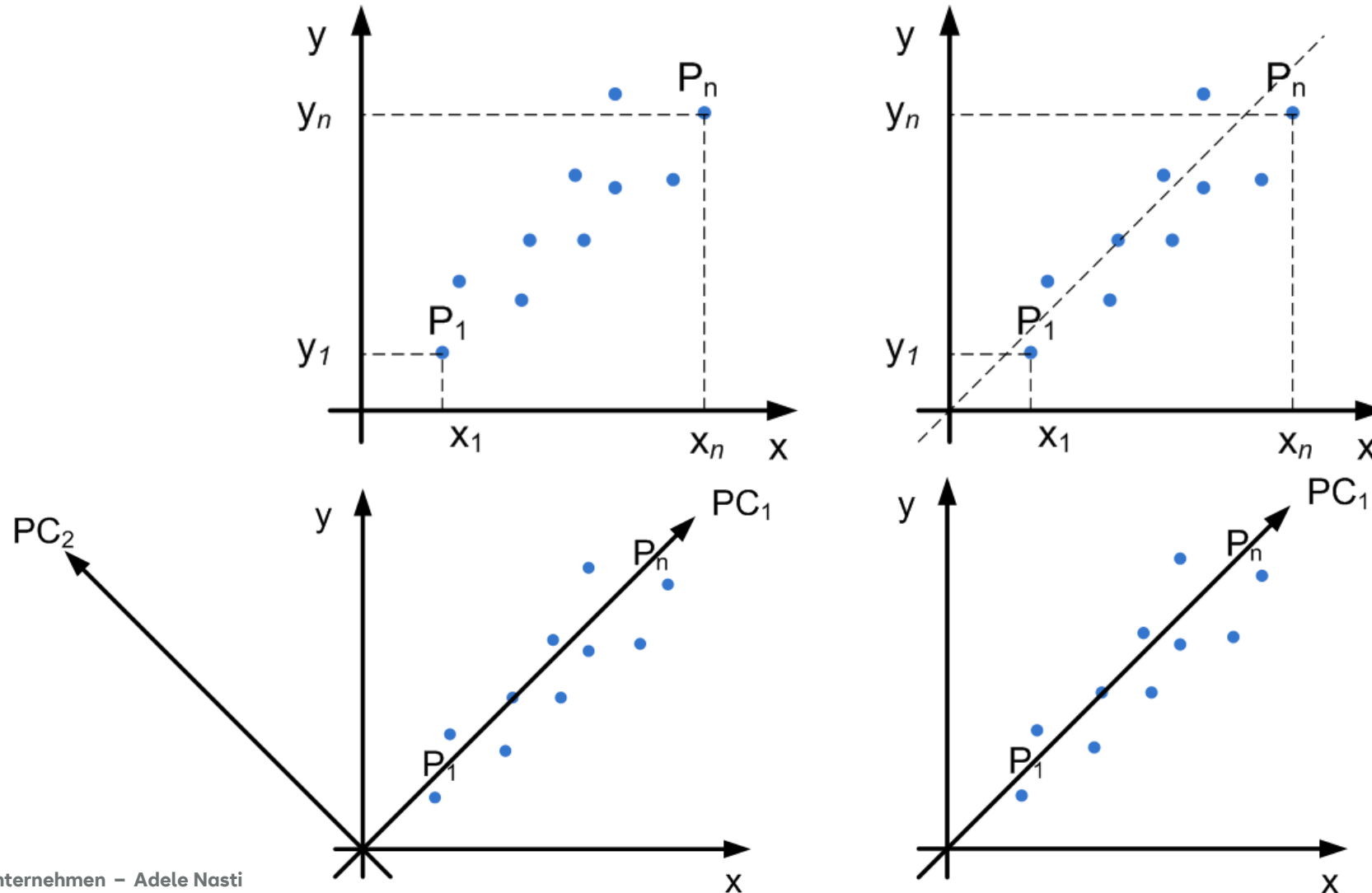




# Principal Component Analysis for dimensionality reduction (2-D $\rightarrow$ 1-D)



# Principal Component Analysis for dimensionality reduction (2-D→1-D)

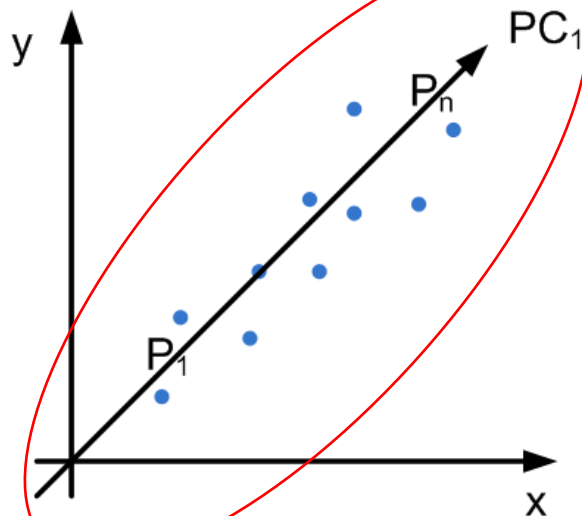


# What do the principal components mean?

The principal components are the directions of maximum variance

How should we interpret them?

The principal components might not always be intuitive to interpret



Schrödinger's cat: half dead & half alive

# Principal Component Analysis for dimensionality reduction (d-D → k-D)

*How do we extract the Principal Components of a dataset?*

Objective: find directions of maximum variance and project them onto a new subspace

1. Standardise the data (mean = 0 and standard deviation = 1)
2. Calculate the covariance matrix A

— Covariance  $(X, Y) = \sum_i (X_i - \text{mean}(X)) (Y_i - \text{mean}(Y)) / n$

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

- Variance  $(a, e, i)$ : **variability** of each parameter
  - Covariance  $(b = d, c = g, f = h)$ : **correlation** between parameters
3. Compute eigenvalues and correspondent eigenvectors
    - $A \mathbf{v} = \lambda \mathbf{v}$
    - $\det(A - \lambda I) = 0$

# Principal Component Analysis for dimensionality reduction ( $d \rightarrow k$ )



- Sort the eigenvalues  $\lambda_1, \lambda_2, \lambda_3 (\dots \lambda_d)$  starting from the largest to identify the most relevant eigenvectors
- Select  $k$  eigenvectors  $\mathbf{v}_1, \dots, \mathbf{v}_k$  corresponding to the  $k$  largest eigenvalues: **principal components**
- Construct a projection matrix  $P$  from the original  $d$ -dimensional space to the  $k$ -dimensional feature subspace ( $k < d$ ) using the first  $k$  eigenvectors

$$P = [\mathbf{v}_1, \dots, \mathbf{v}_k]$$

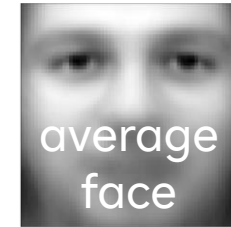
- Transform the original  $d$ -dimensional dataset into a new  $k$ -dimensional dataset using the projection matrix

$$\text{new } k\text{-dimensional dataset} = \text{old } d\text{-dimensional dataset} \times P$$

- Explained variance ratio:  $\lambda_i / \sum \lambda_i$  (variability per component)

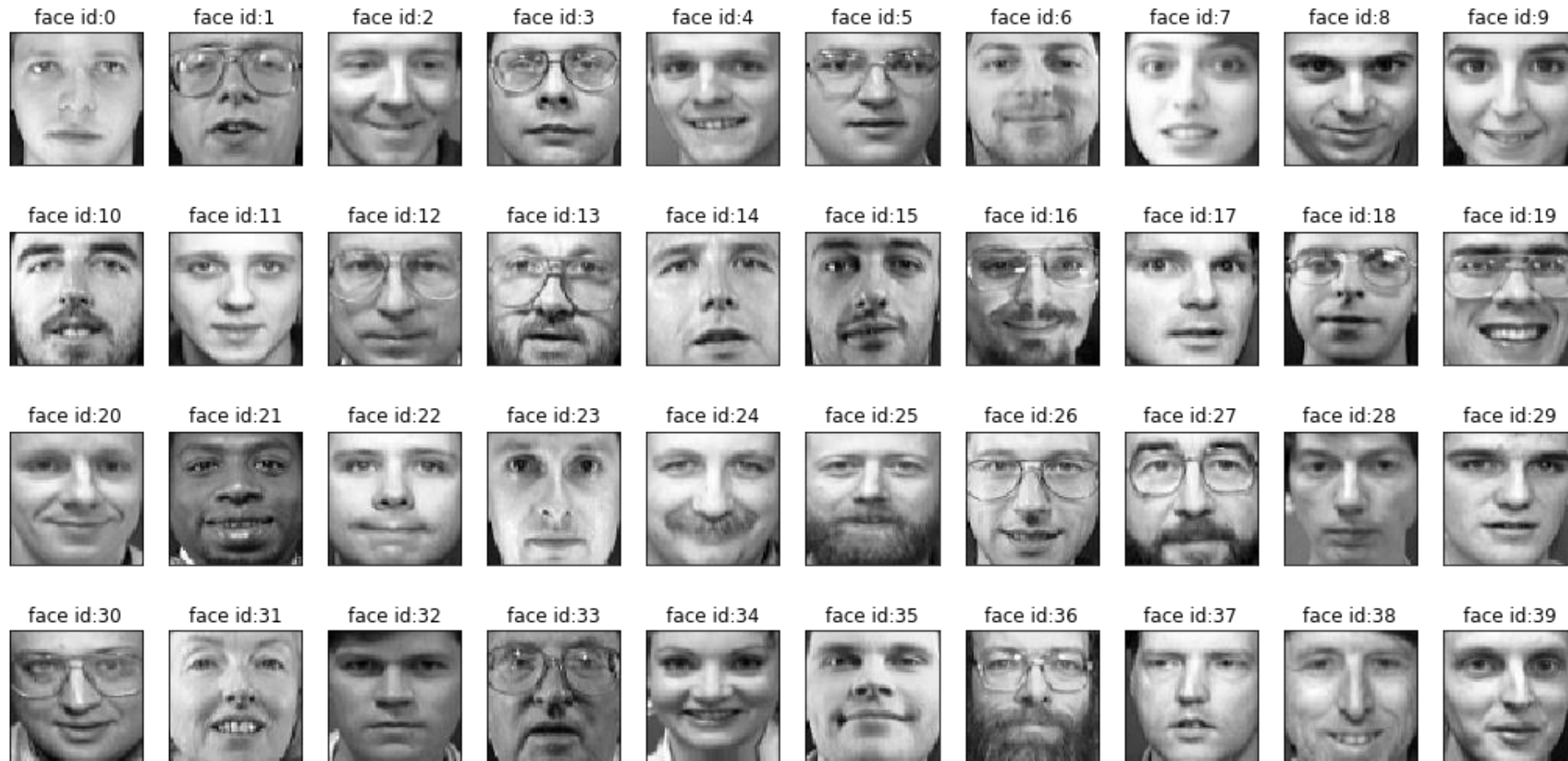
— ***How many eigenvectors do I need to retain most of the information?***

# Example: PCA for face recognition



The Olivetti dataset: 400 (64 x 64 pixel) images of 40 people.  $d = 4096$

→ a lot of redundancy!



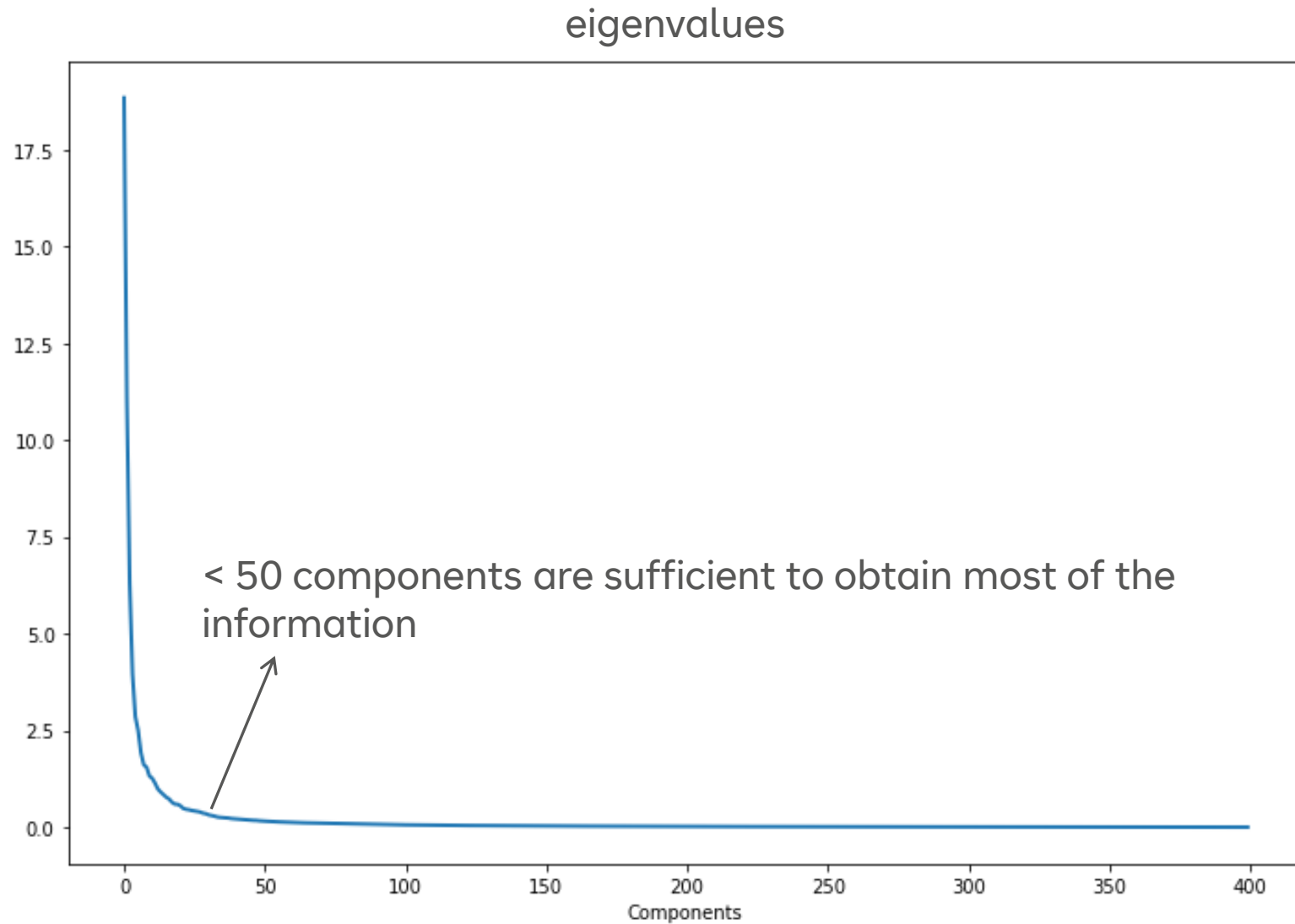
# Example: PCA for face recognition

eigenfaces



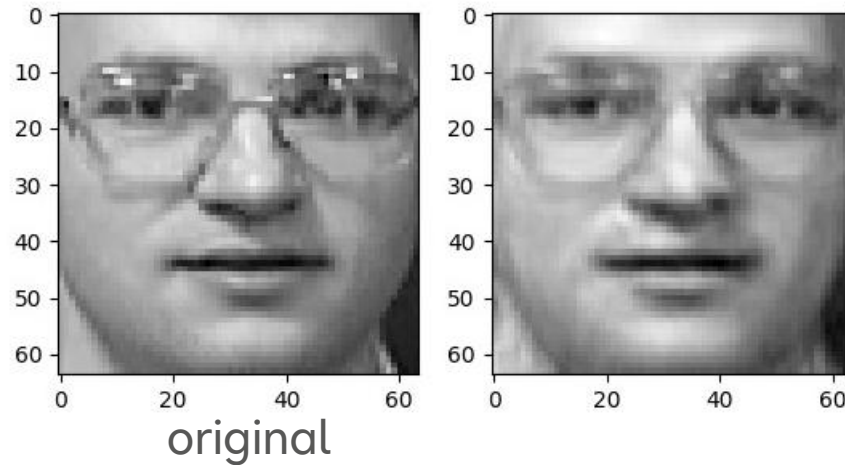
the  
→ appearance  
of  
*features*

# Example: PCA for face recognition

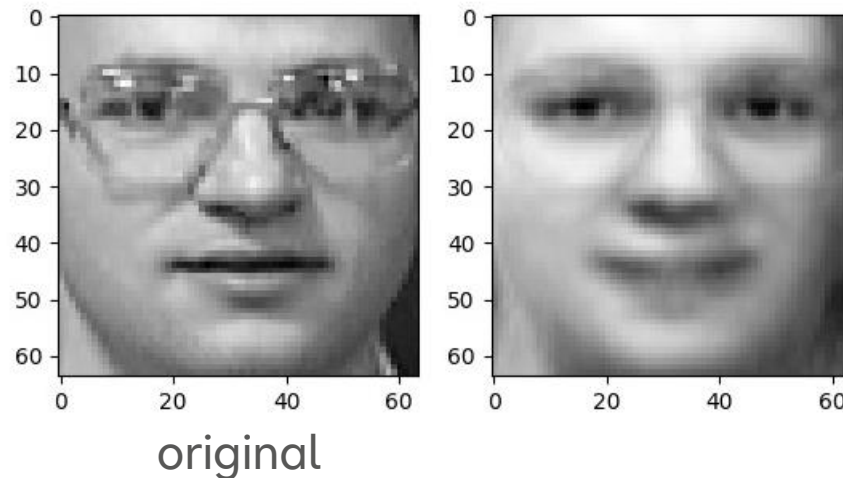




# Example: PCA for face recognition



Face reconstructed taking  
into account **100**  
eigenvectors



Face reconstructed taking  
into account **10** eigenvectors

Not just a beautiful mathematical technique...

...use of PCA outside engineering





PCA uncovers information hidden within hundreds of parameters, enables **condition monitoring** and allows safeguarding test hardware, automating the post processing of experimental data and reducing the overall cost of experiments

# Additional References

- [1] ASME Paper GTP-15-1561 – Journal of Engineering for Gas Turbines and Power
- [2] Patent US 20130320627 A1 – *Nasti et al.*
- [3] Paper GT2020-15623 “Process Automation for Aero Engine Secondary Air System Seal Design”, Adele Nasti
- [4] <https://www.visiondummy.com/2014/04/curse-dimensionality-affect-classification/>
- [5] <https://medium.com/greyatom/what-is-underfitting-and-overfitting-in-machine-learning-and-how-to-deal-with-it-6803a989c76>
- [6] <https://www.shutterstock.com/image-vector/life-death-schrodingers-cat-vector-illustration-1328056802>
- [7] [https://www.kaggle.com/imrandude/olivetti?select=olivetti\\_faces.npy](https://www.kaggle.com/imrandude/olivetti?select=olivetti_faces.npy)
- [8] <https://www.kaggle.com/serkanpeldek/face-recognition-on-olivetti-dataset>

# Examples of Use of AI in Engineering

# 19

# AI in Engineering – MATLAB examples

AI in engineering:

<https://de.mathworks.com/videos/ai-for-engineers-building-an-ai-system-1603356830725.html>

Examples of applications:

<https://de.mathworks.com/campaigns/offers/next/ai-foundational-tasks.html>

Exercise – Life estimation:

<https://de.mathworks.com/help/predmaint/ug/similarity-based-remaining-useful-life-estimation.html>

Engineering examples:

[https://de.mathworks.com/help/stats/examples.html?category=index&s\\_tid=CRUX\\_topnav](https://de.mathworks.com/help/stats/examples.html?category=index&s_tid=CRUX_topnav)

[https://de.mathworks.com/help/predmaint/examples.html?category=preprocess-data&s\\_tid=CRUX\\_topnav](https://de.mathworks.com/help/predmaint/examples.html?category=preprocess-data&s_tid=CRUX_topnav)

# AI in Engineering – MATLAB machine learning toolbox

## Regression algorithms

<https://de.mathworks.com/help/stats/regression-using-dataset-arrays.html>

<https://de.mathworks.com/help/stats/bayesian-linear-regression-using-hamiltonian-monte-carlo.html>

<https://de.mathworks.com/help/stats/partial-least-squares-regression-and-principal-components-regression.html>

## Assess **neural network** performance

<https://de.mathworks.com/help/stats/assess-neural-network-classifier-performance.html>

# Software Development

# 20



# Simulation software

Multiple **commercial codes** are available to address sophisticated engineering problems

*CAD Tools, Finite Element Analysis Structural Analysis Tools, Computational Fluid Dynamics Tools*

as well as tools for building **Integrated Design Systems** to couple these tools together and perform Design Studies, Optimisation or Regression via Machine Learning

Before approaching an engineering simulation problem it is important to do a thorough research of the state-of-the-art in simulation software to address that specific problem and down-select the most appropriate tool or set of tools on the basis of our requirements (capability, speed, cost, etc.)

**But** not everything can be solved only with commercial tools. In particular for niche applications or novel technologies it is very often necessary in industry to develop software from scratch or modify someone else's software. Even when using commercial software for the solvers, we may have to create bespoke software for pre or post processing, or for specific functionalities

Important: **developing software is not just writing code!**

# Software development: Introduction

**Software Development at Microsoft Observed:** it's about *people... working together* (2005)

To understand Microsoft developers' typical tools and work habits and their level of satisfaction with these, we performed two surveys and eleven interviews with developers across all business divisions. This report provides a summary of the resulting data. From the **set of potential problems** we gave them, **the top three that Microsoft developers agree they have are: understanding the rationale behind a piece of code (66%); having to switch tasks often because of requests from teammates or managers (62%); and being aware of changes to code elsewhere that impact their own code (61%).** The most notable take-away from the data is that developers go to great lengths to create and maintain rich mental models of code and don't rely on external representations. The mental nature of these models requires frequent, disruptive, face-to-face meetings to keep individuals' models in sync, which greatly slow the rate at which a newcomer to a team can become productive. These interruptions also burden more senior development team members, as they have to recover what they were doing in the code following the interruption from team members.

<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-2005-140.pdf>

# Software development

**Software Development at Microsoft Observed:** it's about *people... working together* (2005)

<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-2005-140.pdf>

| Problem                                                                                          | % Agree |
|--------------------------------------------------------------------------------------------------|---------|
| <b>Code Understanding</b>                                                                        |         |
| Understanding the rationale behind a piece of code                                               | 66%     |
| Understanding code that someone else wrote                                                       | 56%     |
| Understanding the history of a piece of code                                                     | 51%     |
| Understanding code that I wrote a while ago                                                      | 17%     |
| <b>Task Switching</b>                                                                            |         |
| Having to switch tasks often because of requests from my teammates or manager                    | 62%     |
| Having to switch tasks because my current task gets blocked                                      | 50%     |
| <b>Modularity</b>                                                                                |         |
| Being aware of changes to code elsewhere that impact my code                                     | 61%     |
| Understanding the impact of changes I make on code elsewhere                                     | 55%     |
| <b>Links between Artifacts</b>                                                                   |         |
| Finding all the places code has been duplicated                                                  | 59%     |
| Understanding who "owns" a piece of code                                                         | 50%     |
| Finding the bugs related to a piece of code                                                      | 41%     |
| Finding code related to a bug                                                                    | 28%     |
| Finding out who is currently modifying a piece of code                                           | 16%     |
| <b>Team</b>                                                                                      |         |
| Convincing managers that I should spend time rearchitecting, refactoring, or rewriting code      | 43%     |
| Convincing developers on other teams within Microsoft to make changes to code I depend on        | 42%     |
| Getting enough time with senior developers more knowledgeable about parts of code I'm working on | 34%     |
| <b>Expertise Finding</b>                                                                         |         |
| Finding the right person to talk to about a piece of code                                        | 39%     |
| Finding the right person to talk to about a bug                                                  | 38%     |
| Finding the right person to review a change before check-in                                      | 19%     |

**Table 1:** Percent of developers who agreed that "This is a serious problem for me." (The survey participants were presented the questions in a different order, and without categorization.)

# Agile practices

**Software Development at Microsoft Observed:** it's about *people... working together* (2005)

| Agile Practice                                                                                    | % Agree |
|---------------------------------------------------------------------------------------------------|---------|
| Collective code ownership within the team                                                         | 49%     |
| "Sprints," i.e. a development cycle that last four (or so) weeks                                  | 42%     |
| An intentional policy to involve customers (internal or external) deeply into design and planning | 33%     |
| "Scrum meetings," i.e. a brief daily status meeting including all stakeholders                    | 25%     |
| "Burndown" estimate or chart, i.e. a measure of the time remaining in the sprint                  | 24%     |
| An intentional policy of preferring face-to-face over electronic communications                   | 16%     |
| Pair programming, i.e. developers working together, shoulder-to-shoulder on a problem             | 16%     |
| A "bullpen" or other open-floorplan space for the team                                            | 10%     |

**Table 2:** *Percent of developers who agreed that their teams use these agile development practices. Adoption of agile methodologies is happening piecemeal and in islands around the company.*

<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-2005-140.pdf>

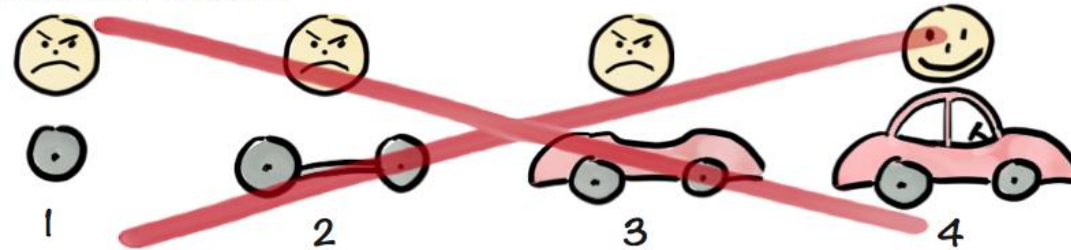
# Agile software development manifesto

We are uncovering better ways of developing software...

Individuals and interactions over processes and tools  
Working software over comprehensive documentation  
Customer collaboration over contract negotiation  
Responding to change over following a plan

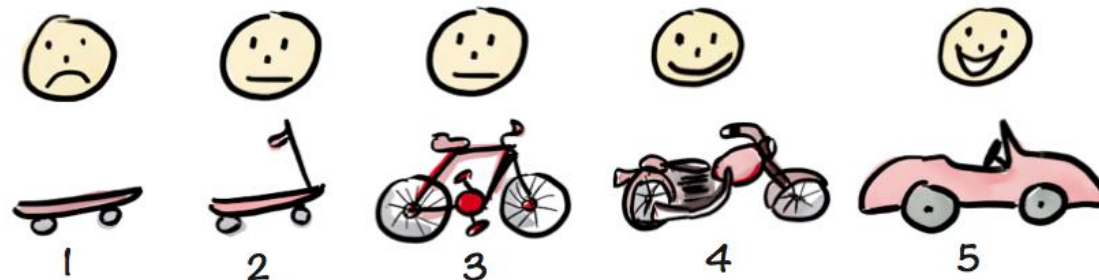
That is, while there is value in the items on the right, we value the items on the left more

**Not like this....**



*Requirements vs  
user stories*

**Like this!**

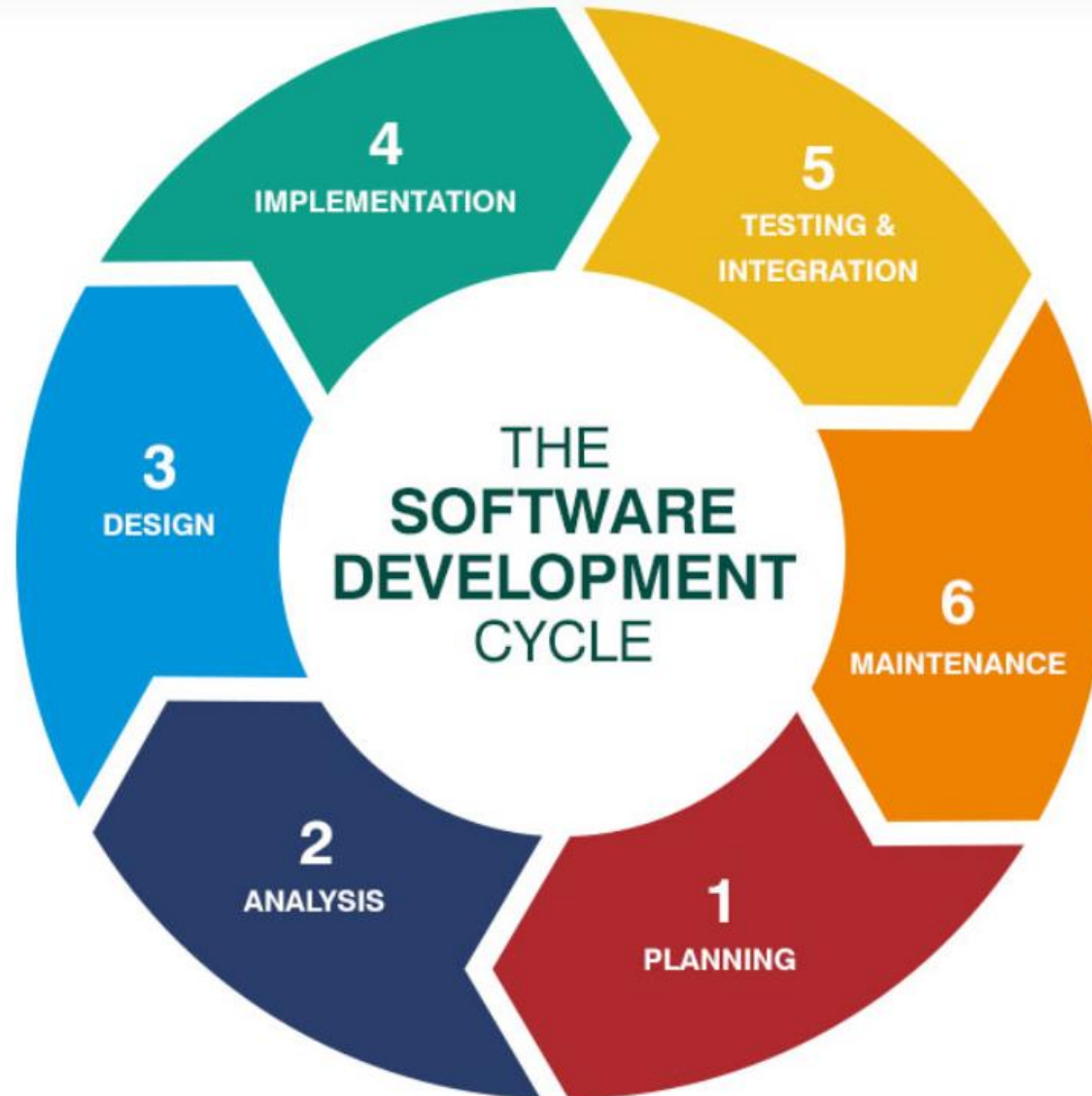




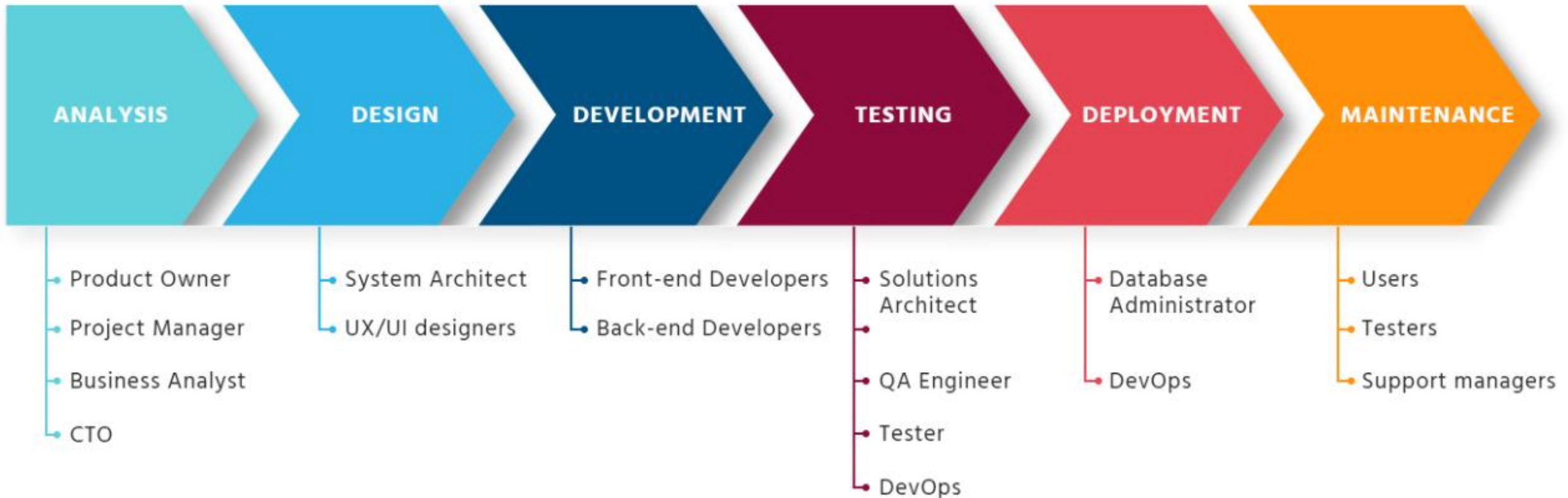
# Software development lifecycle

**Developing software is not just writing code!**

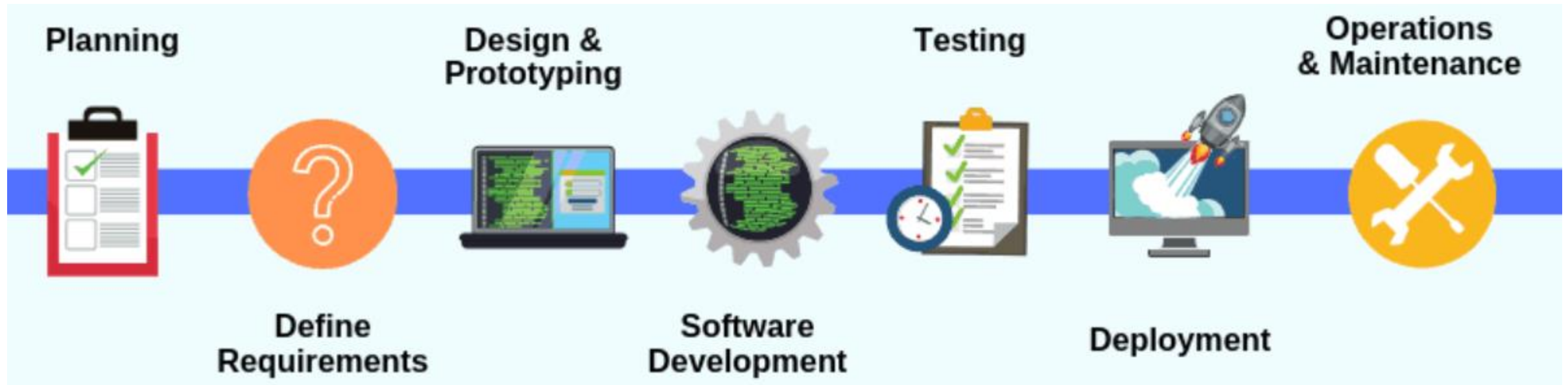
Writing professional software is a complex process



# Software development lifecycle key stakeholders



# Phases of the software development lifecycle





# SOFTWARE DEVELOPMENT LIFE CYCLE



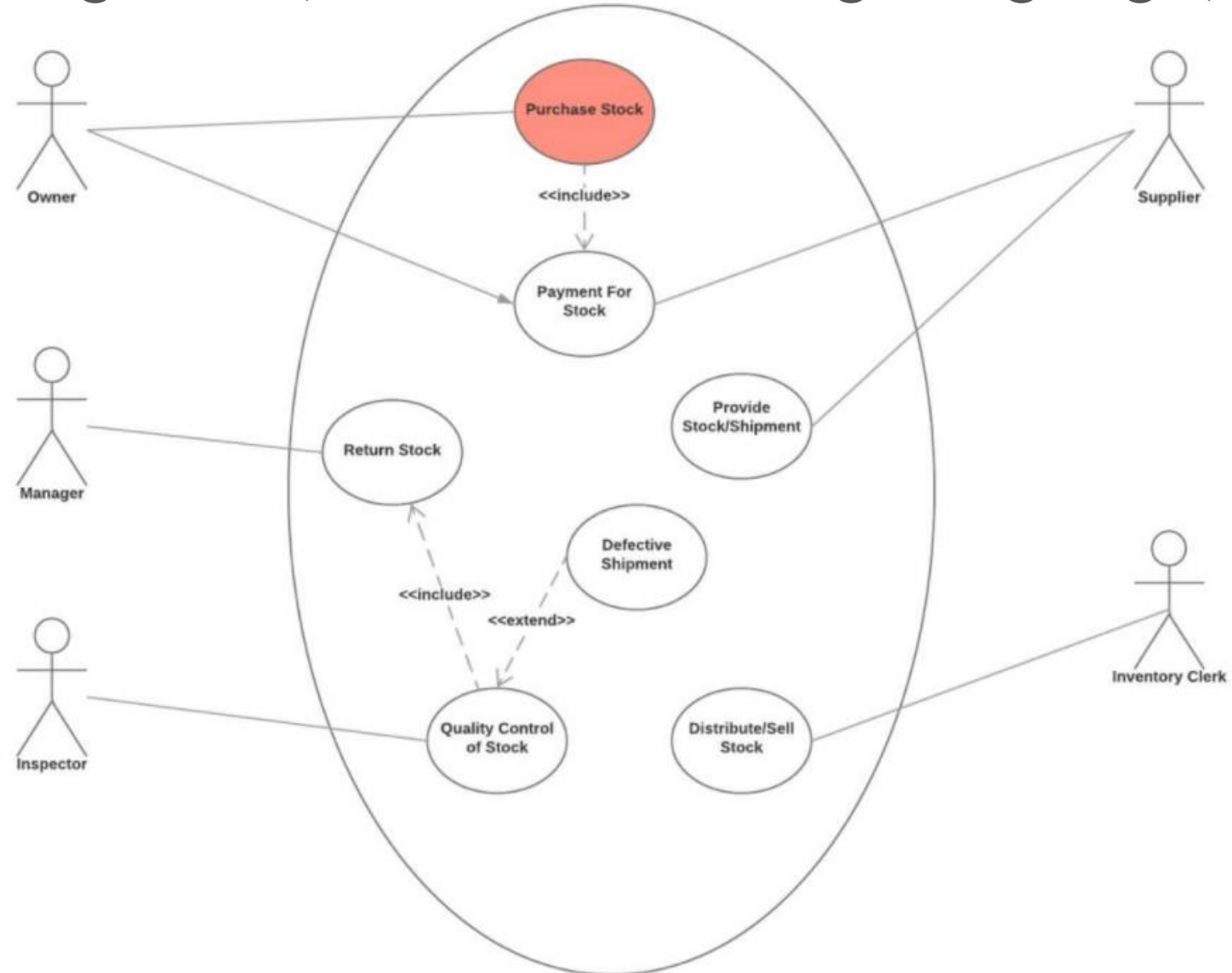
**Developing software  
is not just writing code!**

# Examples of UML diagrams (Unified Modelling Language)

## Use case diagram

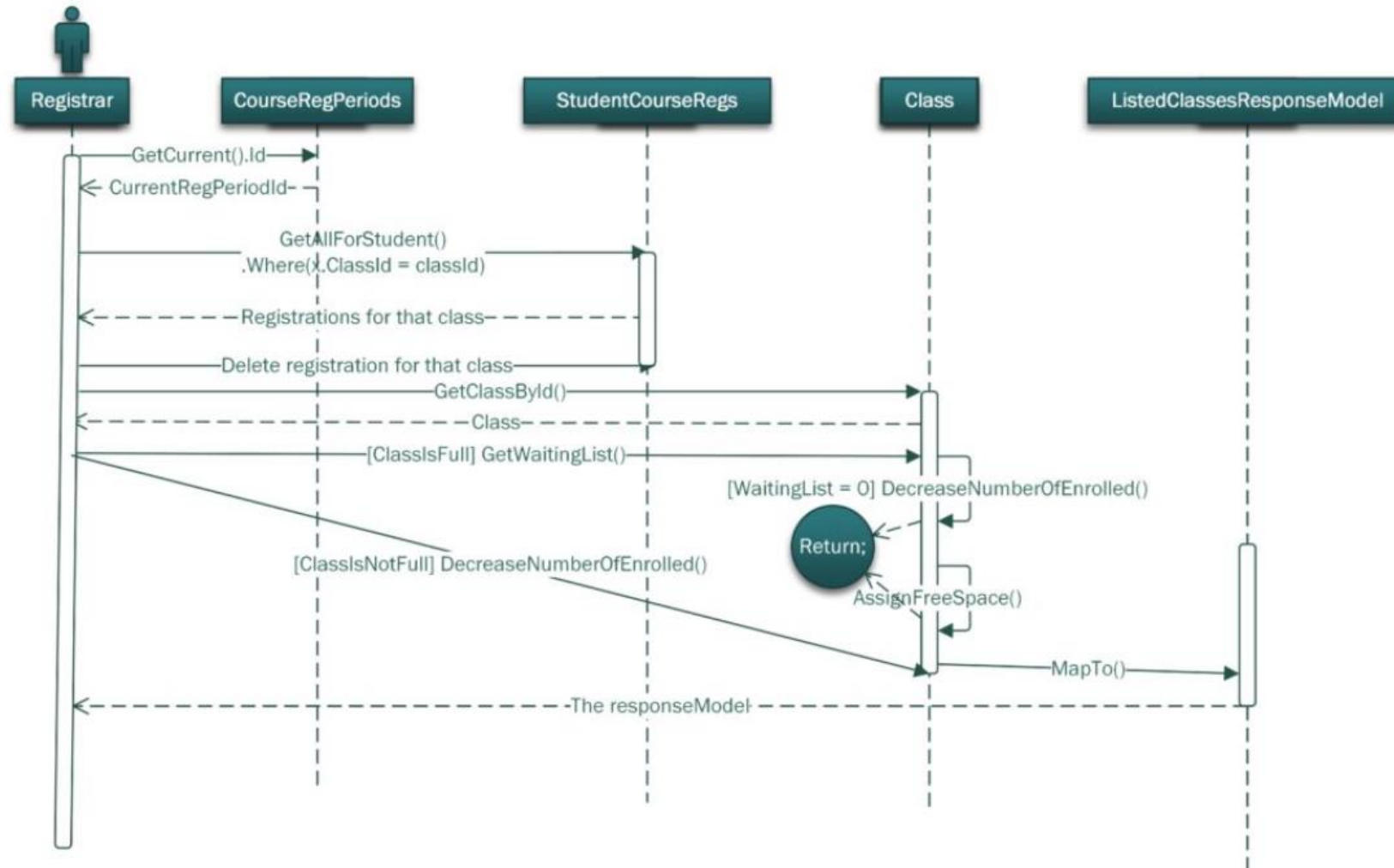
### Inventory management system

- Functional requirements
- Actors
- Relationships



# Examples of UML diagrams

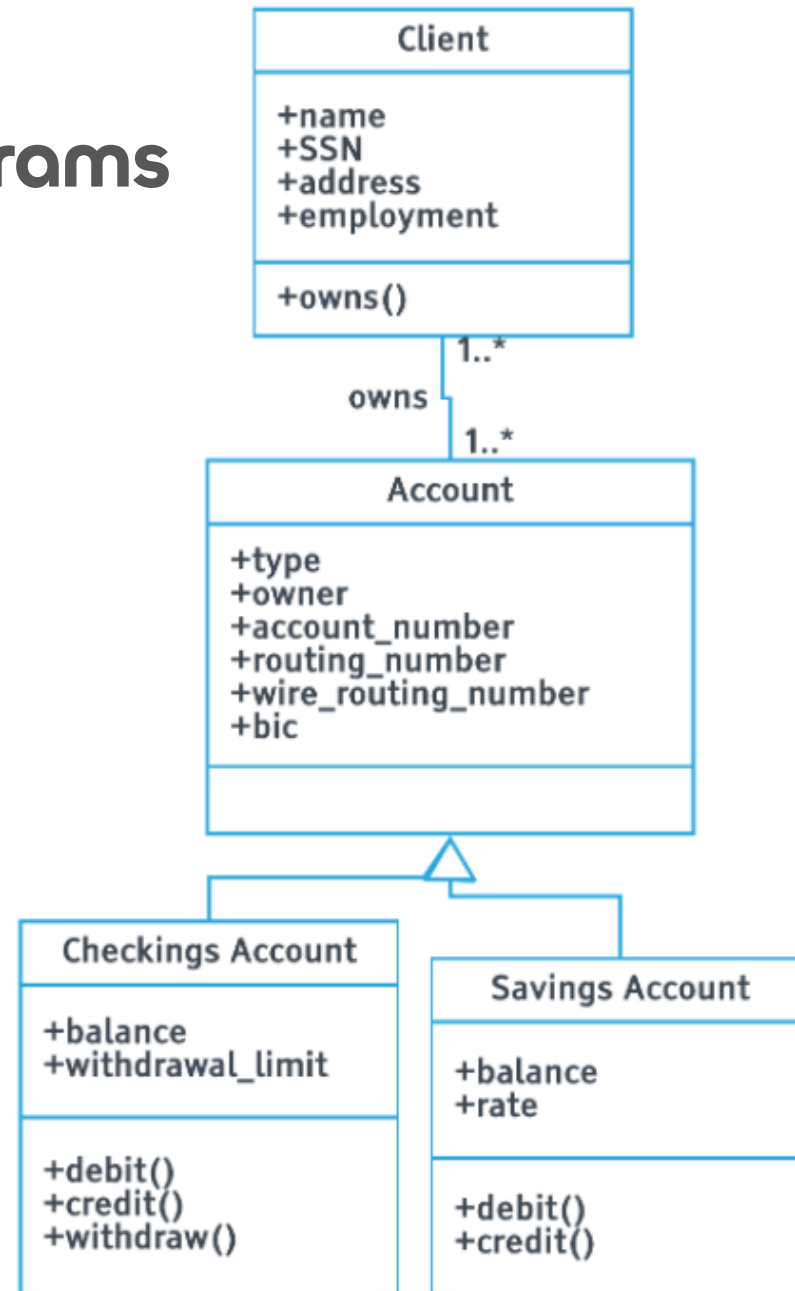
## Sequence diagram: Course registration system



# Examples of UML diagrams

## Class diagram:

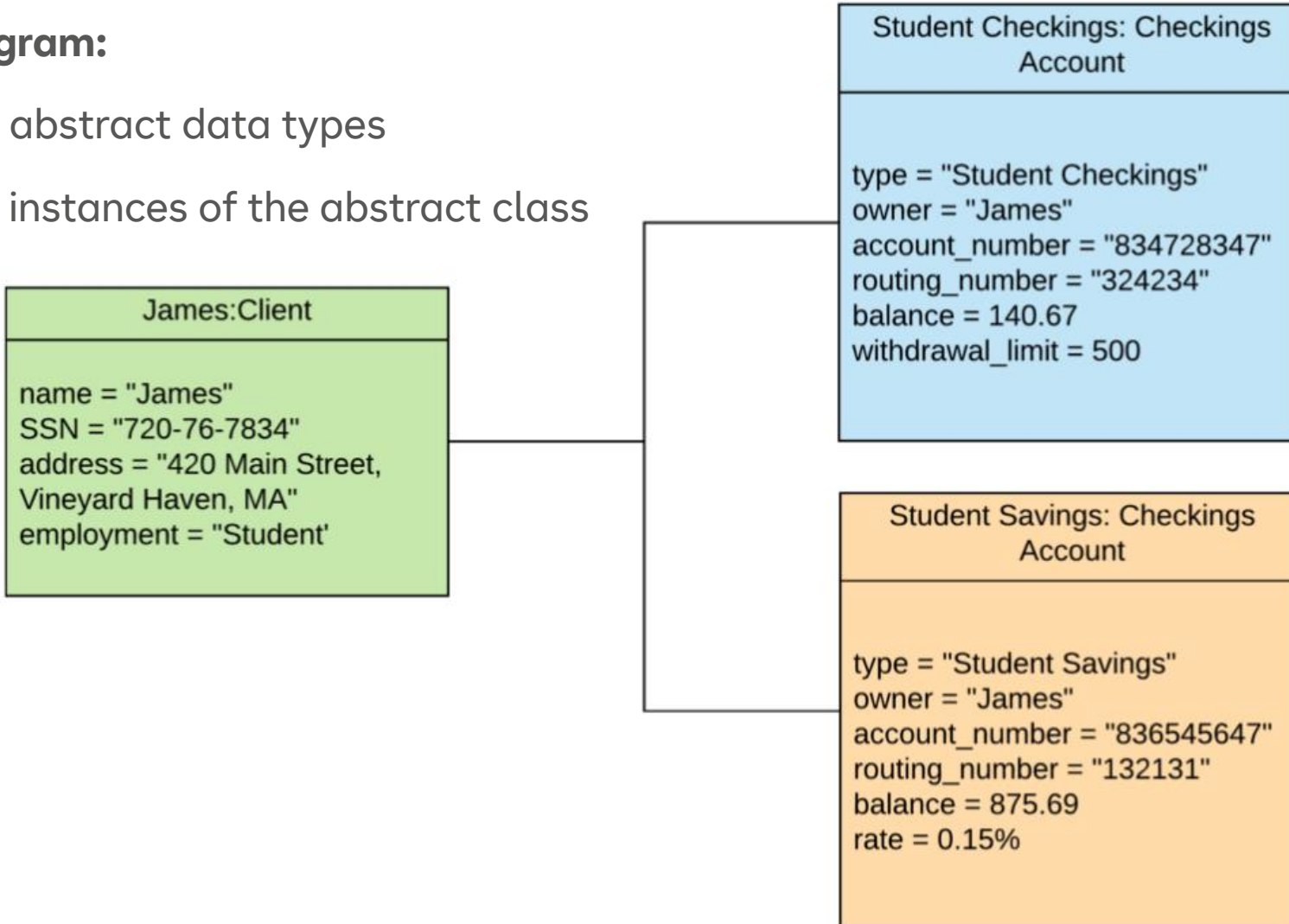
- *Class name, attributes and operations for each class*
- *How the classes interact*



# Examples of UML diagrams

## Object diagram:

- Classes: abstract data types
- Objects: instances of the abstract class



# Software Implementation

Some Coding Best Practices (absolutely not a comprehensive list...)

- Architecture and Design before starting to write code!
- Comment the code – imagine someone else will have to understand it. Use diagrams to describe design
- Test thoroughly the code and don't leave the testing to the 'end' (agile, test-driven development)
- If a piece of code gets repeated more than once, this is a sign that it should become a separate functionality available as a *module* in the *library*
- Modularity is key and it also enables easier re-architecting
- Appropriately manage dependencies
- Code performance: optimise for speed if needed
- Think about reducing the long-term maintenance cost (*technical debt* will cost later)
- Think about extensibility
- Document the code!

# Software Testing

## Code Verification

### Types of tests

Unit tests

Integration tests

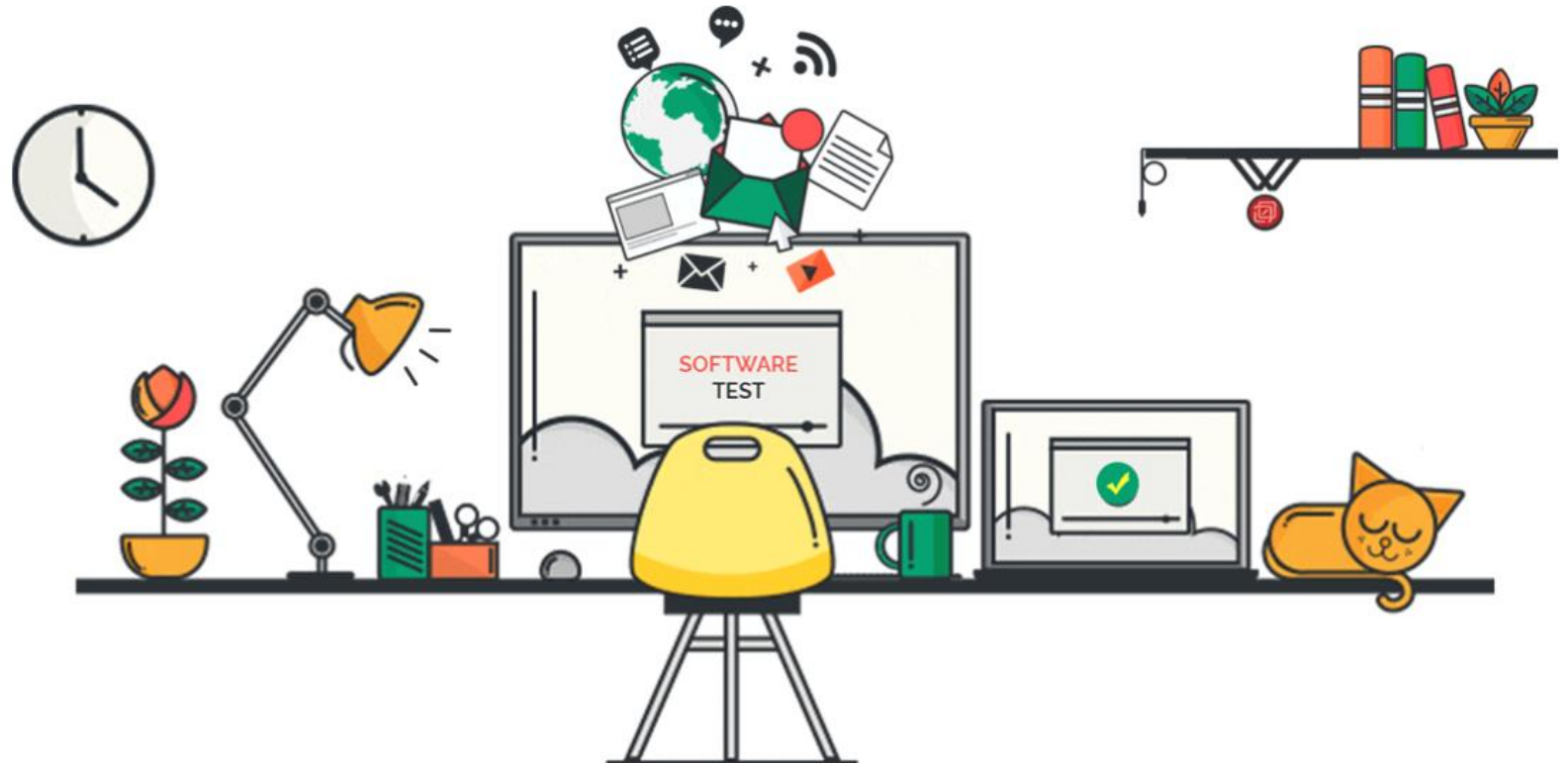
System-level tests

User Acceptance tests

...

### Quality assurance and test coverage

### Automated tests and test reports





# Releases

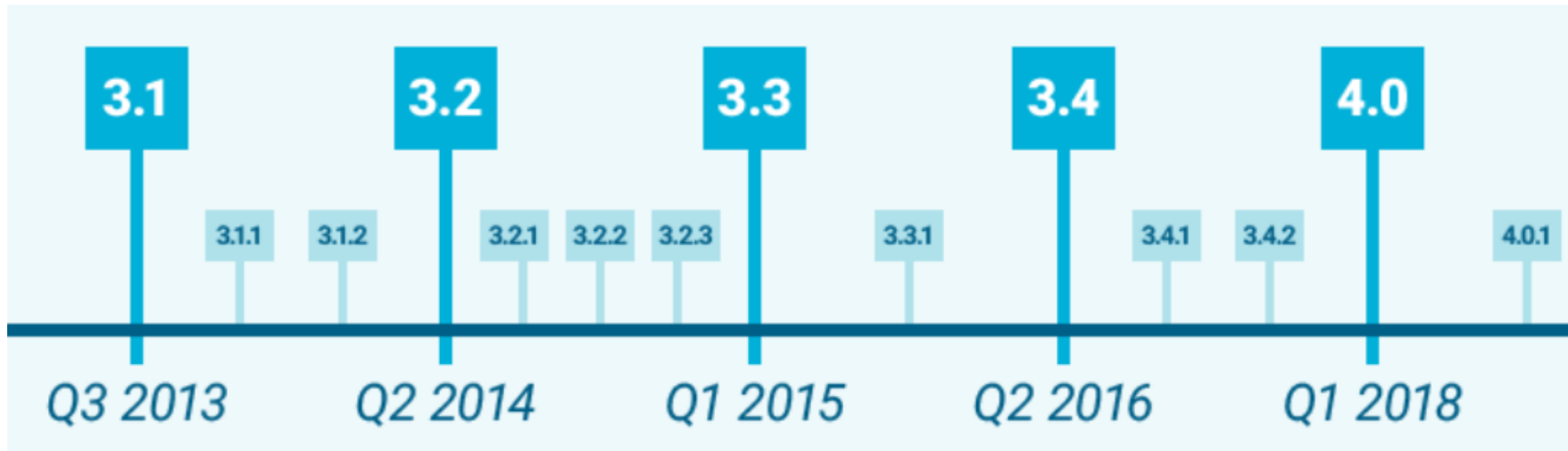
**Releases** of a software as it matures:

Dev, Trials, Prod Releases

Beta testing – testing by limited down-selected users

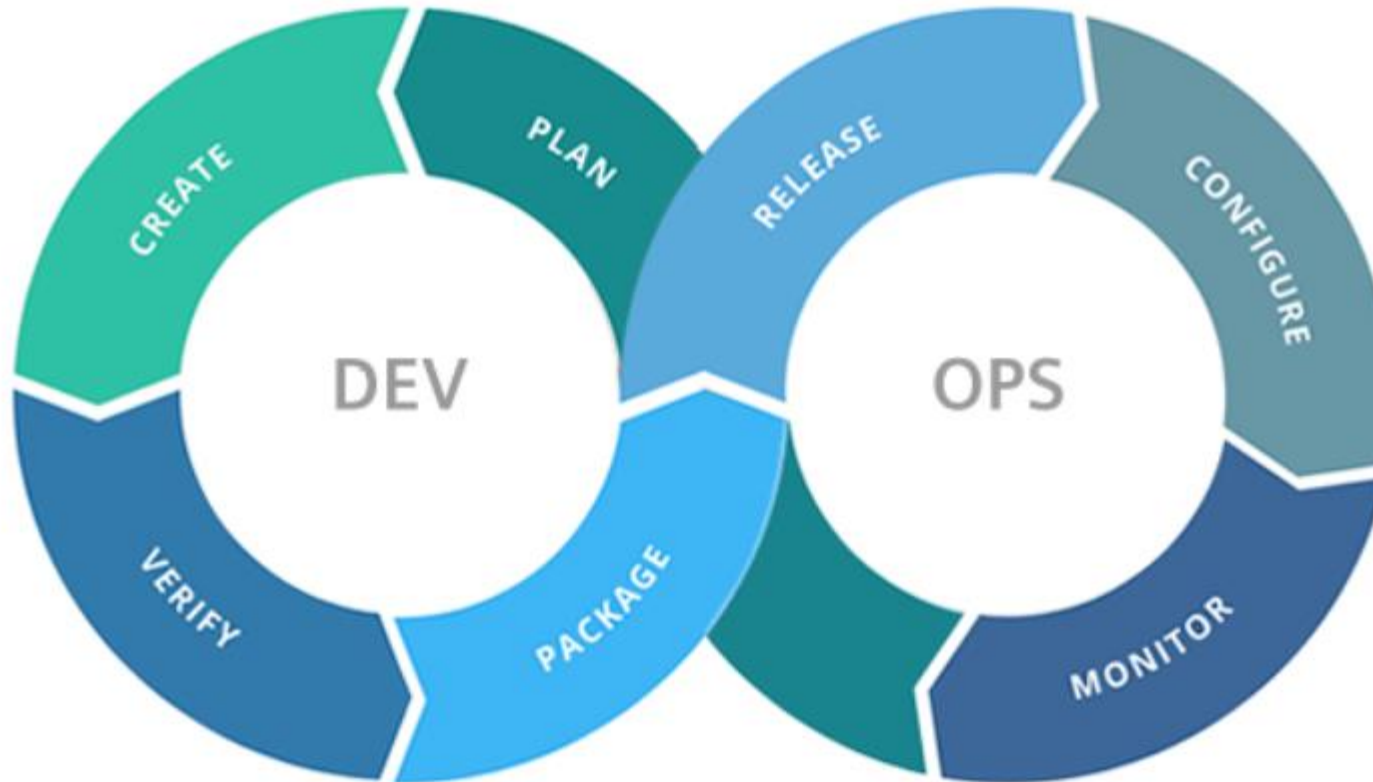
## Software versioning

E.g. *major.minor.bug\_fix*

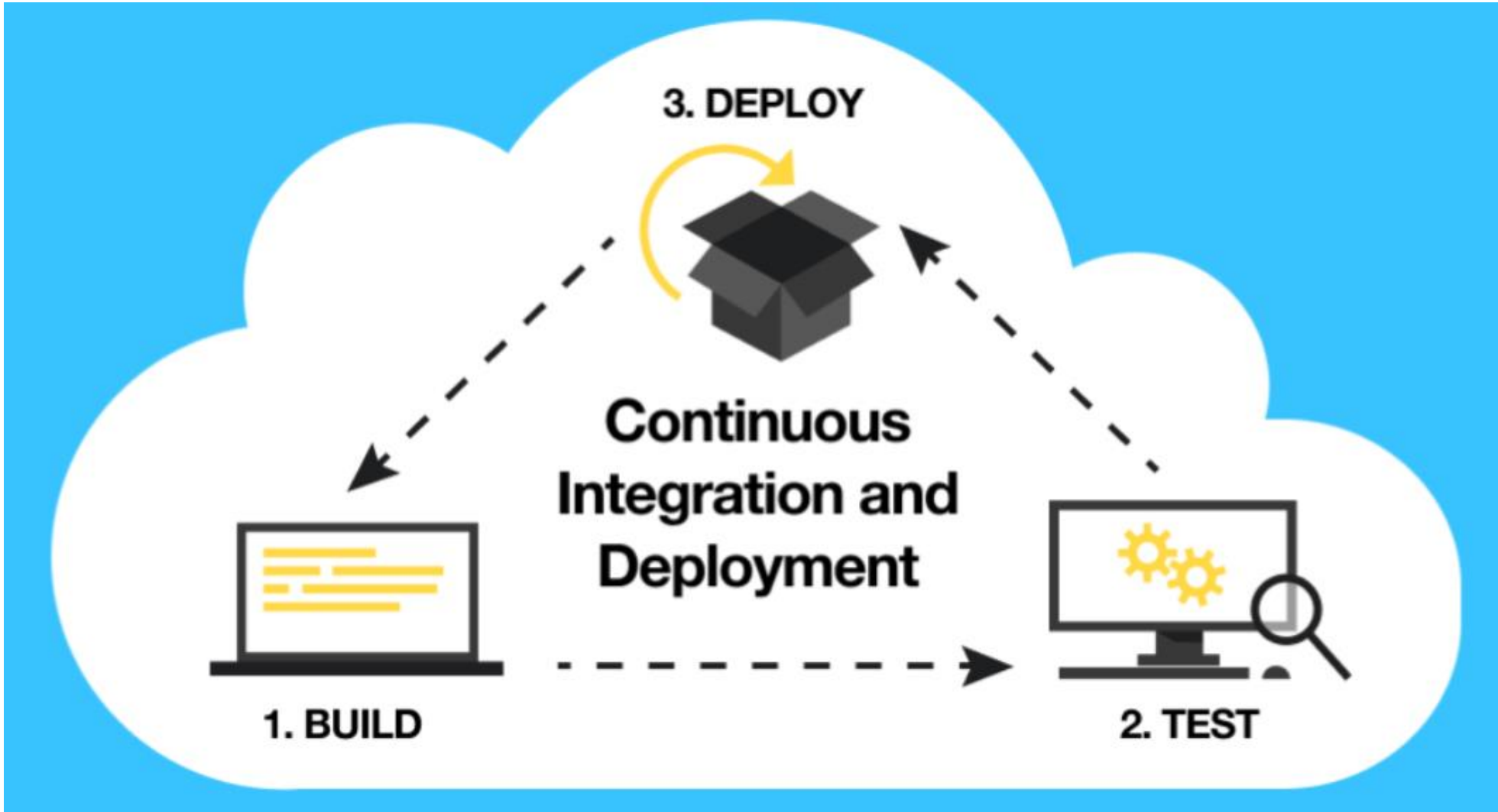




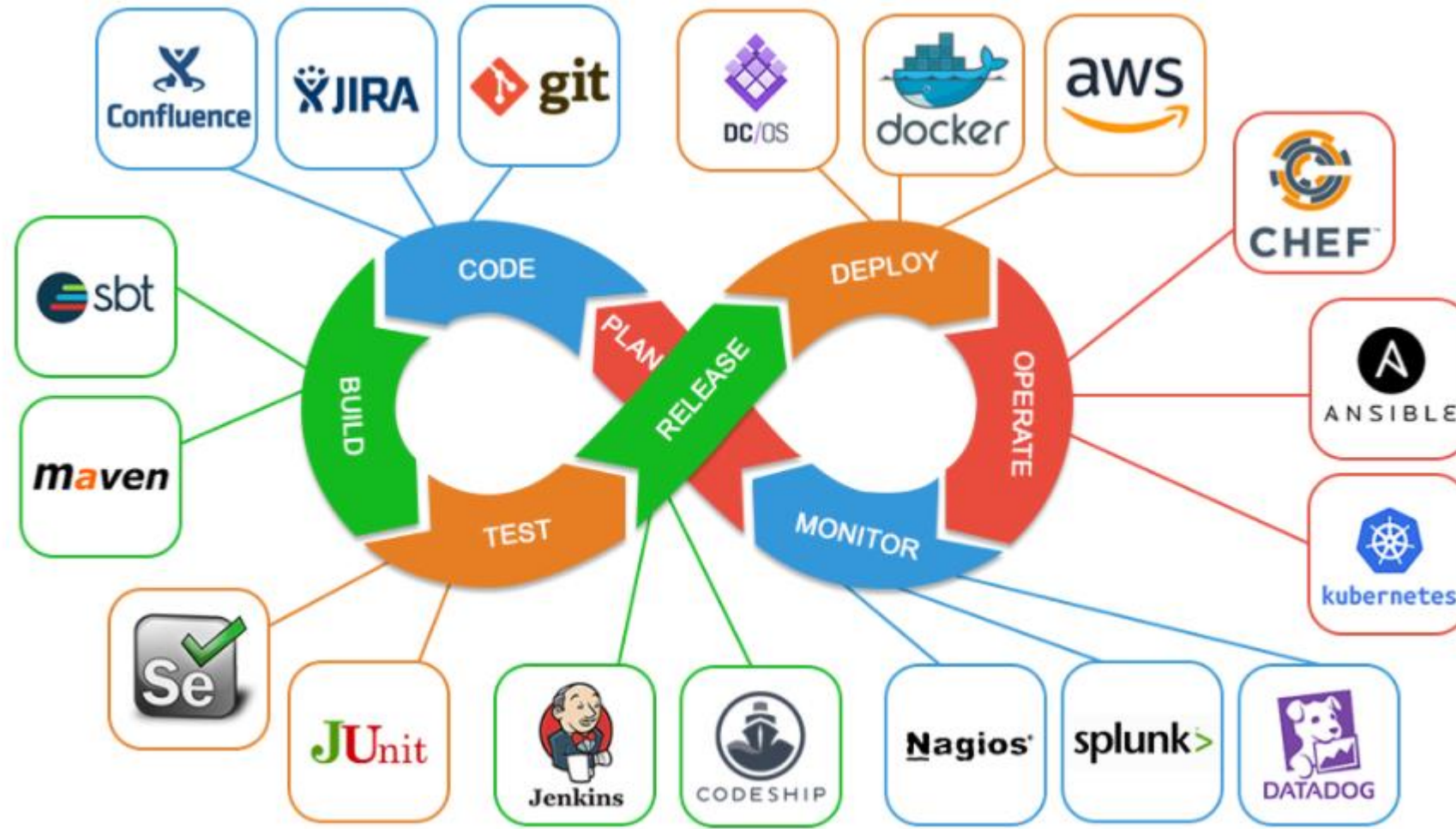
# DevOps – Development & Operations



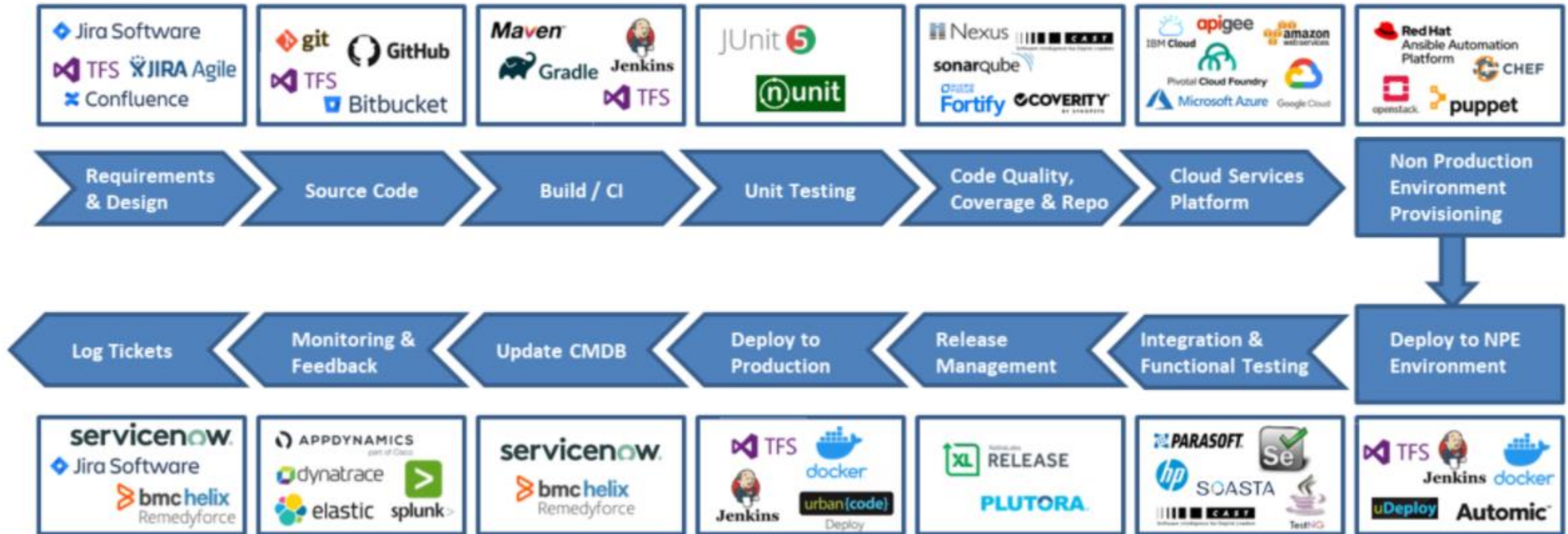
# Deployment



# Further reading and software landscape



# Further reading and software landscape



# Software version control: Git and GitHub

<https://docs.github.com/en/get-started/using-git/about-git>

A version control system, or VCS, tracks the history of changes as people and teams collaborate on projects together. As developers make changes to the project, any earlier version of the project can be recovered at any time.

A repository, or Git project, encompasses the entire collection of files and folders associated with a project, along with each file's revision history. The file history appears as snapshots in time called commits. The commits can be organized into multiple lines of development called branches. a Git repository also allows for: interaction with the history, cloning the repository, creating branches, committing, merging, comparing changes across versions of code, and more.

GitHub is a platform that hosts Git repositories. using the GitHub flow, developers simply create a branch to work on updates, commit changes to save them, open a pull request to propose and discuss changes, and merge pull requests once everyone is on the same page.

# Git and GitHub trainings

<https://docs.github.com/en/get-started/using-git/about-git>

<https://github.com/git-guides/install-git>

<https://github.com/>

<https://git-scm.com/docs/gittutorial>

<https://www.freecodecamp.org/news/what-is-git-and-how-to-use-it-c341b049ae61/>

<https://www.coursera.org/learn/introduction-git-github>



# Microsoft Azure Dev Ops

## Azure DevOps

<https://azure.microsoft.com/en-us/blog/introducing-azure-devops/>

Example of platform for backlog management, agile software development, software version control and configuration management, automated testing, build and release of software

<https://azure.microsoft.com/en-us/services/devops/#overview>

Linked to **Git**: free and open source distributed version control system designed to handle software development projects



### Azure Boards

Deliver value to your users faster using proven agile tools to plan, track, and discuss work across your teams.



### Azure Pipelines

Build, test, and deploy with CI/CD that works with any language, platform, and cloud. Connect to GitHub or any other Git provider and deploy continuously.



### Azure Repos

Get unlimited, cloud-hosted private Git repos and collaborate to build better code with pull requests and advanced file management.

# Multi-disciplinary Simulation

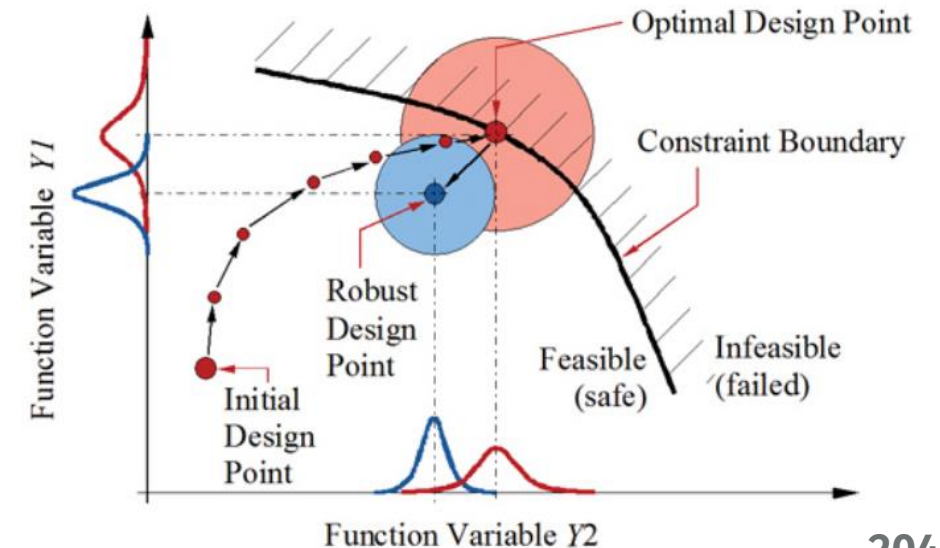
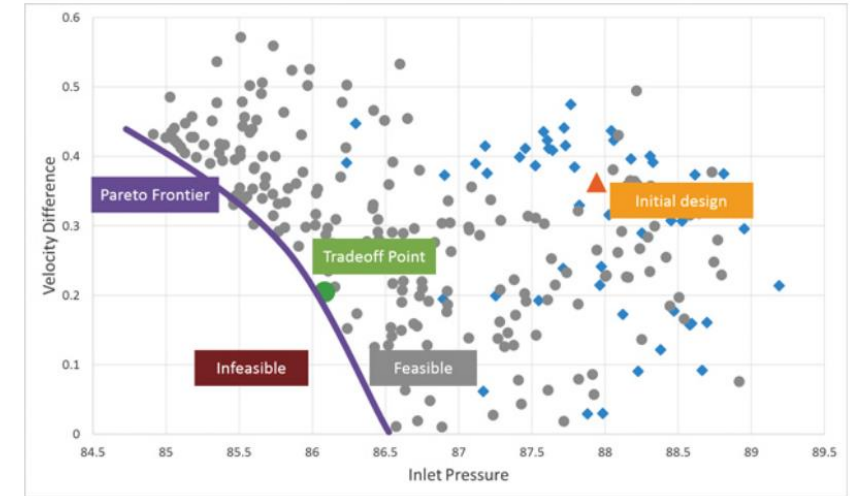
# 21



# Design Optimisation Software

## Design Space Exploration & Design Optimization Software Products

<https://www.digitalengineering247.com/article/design-optimization-past-present-and-future/>



# NASA OpenMDAO Tool

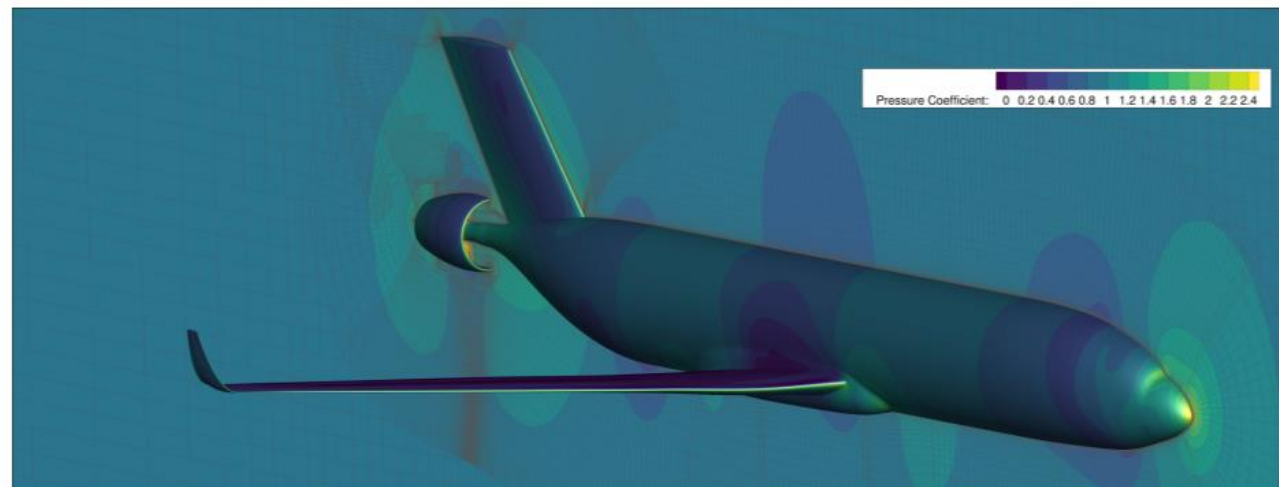
Open source framework for multidisciplinary optimization

<https://openmdao.org/>

[https://openmdao.org/newdocs/versions/latest/basic\\_user\\_guide/basic\\_user\\_guide.html#multidisciplinary-optimization](https://openmdao.org/newdocs/versions/latest/basic_user_guide/basic_user_guide.html#multidisciplinary-optimization)

<https://link.springer.com/content/pdf/10.1007/s00158-019-02211-z.pdf>

<https://ntrs.nasa.gov/api/citations/20100042271/download>  
[s/20100042271.pdf](https://ntrs.nasa.gov/api/citations/20100042271/download)



# Beyond simulation

# 22

# The digital twin

The **digital twin** is the digital representation of a physical product throughout its lifecycle: the **virtual product**.

DESIGN &  
ANALYSIS

MANUFACTURE

BUILD &  
ASSEMBLY

EXPERIMENTAL  
TESTING

SERVICE

*Key steps of an engineering product lifecycle*

At each step of the product lifecycle, the digital twin offers the opportunity to optimise and de-risk the real product, simulating in the computer the product behaviour for specific operating conditions.

Feeding data from the real product operation into a system-level simulation model, it is possible to generate a different virtual model for each real product (the *digital twin*). This enables optimisation of the key performance attributes of each product and predictions on future operation (e.g. performance degradation)

Given that the cost of simulation is << than the cost of experimental testing, the virtual product allows further de-risking of the real product operation. The digital twin does not replace experimental testing.

# The virtual product and the real product

## THE VIRTUAL PRODUCT

DESIGN &  
ANALYSIS



MANUFACTURE



BUILD &  
ASSEMBLY



EXPERIMENTAL  
TESTING



SERVICE

Product as  
designed in  
engineering  
drawings



Manufacturing  
variations



Assembly &  
build tolerances



Aspects of  
physics not  
captured by  
simulation



Real product  
operating  
conditions

## THE REAL PRODUCT

The product lifecycle introduces multiple variations, it is crucial for the **digital twin** to capture as many **real product attributes** as possible, in order to provide the **most sophisticated digital version of the reality**.

# Simulation at each step of the product lifecycle

## THE VIRTUAL PRODUCT

**DESIGN &  
ANALYSIS**

**MANUFACTURE**

**BUILD &  
ASSEMBLY**

**EXPERIMENTAL  
TESTING**

**SERVICE**

Product as  
designed in  
engineering  
drawings

Manufacturing  
variations

Assembly &  
build tolerances

Aspects of  
physics not  
captured by  
simulation

Real product  
operating  
conditions

Simulate  
nominal (as  
designed)  
product

Simulate  
product as  
manufactured

Simulate  
product as built

Calibrate  
simulation  
models

Feed product  
operation data  
into simulation  
models

# Not only simulation

## THE VIRTUAL PRODUCT

**DESIGN &  
ANALYSIS**

**MANUFACTURE**

**BUILD &  
ASSEMBLY**

**EXPERIMENTAL  
TESTING**

**SERVICE**

Product as  
designed in  
engineering  
drawings

Manufacturing  
variations

Assembly &  
build tolerances

Aspects of  
physics not  
captured by  
simulation

Real product  
operating  
conditions

**ENABLERS**

Measurement  
systems &  
Manufacturing  
processes

Build &  
assembly  
procedures

Sensors & IT for  
experimental &  
operations data

Robotics for  
self-  
maintenance

# The digital twin

## A multi-disciplinary problem

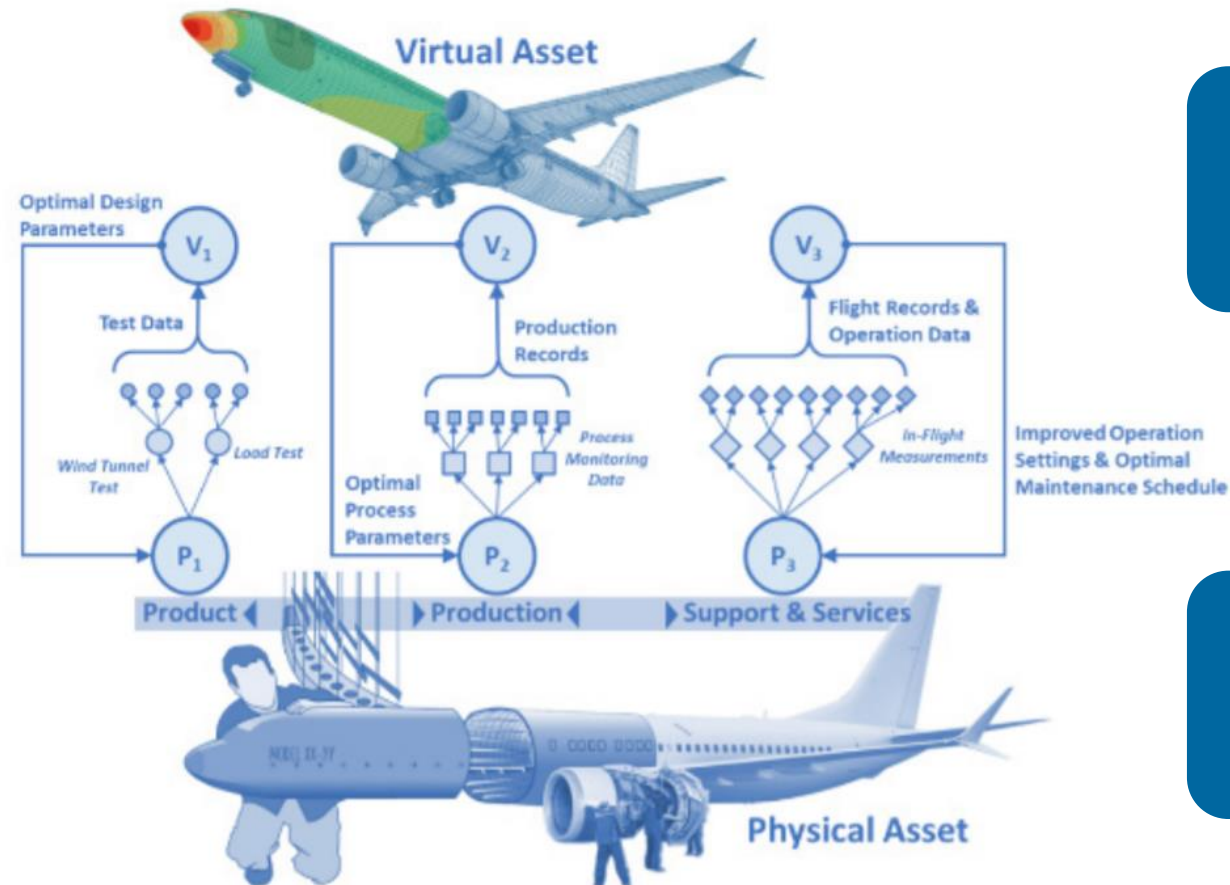
[https://www.aiaa.org/docs/default-source/uploadedfiles/issues-and-advocacy/policy-papers/digital-twin-institute-position-paper-\(december-2020\).pdf](https://www.aiaa.org/docs/default-source/uploadedfiles/issues-and-advocacy/policy-papers/digital-twin-institute-position-paper-(december-2020).pdf)

Simulation

Sensors & IT  
infrastructure

Measurement  
systems,  
manufacture,  
build & assembly

Robotics





# The digital twin

*A hard multi-disciplinary technical problem of great potential across industry*

Siemens

<https://www.youtube.com/watch?v=ObGhB9CCHP8>

Digital twin cities – ASME report

<https://www.youtube.com/watch?v=HftDI09LVI0>

General Electric – Power digital solutions

<https://www.youtube.com/watch?v=5Ji8dZqkECM>

*Every engineering sector has a great interest in making digital twins work end to end*

# Vielen Dank für Ihre Aufmerksamkeit!

**Kontakt:**

Adele Nasti  
SRH Berlin  
[adele.nasti@srh.de](mailto:adele.nasti@srh.de)  
[info@srh.de](mailto:info@srh.de)  
[www.srh.de](http://www.srh.de)